

Neural networks basics

by C. Yao Lai for 2026 Glamacles summer school

We would love to know what you're working on that can be added to the list!

ML for cryosphere observations

Data creation (continent-wide dataset marked by *):

- Icebergs (Barbat et al. 2019*, Rezvanbehbahani, Stearns et al. 2020)
- Sea ice (Cooke et al. 2019, Landy et al. 2022*)
- Calving front /termini (Baumhoer et al. 2019, Mohajerani et al. 2019, Zhang et al. 2019, CALFIN: Cheng et al. 2021, AutoTerm: Zhang et al. 2023*...etc)
- Fractures (Ice-shelf: Lai et al. 2020*, Ice sheet: Chudley et al. 2021, Ice shelf + sheet: Surawy-Stepney et al. 2023...etc)
- Bed topography (DeepBedMap: Leong & Horgan 2020*)
- Internal ice layers using radar data (Rahnemoonfar et al. 2021)
- Grounding lines using InSAR data (Mohajerani et al. 2021)
- Greenland firn aquifers (Shang et al. 2022*)
- Blue ice- Antarctica (Tollenaar et al. 2024*)
- Supraglacial water – Antarctica (Dirscherl et al. 2020, Dell et al. 2021, 2024, Baumhoer et al. 2026)
- Supraglacial water – Greenland (Dunmire et al. 2021*, 2025*, Ryan et al. 2026)

We would love to know what you're working on that can be added to the list!

ML for cryosphere modeling

- **Surrogate/emulation model (forward modeling):**

NN (Tarasov et al. 2012), GP (Edwards et al. 2021)

CNN (IGM: Jouvét et al. 2021, 2023a,b, Verjans & Robel 2024)

GNN (Rahnemoonfar and Koo 2025, Koo, Cheng et al. 2025, Propp et al. arxiv, FloeNet: Gregory et al. arxiv)

DeepONet (Howard, Perego et al. 2023, He, Perego et al. 2023)

LSTM (Katwyk et al. 2023)

- **Data assimilation (inverse modeling):**

Bayesian inference + NN (Brinkerhoff et al. 2021)

PINN (Riel, Minchew et al. 2021, Diffice: Wang, Lai et al. 2022, 2025, PINNICLE: Cheng et al. 2024, 2025, Steidl, Bamber et al. 2025, Krishna et al. 2026)

GP (Downs et al. 2022, Hill et al. 2025a,b, Brinkerhoff et al. 2025)

UDE (ODINN: Bolibar, Sapienza et al. 2023)

CNN (DeepBedMap: Leong & Horgan 2020, Jouvét 2023a,b, DeepTopoNet: Tama et al. 2025)

- **Downscaling**

Generative models (Bochow et al. 2026)

Outline

- Why neural networks?
- NN basics:
 - Universal function approximation
 - Gradient descent
 - Back propagation
- Advanced topics: challenges of using NN in scientific problems

So...why neural networks?

- Very often in science we want to know A as a function of B . NN has the flexibility to fit that function without prior assumptions about the functional form, particularly useful when the function is high dimensional (it is a universal function approximator).
- Leverages GPU (hardware) and open-source software (e.g. tensorflow, Pytorch, JAX). The optimization method is well-developed and user friendly. It can usually be easily adopted for different problems.
- We will talk about the challenges after learning about the basics of NN!

Definitions that may be useful

Model: A representation of data to make predictions. Models can be ML-based, physics-based, or coupled.

Simulation: Physics-based models solved numerically. For example the General Circulation Model is a climate simulation that solves the Navier-Stokes equation...etc.

Emulator: In this article emulator refers to a subset of models that fit the data, bypassing solving physics-based equations.

Definitions that may be useful

Machine Learning for Climate Physics and Simulations

Ching-Yao Lai,¹ Pedram Hassanzadeh,² Aditi Sheshadri³, Maike Sonnewald⁴, Raffaele Ferrari⁵, Venkatramani Balaji⁶

¹Department of Geophysics, Stanford University, Stanford, CA 94305, USA;
email: cyaolai@stanford.edu

²Department of Geophysical Sciences and Committee on Computational and Applied Mathematics, University of Chicago, Chicago, IL 60637, USA

³Department of Earth System Sciences, Stanford University, Stanford, CA 94305, USA

⁴Department of Computer Science, University of California Davis, Davis, CA 95616, USA

⁵Department of Earth, Atmospheric, and Planetary Sciences, Massachusetts Institute of Technology, Cambridge, MA 02139, USA

⁶Schmidt Sciences, USA

Lai et al., Annu. Rev. Condens. Matter Phys. (2025). doi.org/10.1146/annurev-conmatphys-043024-114758

Simulation:

Physics-based models solved numerically. For example the General Circulation Model is a climate simulation that solves the Navier-Stokes equation...etc.

Emulator: In this article emulator refers to a subset of models that fit the data, bypassing solving physics-based equations.

Definitions that may be useful

A nonlinear partial differential equation:

$$\frac{\partial u(x, t)}{\partial t} = N(u(x, t), \theta(x, t))$$

Forward problem:

Given initial/boundary conditions, model parameter $\theta(x, t)$, find $u(x, t)$

Inverse problem:

Given $u(x, t)$, find model parameter $\theta(x, t)$

ML methods in modeling physical systems

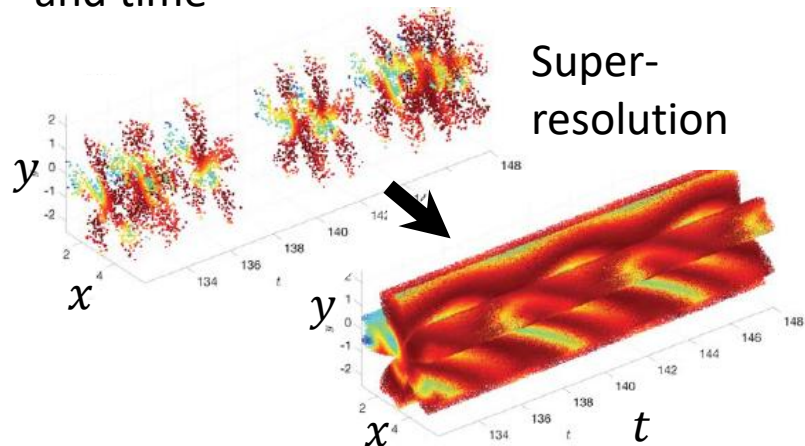
- Inverse problems

Denoise and interpolate **Sparse Data**.
PDE-constrained optimization

e.g. **Physics-informed neural networks** (Raissi et al. 2019),
Generative models

Given: sparse data in space and time

Predict: dense, continuous data in space and time



- Emulation/surrogate

Discover a model **parameterized by ML** that best fits a given dataset.

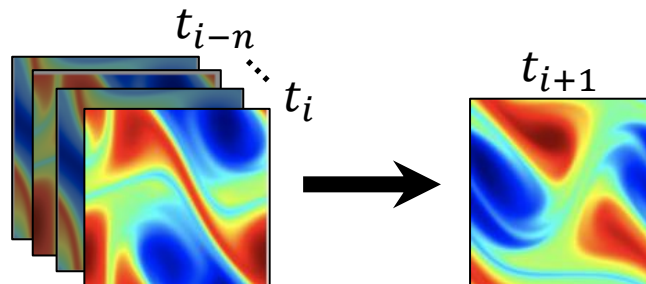
e.g. **Fourier Neural Operator** (Li et al 2021); **DeepONet** (Lu et al 2021), **NN**, **Generative models**

Given: data from previous time steps

$x(t_i), \dots, x(t_{i-n})$

Predict: future time steps

$x(t_{i+1}), \dots, x(t_{i+n})$



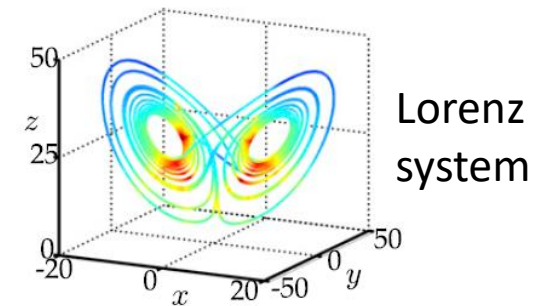
Emulating fluid simulation

- Symbolic regression

Discover a **mathematical expression** that best fits a given dataset.

e.g. **SINDy** (Brunton et al. 2016),
PDEFind (Rudy et al., 2017)

Given: $\dot{x}(t), \dot{y}(t), \dot{z}(t)$ at lots of times



Lorenz system

Predict:
Analytical model

$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= x(\rho - z) - y \\ \dot{z} &= xy - \beta z.\end{aligned}$$

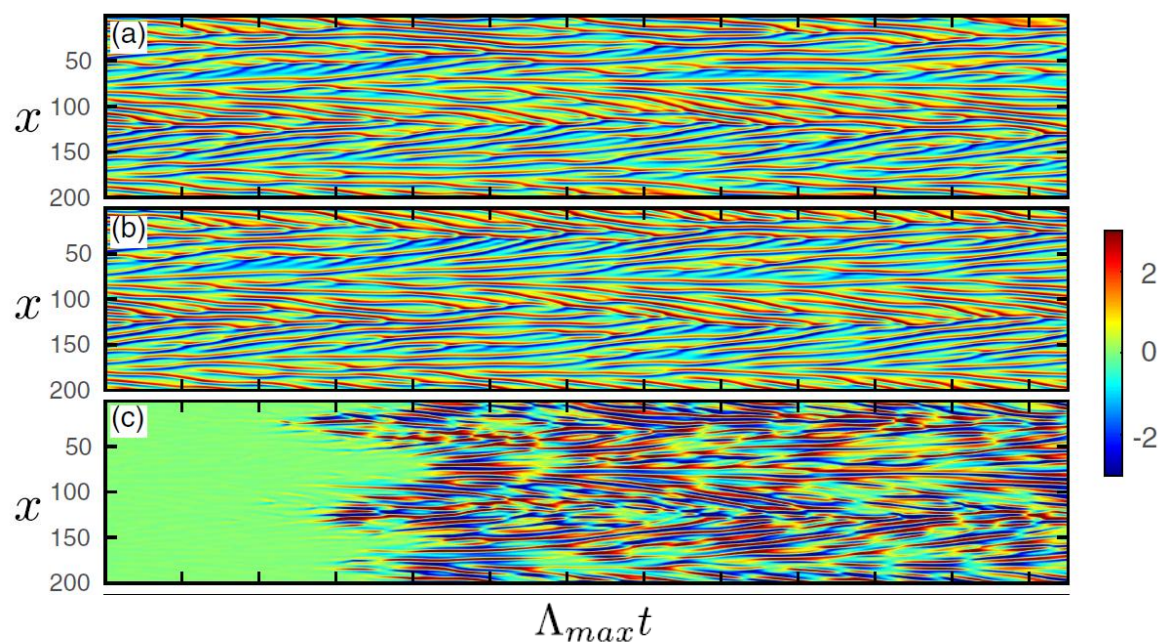
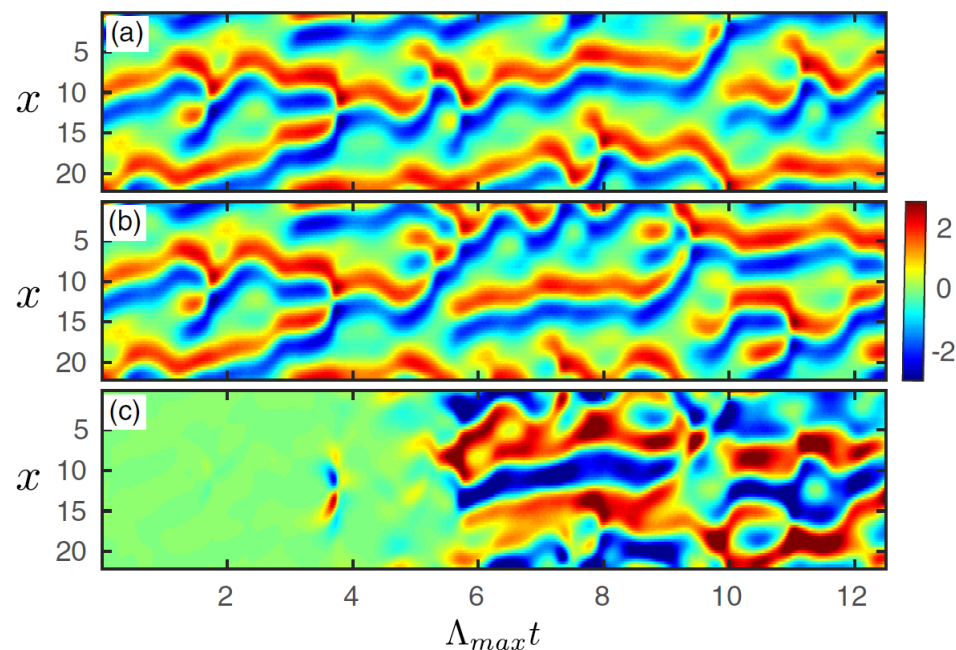
Surrogate/emulation (forward in time)

, 024102 (2018)

$$\frac{\partial \mathbf{z}(x, t)}{\partial t} = \mathbf{f}(x, t), \quad \text{where } x \in \mathbb{R}^d \quad \text{and} \quad \mathbf{z}, \mathbf{f} \in \mathbb{R}^n,$$

Chaotic Systems from Data: A
broach

Find θ such that $\text{ML}(\theta) = \mathbf{f}(x, t)$, mapping the rate of change of states $\mathbf{z}(x, t)$ (ML: $\mathbf{z}(x, t_i) \mapsto \mathbf{z}(x, t_{i+1})$)



Physics-based simulations of weather

Model of weather:

$$\frac{\partial \mathbf{z}(x, t)}{\partial t} = \mathbf{f}(x, t), \quad \text{where } x \in \mathbb{R}^d \quad \text{and} \quad \mathbf{z}, \mathbf{f} \in \mathbb{R}^n,$$

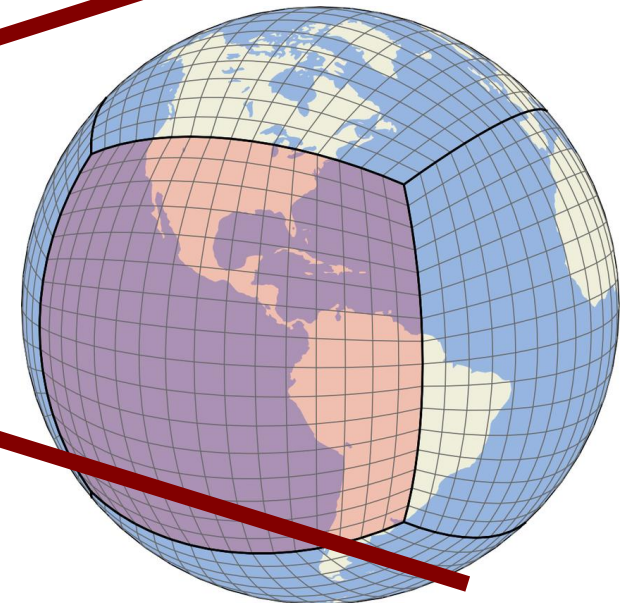
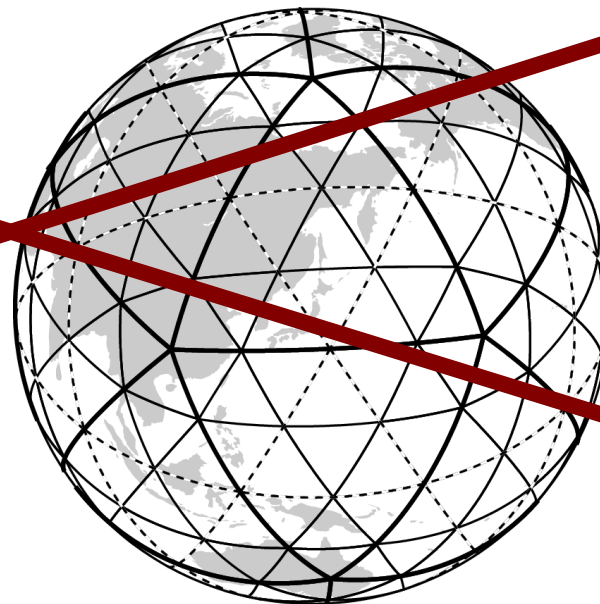
Governing Equations in physics-based simulations of weather prediction model

$$\frac{D\vec{u}}{Dt} + 2\Omega \times \vec{u} = -\frac{1}{\rho} \nabla p - \nabla \Phi + \vec{F} \quad (\text{Momentum})$$

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \vec{u}) = 0 \quad (\text{Mass})$$

$$p = \rho R T \quad (\text{Ideal Gas Law})$$

$$c_p \frac{DT}{Dt} - \frac{1}{\rho} \frac{Dp}{Dt} = \dot{Q} \quad (\text{Energy})$$



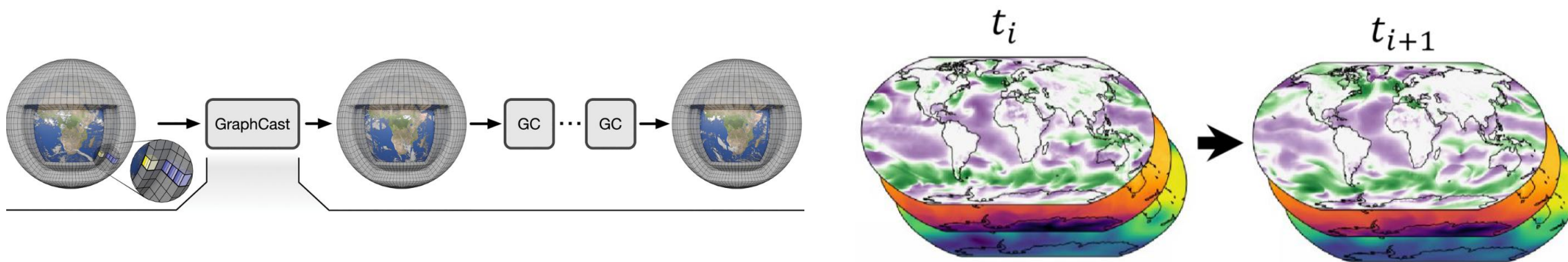
Bauer, P., Thorpe, A. & Brunet, G. The quiet revolution of numerical weather prediction. *Nature* **525**, 47–55 (2015).

Surrogate/emulation (forward in time)

Model of weather:

$$\frac{\partial \mathbf{z}(x, t)}{\partial t} = f(x, t), \quad \text{where } x \in \mathbb{R}^d \quad \text{and} \quad \mathbf{z}, f \in \mathbb{R}^n,$$

Find θ such that $\text{ML}(\theta) = f(x, t)$, mapping the rate of change of states $\mathbf{z}(x, t)$ (ML: $\mathbf{z}(x, t_i) \mapsto \mathbf{z}(x, t_{i+1})$)



Deepmind GraphCast model (Lam et al, Science 2023) trained a graph neural network on physics-based simulations of weather prediction model to learn the mapping of states of the global atmosphere from t_i to t_{i+1} , then from t_{i+1} to t_{i+2} , and so on.

ML weather forecast models

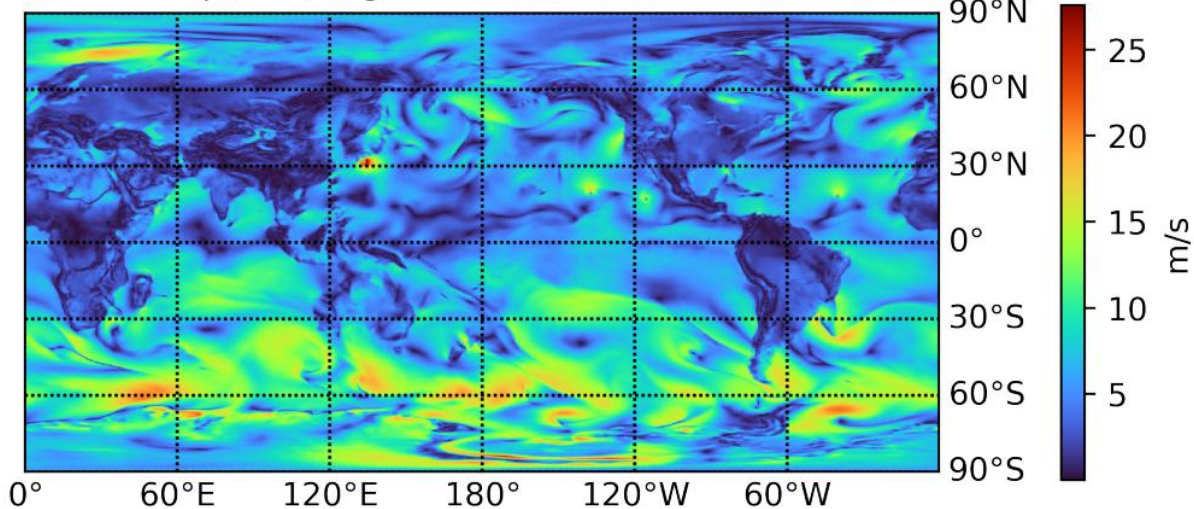
ECMWF is now running a series of data-driven forecasts as part of its experimental suite. These machine-learning based models are very fast, and they produce a 10-day forecast with 6-hourly time steps in approximately one minute. The outputs are available in graphical form.

Neural networks are used to emulate the weather
(Pangu-Weather: Bi et al. (2023), Nature)

Which one is real?

A

10m Wind Speed, Pangu-Weather, Forecast Time: 72 hours

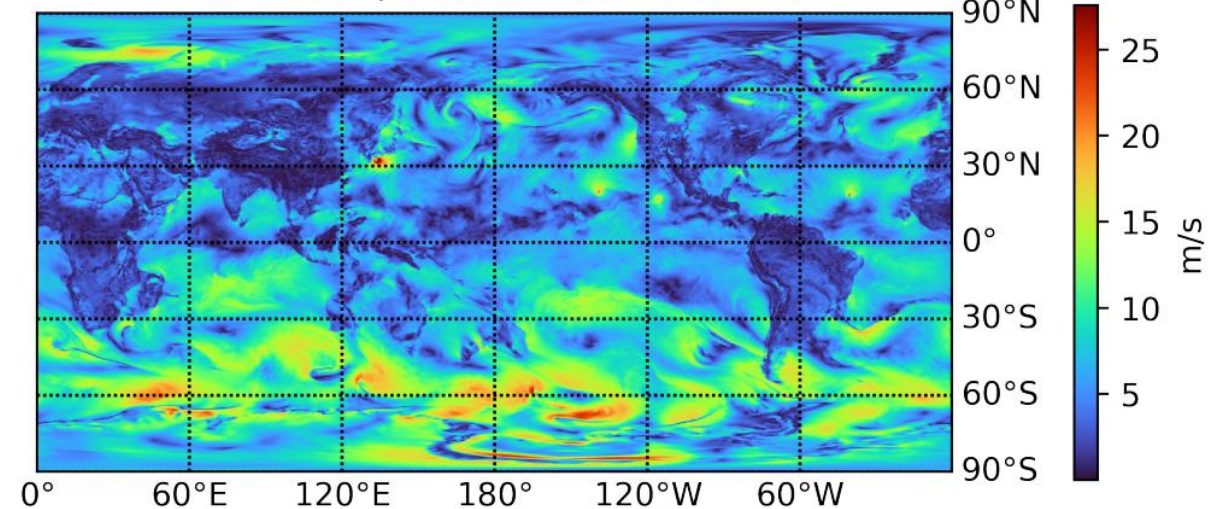


AI weather forecast

Bi et al (2023) Nature

B

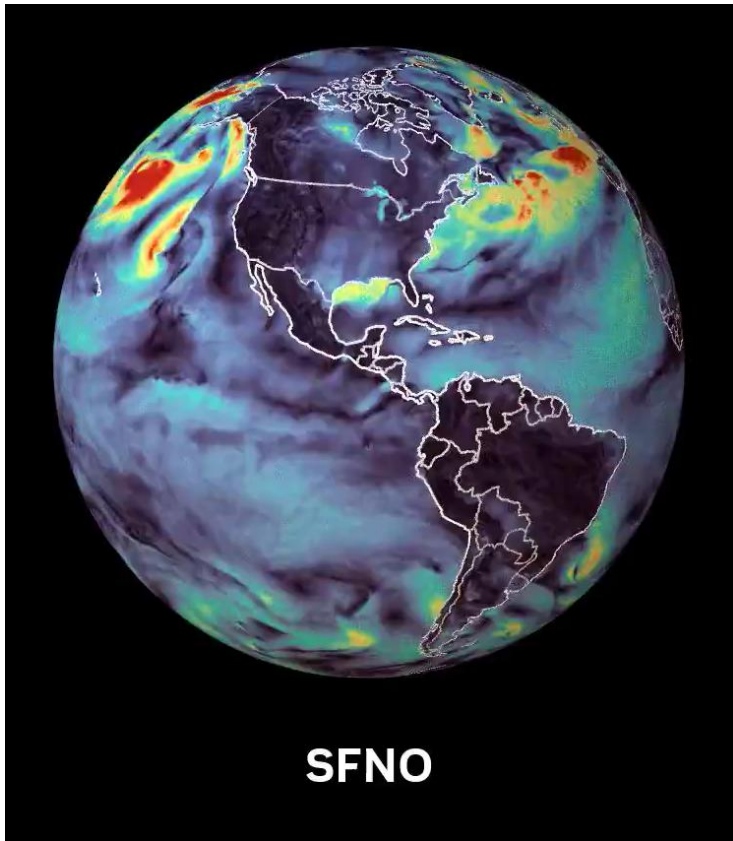
10m Wind Speed, ERA5 (Ground-Truth)



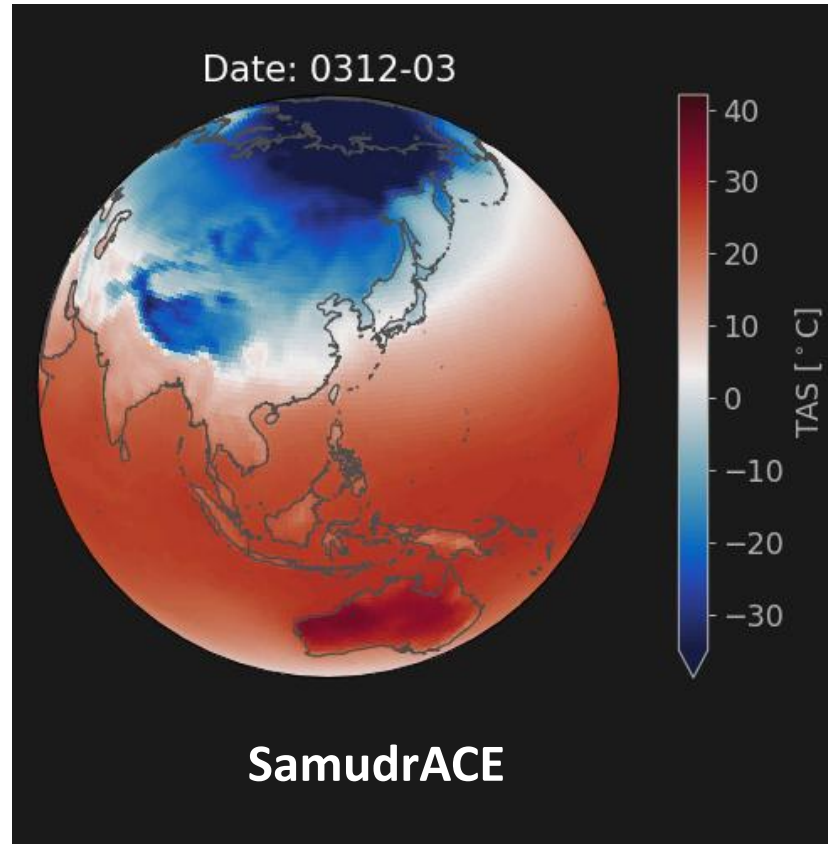
Training data

(Numerical weather prediction simulation)

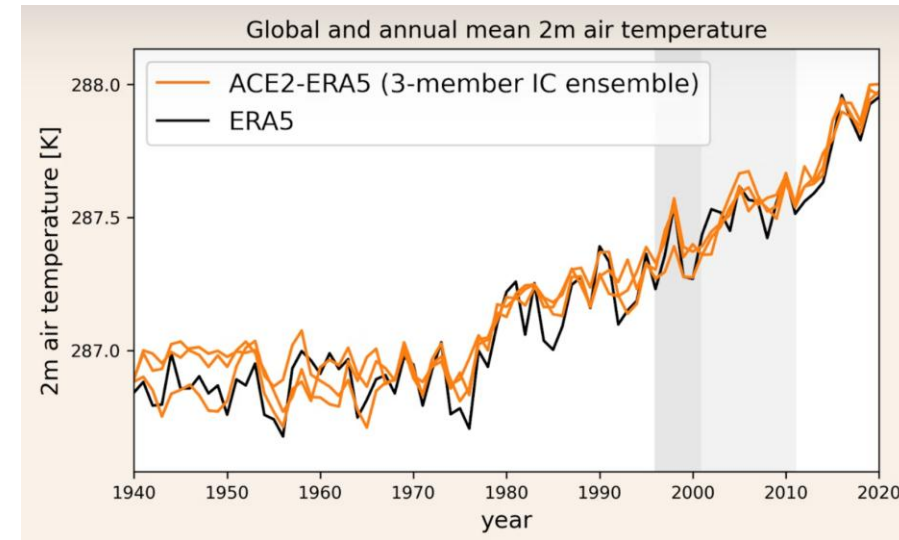
Stable rollout for 60 days



FourCastNet3: Bonev et al., arxiv



SamudrACE: Duncan et al., arXiv



ACE2: Watt-Meyer et al. (2025)

ML for cryosphere modeling

- **Surrogate/emulation model (forward modeling):**

NN (Tarasov et al. 2012), GP (Edwards et al. 2021)

CNN (IGM: Jouvét et al. 2021, 2023a,b, Verjans & Robel 2024)

GNN (Rahnemoonfar and Koo 2025, Koo, Cheng et al. 2025, Propp et al. arxiv, FloeNet: Gregory et al. arxiv)

DeepONet (Howard, Perego et al. 2023, He, Perego et al. 2023)

LSTM (Katwyk et al. 2023)

- **Data assimilation (inverse modeling):**

Bayesian inference + NN (Brinkerhoff et al. 2021)

PINN (Riel, Minchew et al. 2021, Diffice: Wang, Lai et al. 2022, 2025, PINNICLE: Cheng et al. 2024, 2025, Steidl, Bamber et al. 2025, Krishna et al. 2026)

GP (Downs et al. 2022, Hill et al. 2025a,b, Brinkerhoff et al. 2025)

UDE (ODINN: Bolibar, Sapienza et al. 2023)

CNN (DeepBedMap: Leong & Horgan 2020, Jouvét 2023a,b, DeepTopoNet: Tama et al. 2025)

- **Downscaling**

Generative models (Bochow et al. 2026)

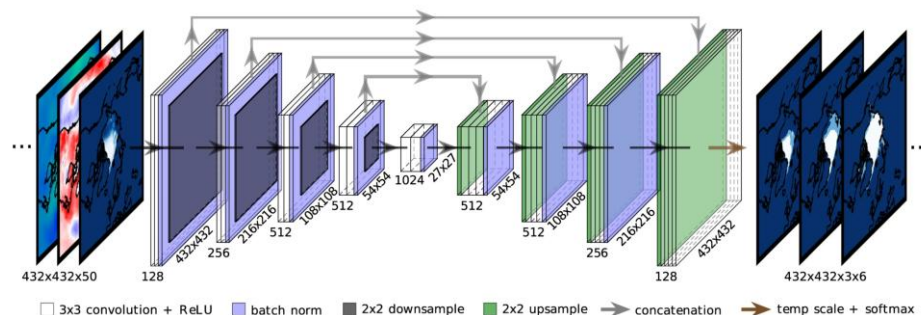
Surrogate/emulation (forward in time)

- CNN learns the **input-output** relationship from lots of training data.
- Input = ? Output = ?

Seasonal Arctic sea ice forecasting with probabilistic deep learning

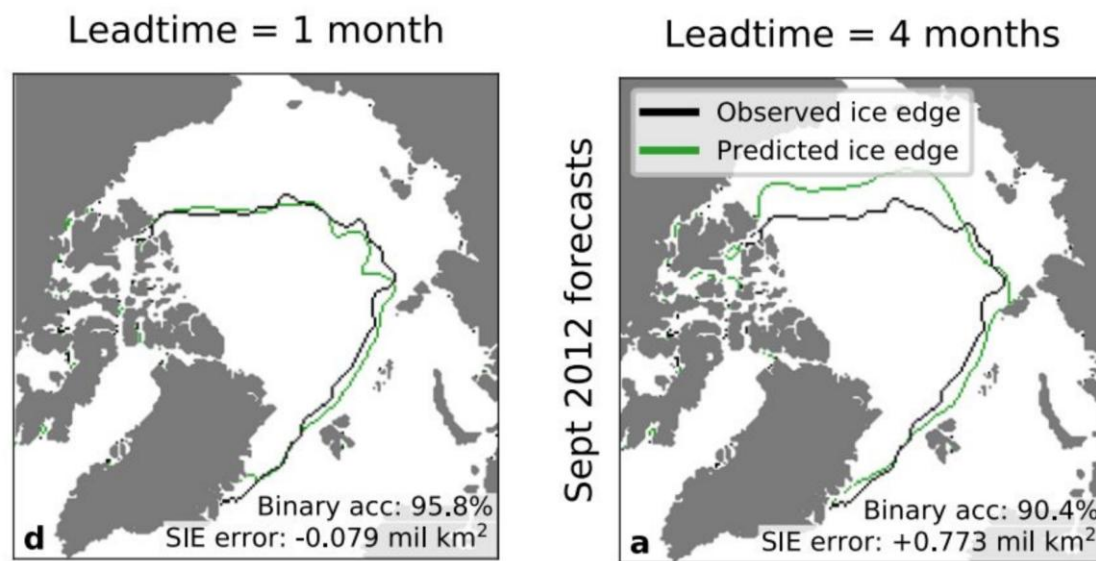
Tom R. Andersson , J. Scott Hosking, María Pérez-Ortiz, Brooks Paige, Andrew Elliott, Chris Russell, Stephen Law, Daniel C. Jones, Jeremy Wilkinson, Tony Phillips, James Byrne, Steffen Tietsche, Beena Balan Sarojini, Eduardo Blanchard-Wrigglesworth, Yevgeny Aksenov, Rod Downie & Emily Shuckburgh

Nature Communications **12**, Article number: 5124 (2021) | [Cite this article](#)



Input: 50 different climate variables (e.g. temperature, wind speed...etc)

Output: 3 sea ice concentration classes in 6 forecast months



Surrogate/emulation (between variables)

Model of glaciers:

Mass balance:
$$\frac{\partial h}{\partial t} + \nabla \cdot (\bar{\mathbf{u}}h) = \text{SMB}$$

Momentum balance:

$$\begin{aligned} \partial_x(4h\eta\partial_x u + 2h\eta\partial_y v) + \partial_y(h\eta(\partial_x v + \partial_y u)) - \beta^2 u &= \rho gh(\partial_x s \cos \alpha - \sin \alpha), \\ \partial_y(4h\eta\partial_y v + 2h\eta\partial_x u) + \partial_x(h\eta(\partial_y u + \partial_x v)) - \beta^2 v &= \rho gh\partial_y s \cos \alpha. \end{aligned}$$

Expensive!

$$\beta = c^{-1/2m} |\mathbf{v}_b|^{\frac{1-m}{2m}}$$

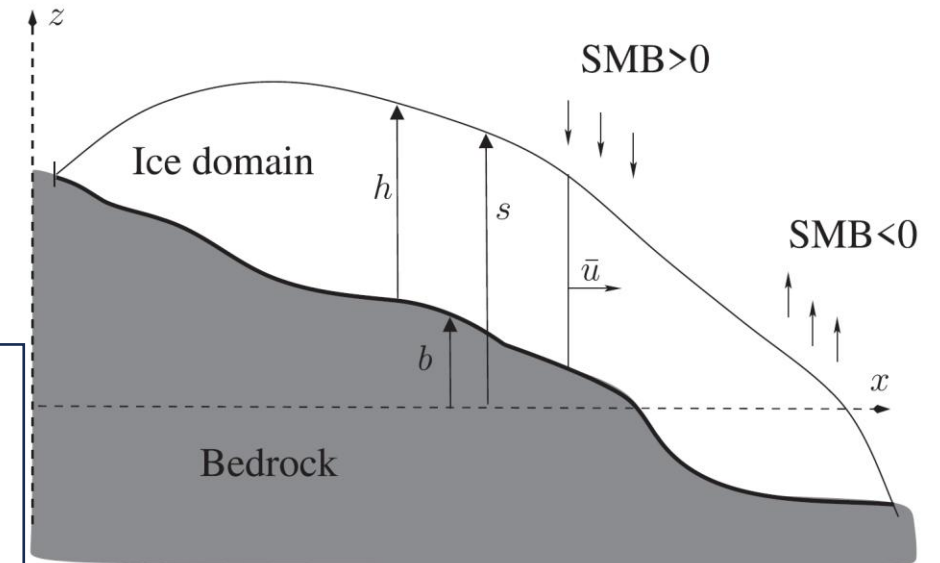


Fig. 2. Cross-section of a glacier with notations.

$$\begin{aligned} \mathcal{M}: \quad & \left\{ h, \frac{\partial s}{\partial x}, \frac{\partial s}{\partial y}, c \right\} \rightarrow \{\bar{u}, \bar{v}\} \\ & \mathbb{R}^{N_X \times N_Y \times 4} \rightarrow \mathbb{R}^{N_X \times N_Y \times 2} \end{aligned}$$

Surrogate/emulation (between variables)

Model of glaciers:

Mass balance:
$$\frac{\partial h}{\partial t} + \nabla \cdot (\bar{\mathbf{u}}h) = \text{SMB}$$

Momentum balance (2D; in x,y):

$$\begin{aligned} \partial_x(4h\eta\partial_x u + 2h\eta\partial_y v) + \partial_y(h\eta(\partial_x v + \partial_y u)) - \beta^2 u &= \rho gh(\partial_x s \cos \alpha - \\ \partial_y(4h\eta\partial_y v + 2h\eta\partial_x u) + \partial_x(h\eta(\partial_y u + \partial_x v)) - \beta^2 v &= \rho gh\partial_y s \cos \alpha. \end{aligned}$$

Expensive!

$$\beta = c^{-1/2m} |\mathbf{v}_b|^{\frac{1-m}{2m}}$$

$$\begin{aligned} \mathcal{M}: \quad & \left\{ h, \frac{\partial s}{\partial x}, \frac{\partial s}{\partial y}, c \right\} \longrightarrow \{\bar{u}, \bar{v}\} \\ & \mathbb{R}^{N_X \times N_Y \times 4} \longrightarrow \mathbb{R}^{N_X \times N_Y \times 2} \end{aligned}$$

Cheap!

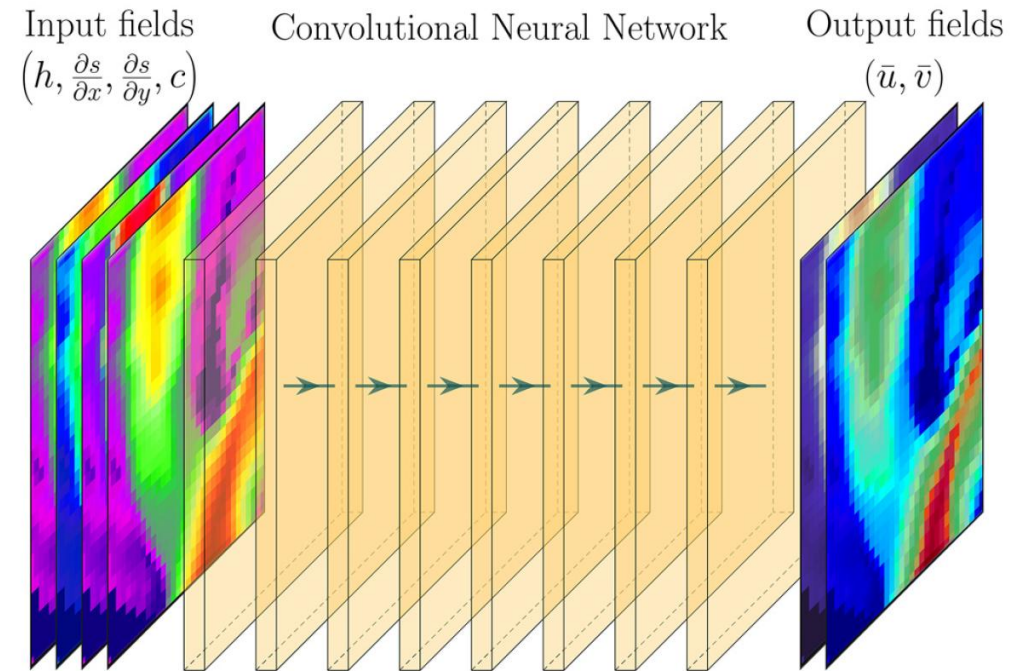


Fig. 3. The function we aim to emulate by learning from hybrid SIA + SSA or Stokes realizations maps geometrical fields (thickness and surface slopes) and basal sliding parametrization to ice flow fields.

Surrogate/emulation (between variables)

Model of glaciers:

Mass balance:
$$\frac{\partial h}{\partial t} + \nabla \cdot (\bar{\mathbf{u}}h) = \text{SMB}$$

Momentum balance (3D; in x,y,z):

$$-\nabla \cdot \boldsymbol{\sigma} = \rho \mathbf{g}, \quad \text{in } V, \quad \boldsymbol{\sigma} = \boldsymbol{\tau} - P\mathbf{I}, \quad \boldsymbol{\tau} = 2\mu D(\mathbf{u}),$$

$$D(\mathbf{u}) =$$

$$\begin{pmatrix} \partial_x u_x & \frac{1}{2}(\partial_y u_x + \partial_x u_y) & \frac{1}{2}(\partial_z u_x) \\ \frac{1}{2}(\partial_y u_x + \partial_x u_y) & \partial_y u_y & \frac{1}{2}(\partial_z u_y) \\ \frac{1}{2}(\partial_z u_x) & \frac{1}{2}(\partial_z u_y) & -\partial_x u_x - \partial_y u_y \end{pmatrix}$$

$$\boldsymbol{\sigma} \cdot \mathbf{n} = 0, \quad P = 0, \quad \text{on } \Gamma_s$$

$$[(I - \mathbf{nn}^T)\boldsymbol{\tau}] \cdot \mathbf{n} = -c^{-m} |(I - \mathbf{nn}^T) \cdot \mathbf{u}|^{m-1} (I - \mathbf{nn}^T) \cdot \mathbf{u}.$$

$$\longrightarrow \mathcal{N}_\lambda : \{h_H, s_H, A_H, c_H, H_H\} \longrightarrow \{\mathbf{u}_H, \mathbf{v}_H\}$$

$$\mathbb{R}^{N_X \times N_Y \times 5} \longrightarrow \mathbb{R}^{N_X \times N_Y \times N_Z \times 2}$$

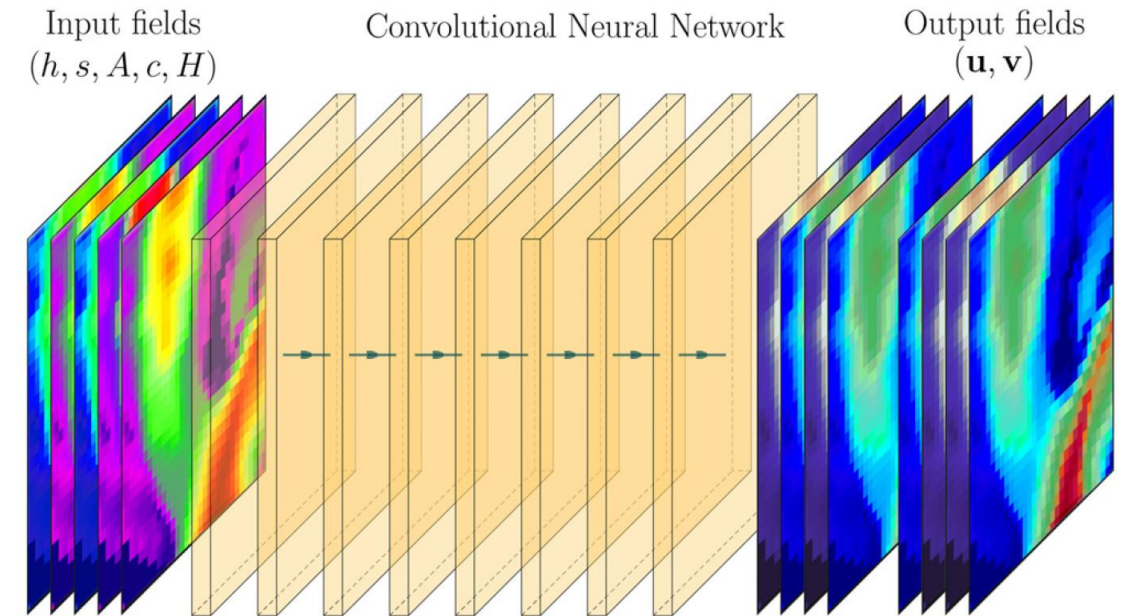


Figure 3. Our emulator consists of a CNN that maps geometrical (thickness and surface topography), ice-flow parameters (shearing and basal sliding) and spatial resolution inputs to 3D ice-flow fields.

Surrogate/emulation (between variables)

Model of glaciers:

Mass balance:
$$\frac{\partial h}{\partial t} + \nabla \cdot (\bar{\mathbf{u}}h) = \text{SMB}$$

Momentum balance (3D; in x,y,z):

$$-\nabla \cdot \boldsymbol{\sigma} = \rho \mathbf{g}, \quad \text{in } V, \quad \boldsymbol{\sigma} = \boldsymbol{\tau} - P\mathbf{I}, \quad \boldsymbol{\tau} = 2\mu D(\mathbf{u}),$$

$$D(\mathbf{u}) =$$

$$\begin{pmatrix} \partial_x u_x & \frac{1}{2}(\partial_y u_x + \partial_x u_y) & \frac{1}{2}(\partial_z u_x) \\ \frac{1}{2}(\partial_y u_x + \partial_x u_y) & \partial_y u_y & \frac{1}{2}(\partial_z u_y) \\ \frac{1}{2}(\partial_z u_x) & \frac{1}{2}(\partial_z u_y) & -\partial_x u_x - \partial_y u_y \end{pmatrix}$$

$$\boldsymbol{\sigma} \cdot \mathbf{n} = 0, \quad P = 0, \quad \text{on } \Gamma_s$$

$$[(I - \mathbf{nn}^T)\boldsymbol{\tau}] \cdot \mathbf{n} = -c^{-m} |(I - \mathbf{nn}^T) \cdot \mathbf{u}|^{m-1} (I - \mathbf{nn}^T) \cdot \mathbf{u}.$$

$$\longrightarrow \mathcal{N}_\lambda : \{h_H, s_H, A_H, c_H, H_H\} \longrightarrow \{\mathbf{u}_H, \mathbf{v}_H\}$$

$$\mathbb{R}^{N_X \times N_Y \times 5} \longrightarrow \mathbb{R}^{N_X \times N_Y \times N_Z \times 2}$$

Physics-informed:

CNN $\lambda = \{\lambda_i, i = 1, \dots, N\}$ that minimise:

$$\lambda = \operatorname{argmin}(\mathcal{J}_{p_H}(\mathcal{N}_\lambda(p_H))), \quad (20)$$

$$\begin{aligned} \mathcal{J}_{p_H}(\mathbf{v}_H) = & \int_{\Omega} \left(\frac{2A_H^{-\frac{1}{n}}}{1 + \frac{1}{n}} \int_{s_H - h_H}^{s_H} |D_H(\mathbf{v}_H)|^{1+\frac{1}{n}} dz \right. \\ & + \frac{c_H^{-m}}{1+m} |\mathbf{v}_H|_M^{1+m} dS \\ & \left. + \rho g \int_{s_H - h_H}^{s_H} (\nabla s_H \cdot \mathbf{v}_H) dz \right) d\Omega. \end{aligned} \quad (18)$$

A Variational LSTM Emulator of Sea Level Contribution From the Antarctic Ice Sheet

Peter Van Katwyk✉, Baylor Fox-Kemper, Hélène Seroussi, Sophie Nowicki, Karianne J. Bergen

Plain Language Summary

An emulator is a computational model that is designed to rapidly approximate another more complex, computationally intensive model. In this work, we propose a new emulator of the Antarctic ice sheet that provides faster and more accurate projections of the future sea level rise due to the melting of the ice sheet. This new emulator uses a technology called neural networks (NNs), a common tool within the field of artificial intelligence. We show that the NN-based emulator produces accurate sea level projections while also being able to quantify the inherent uncertainty associated with future sea level rise. With these improvements, this emulator will enable scientists to better investigate the ice sheet itself and learn how it connects to Earth's dynamic behavior. In addition, due to the efficiency of the proposed emulator, not only are many avenues of research opened for further improvements of ice sheet emulators, but also a wider range of scientists will be able to contribute to future development. This will lead to a better understanding of the drivers of sea level rise as well as more informed decision-making and mitigation efforts.

ML for cryosphere modeling

- **Surrogate/emulation model (forward modeling):**

NN (Tarasov et al. 2012), GP (Edwards et al. 2021)

CNN (IGM: Jouvét et al. 2021, 2023a,b, Verjans & Robel 2024)

GNN (Rahnemoonfar and Koo 2025, Koo, Cheng et al. 2025, Propp et al. arxiv, FloeNet: Gregory et al. arxiv)

DeepONet (Howard, Perego et al. 2023, He, Perego et al. 2023)

LSTM (Katwyk et al. 2023)

- **Data assimilation (inverse modeling):**

Bayesian inference + NN (Brinkerhoff et al. 2021)

PINN (Riel, Minchew et al. 2021, Diffice: Wang, Lai et al. 2022, 2025, PINNICLE: Cheng et al. 2024, 2025, Steidl, Bamber et al. 2025, Krishna et al. 2026)

GP (Downs et al. 2022, Hill et al. 2025a,b, Brinkerhoff et al. 2025)

UDE (ODINN: Bolibar, Sapienza et al. 2023)

CNN (DeepBedMap: Leong & Horgan 2020, Jouvét 2023a,b, DeepTopoNet: Tama et al. 2025)

- **Downscaling**

Generative models (Bochow et al. 2026)

Inversed modeling:



Prediction of ice dynamics relies on the **basal friction** and **viscosity structure** of ice sheet, which are not easily measurable.

Inverse modeling (find basal friction)

- Given u, v at the ice surface
 h at sparse locations & A, s
- Find c and h

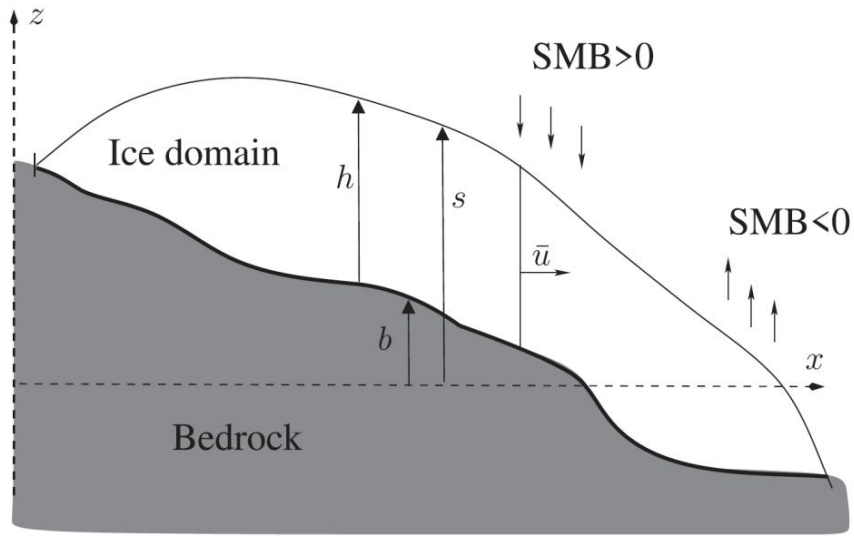


Fig. 2. Cross-section of a glacier with notations.

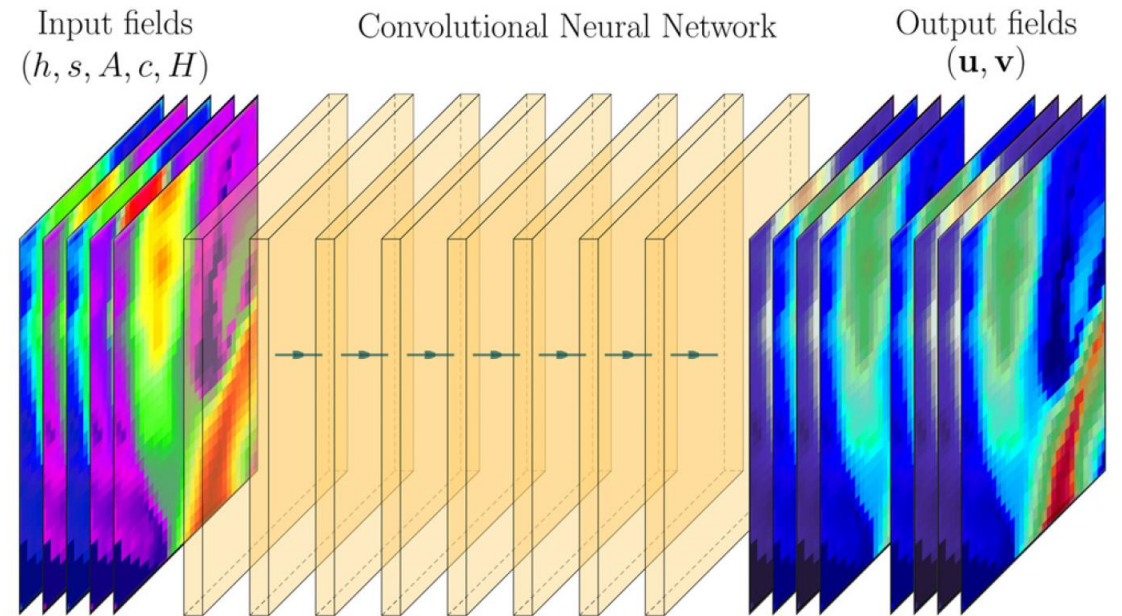


Figure 3. Our emulator consists of a CNN that maps geometrical (thickness and surface topography), ice-flow parameters (shearing and basal sliding) and spatial resolution inputs to 3D ice-flow fields.

Inverse modeling (find basal friction)

- Given u, v at the ice surface
 h at sparse locations & A, s
- Find c and h

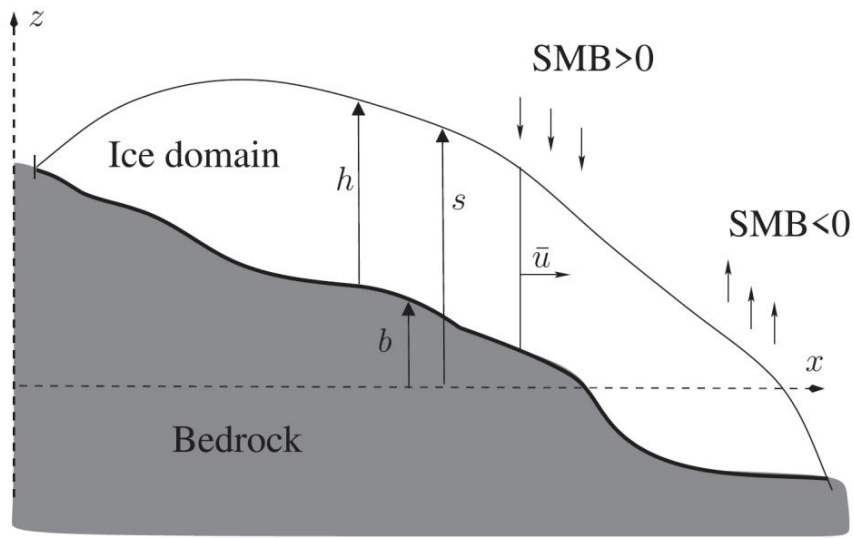
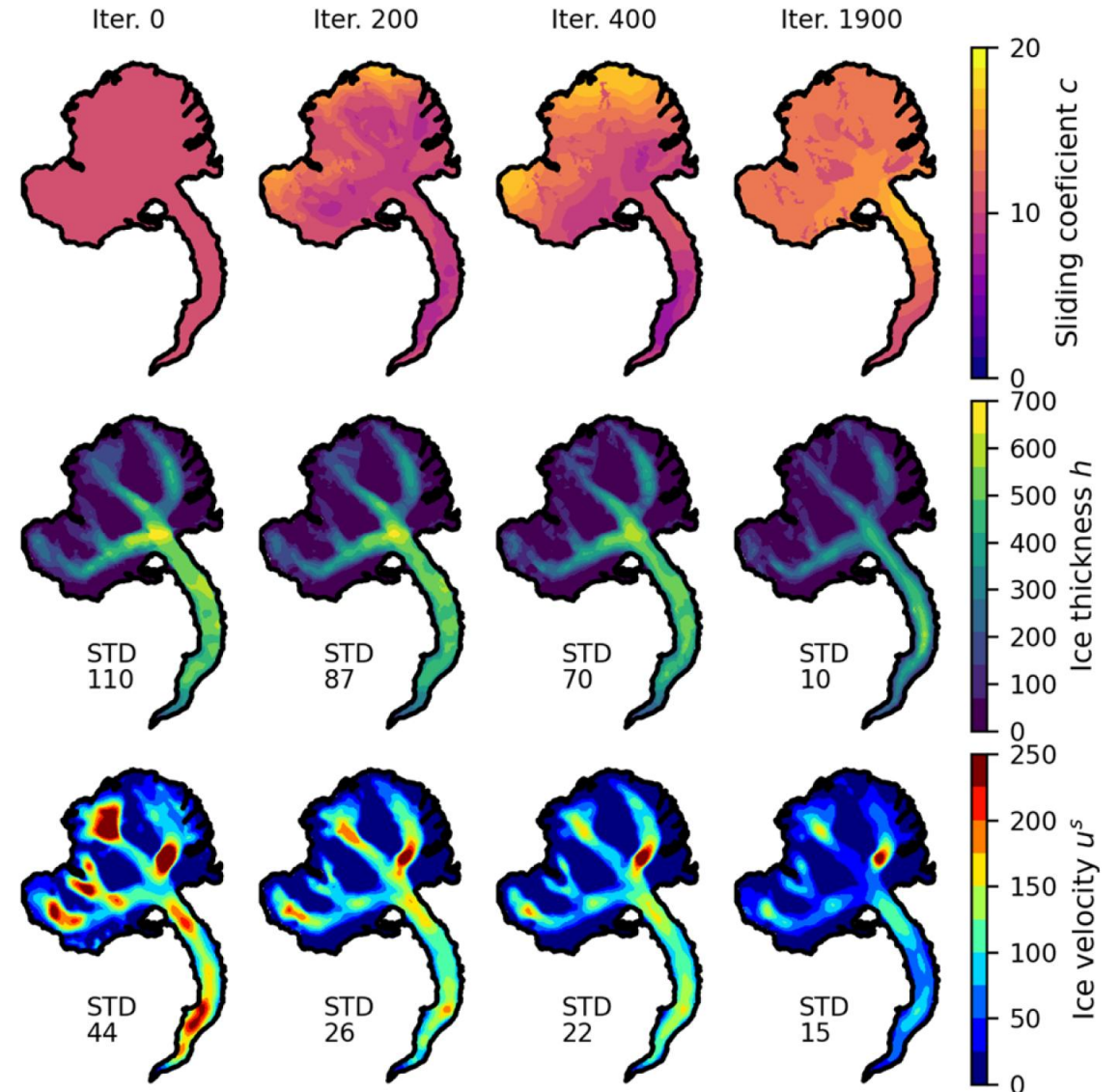
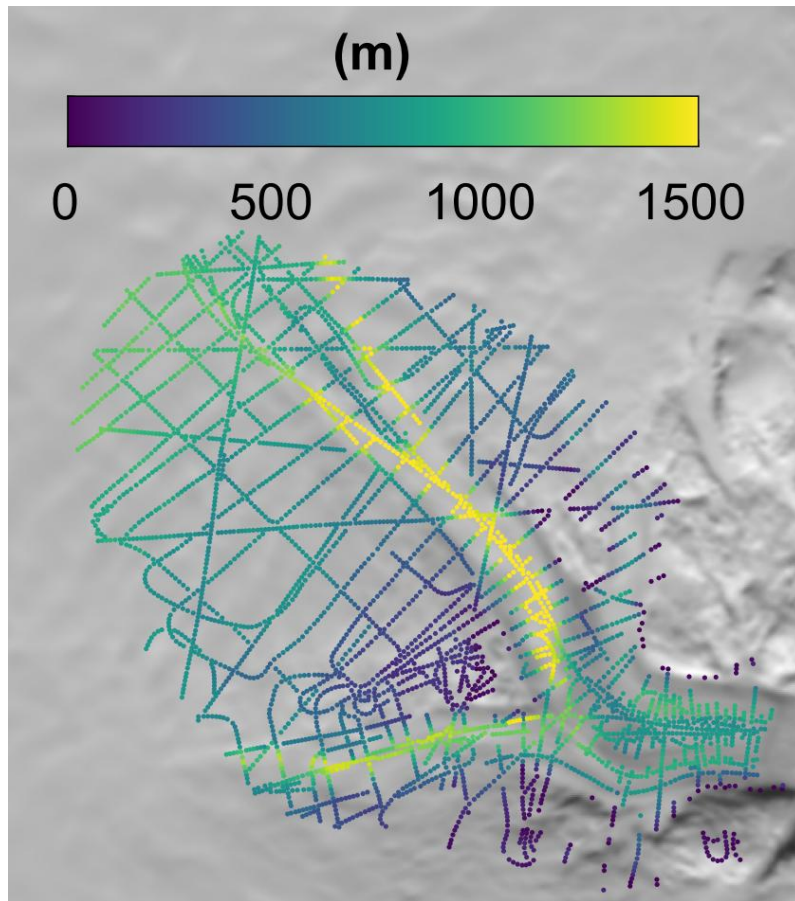


Fig. 2. Cross-section of a glacier with notations.

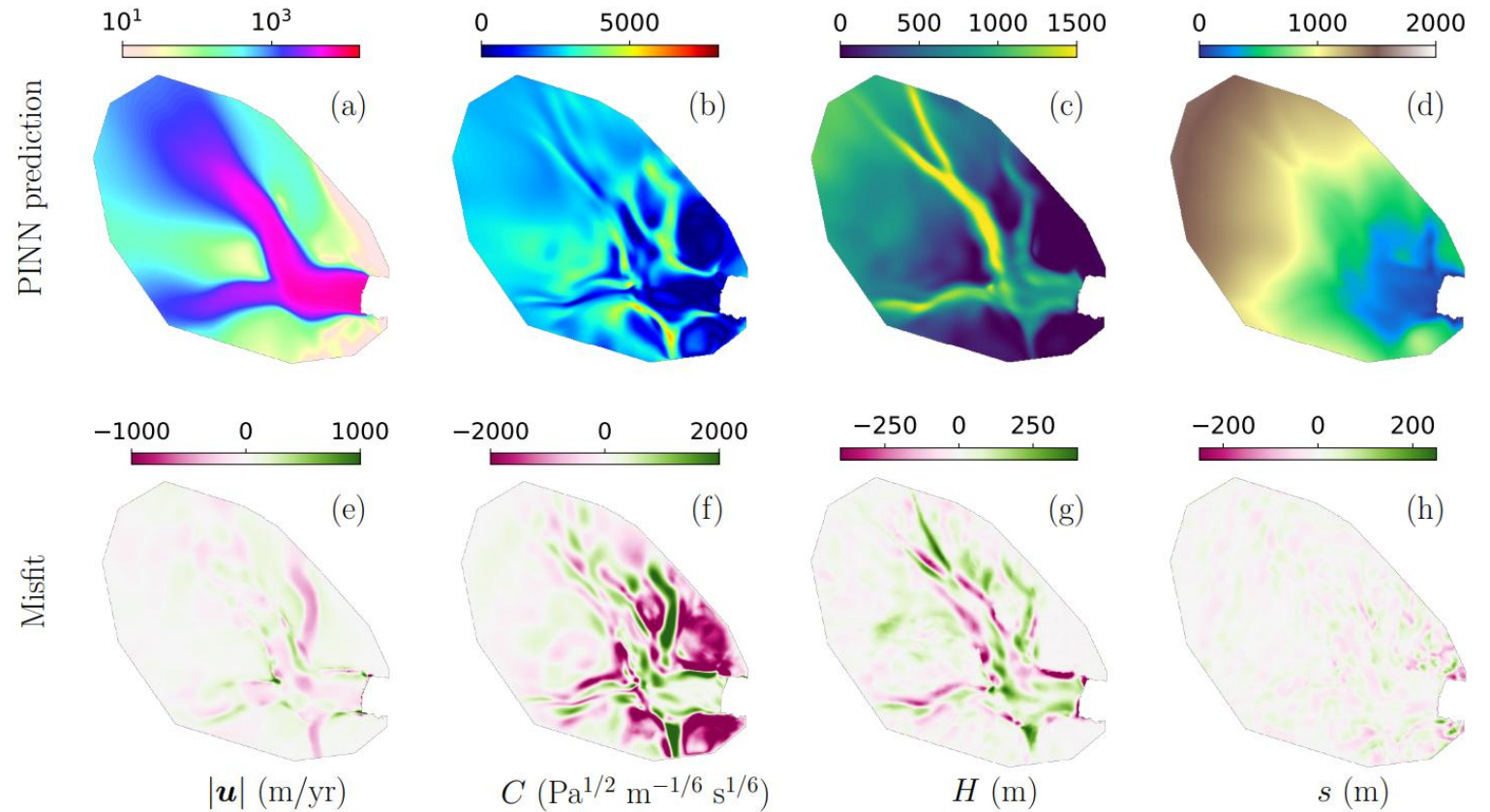


Inverse modeling (find basal friction and thickness)

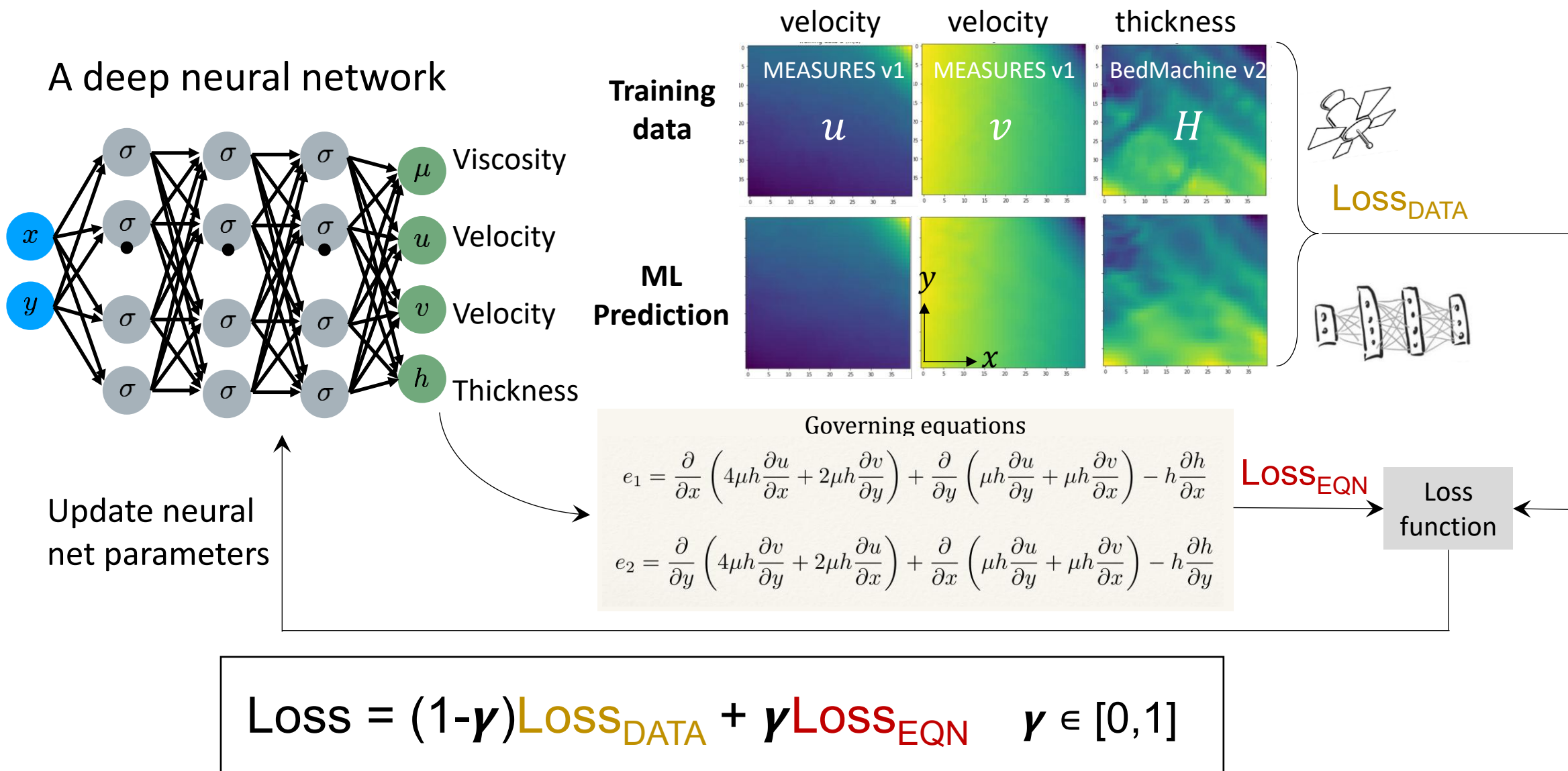
The actual observed thickness



Inferring both ice thickness and basal friction

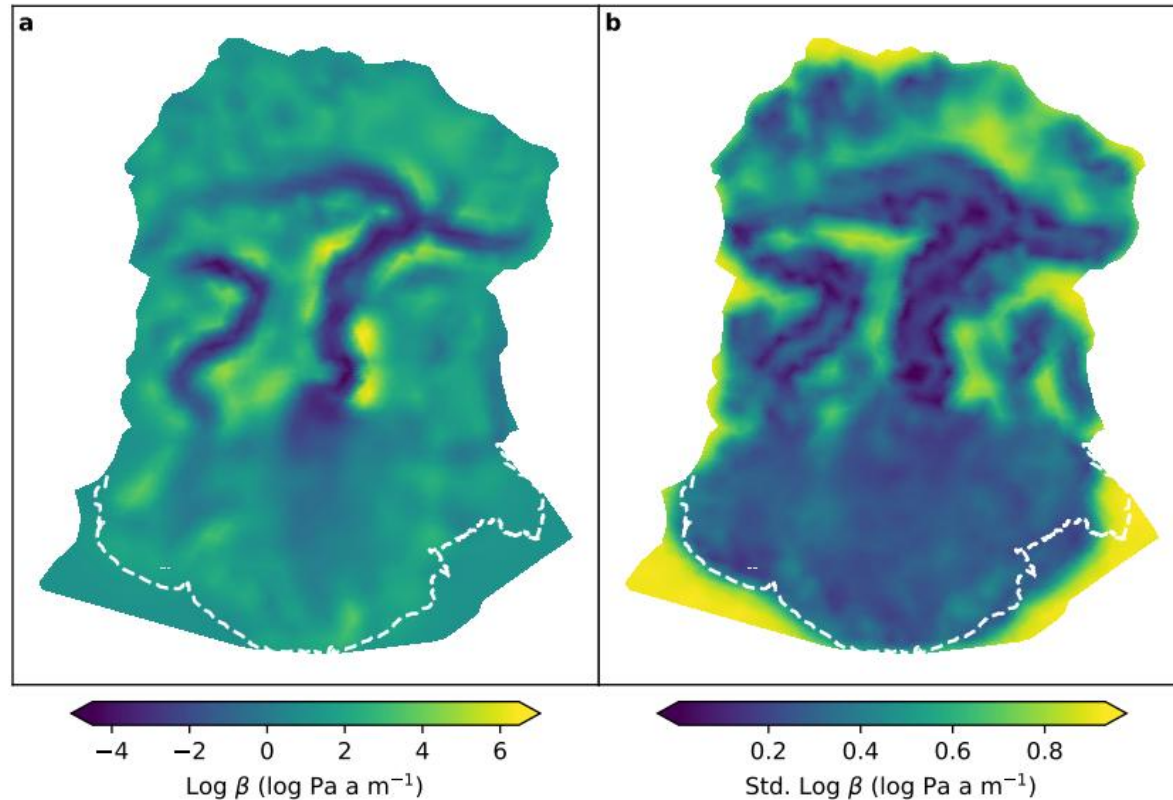


Inverse modeling (find viscosity)

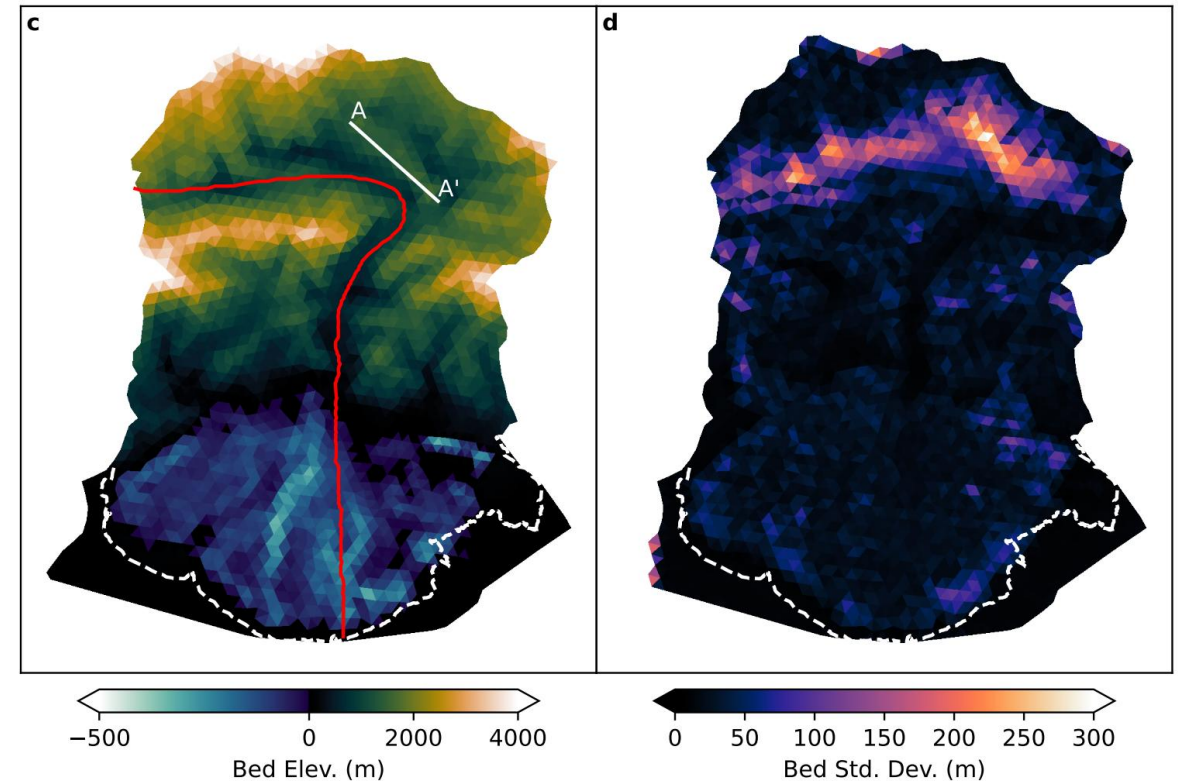


Inverse modeling (find basal friction, bed topography, and surface mass balance with UQ)

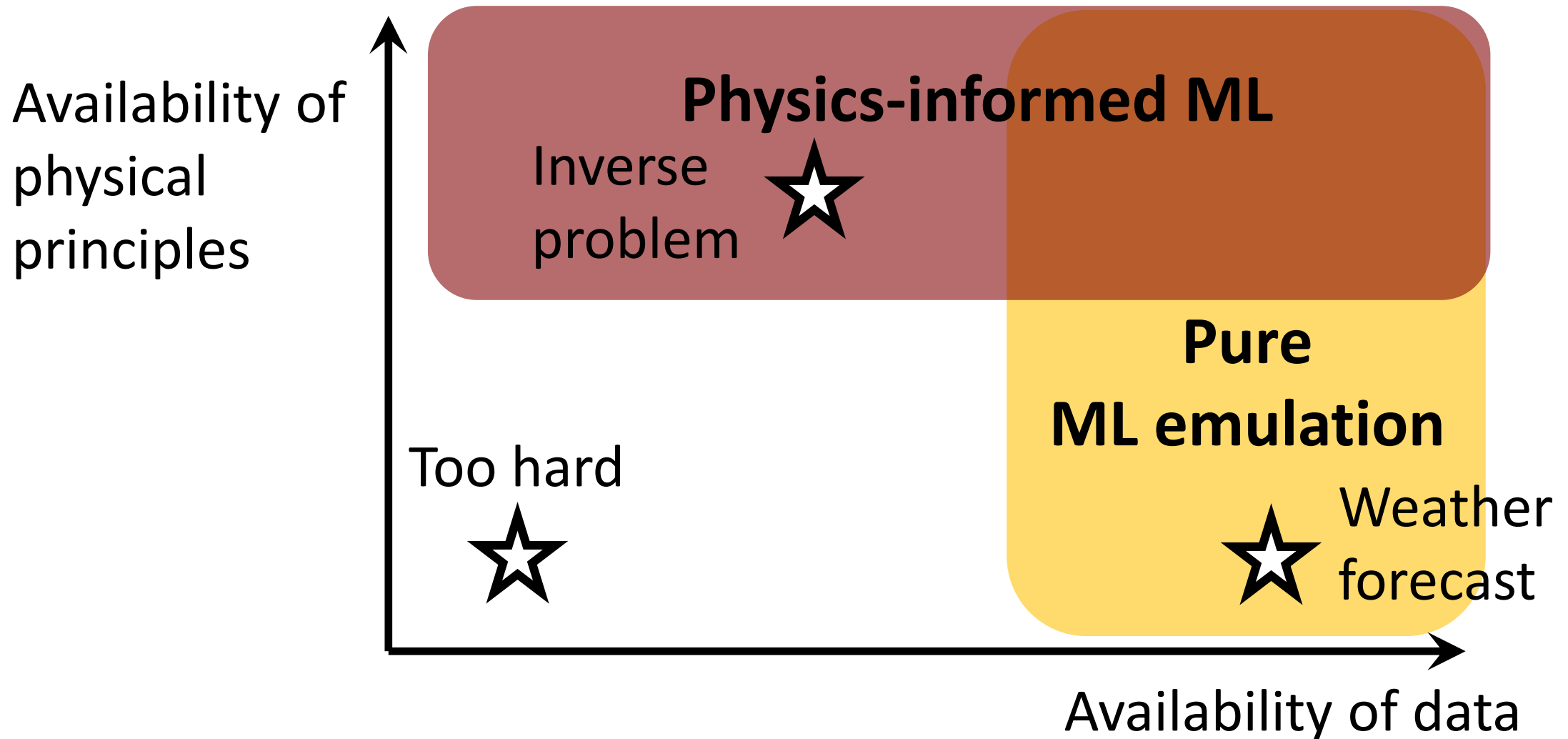
Modelled **basal traction** averaged over 1985- 2019



Inferred **bed topography**



Data and physics



Outline

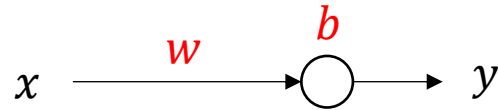
- Why neural networks?
- NN basics:
 - Universal function approximation
 - Gradient descent
 - Back propagation
- Advanced topics: challenges of using NN in scientific problems

What is a neural network?

An analytical model of output y as a function of input x , containing some fitting parameters

1. Linear Regression Model:

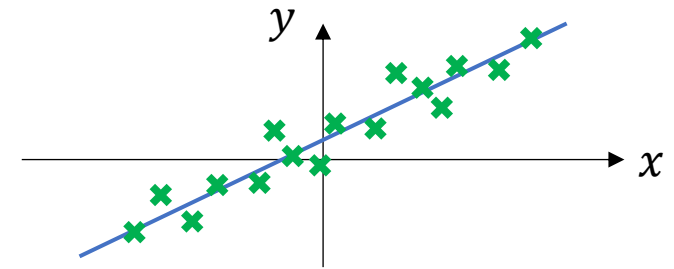
$$y = wx + b$$



Given observations of $\{x_d^i, y_d^i\}_i^n$

Find the w and b that minimizes

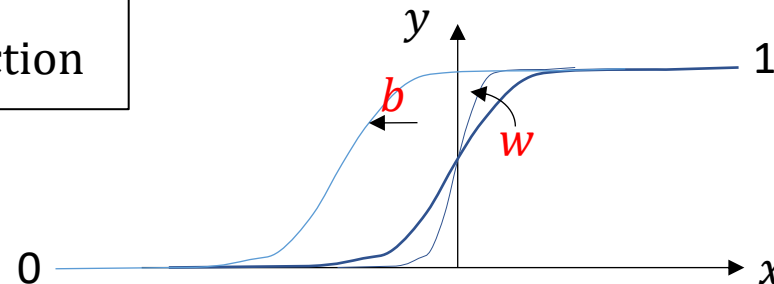
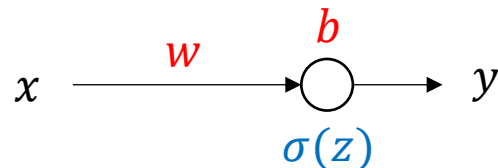
$$J = \sum_{i=1}^n (y(x_d^i) - y_d^i)^2$$



2. Logistic Regression Model: make output 0 to 1

$$y = \sigma(wx + b),$$

where $\sigma(z) = \frac{1}{1 + e^{-z}}$ is a sigmoid function



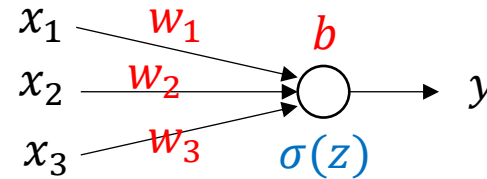
What is a neural network?

An analytical model of output y as a function of input x , containing some fitting parameters

2. Logistic Regression Model: make output -1 to 1

$$y = \sigma(w_1x_1 + w_2x_2 + w_3x_3 + b),$$

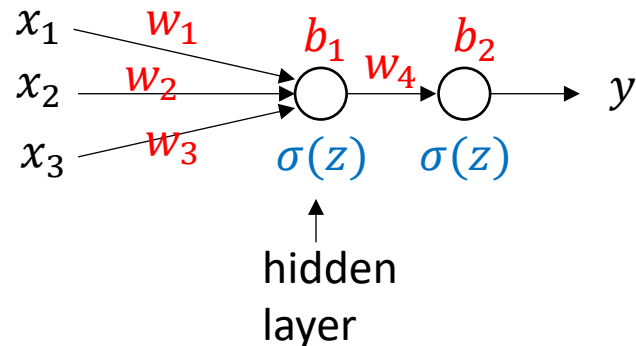
where $\sigma(z) = \frac{1}{1 + e^{-z}}$ is a sigmoid function



3. Neural network:

$$y = \sigma(w_4\sigma(w_1x_1 + w_2x_2 + w_3x_3 + b_1) + b_2),$$

where $\sigma(z)$ is a nonlinear activation function



Common choices of $\sigma(z)$

$\text{sigmoid}(z)$

...

Output ranges from -1 to 1

What is a neural network?

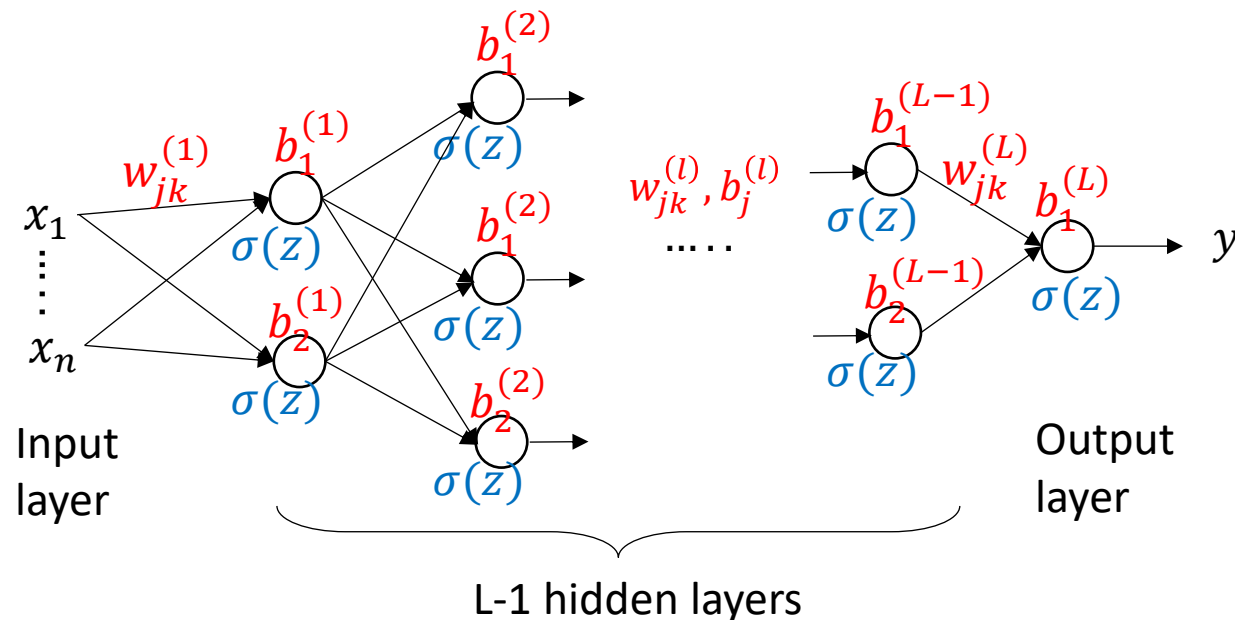
An analytical model of output y as a function of input x , containing some fitting parameters

3. Neural network: maps $x \in \mathbb{R}^n$ to $y \in \mathbb{R}^m$

More generally...

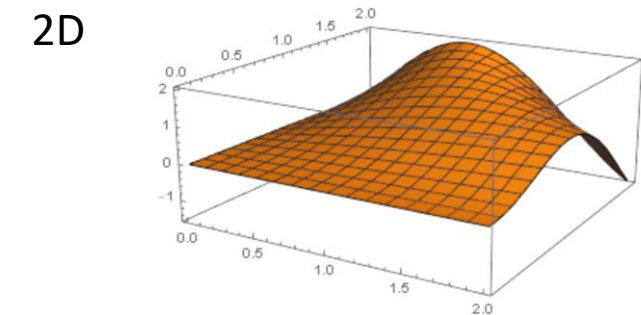
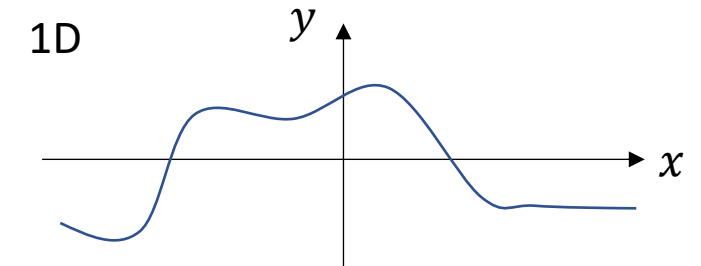
$$y = f_n(w_{jk}^{(l)}, b_j^{(l)}, x_i),$$

where $\sigma(z)$ is a nonlinear activation function



Neural network is a general function approximation

-> Universal function approximation



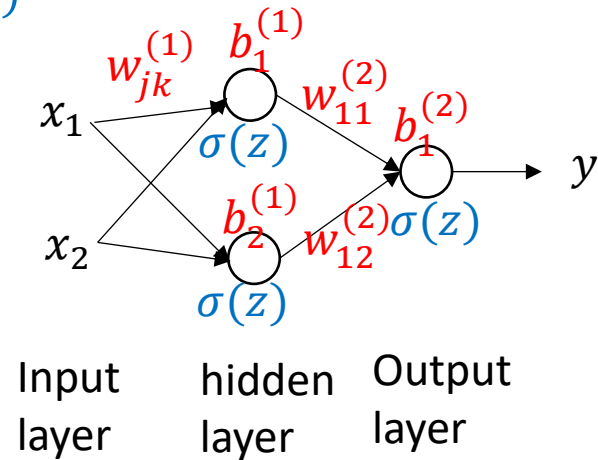
n-Dimension surface....

Why are activation function nonlinear?

Common choices of $\sigma(z)$

$\text{sigmoid}(z)$

...



If activation fn is linear, e.g. $\sigma(z) = z \dots$

Input x_1, x_2

In hidden layer, $j=1,2$

$$z_j^{(1)} = \sum_{k=1}^2 w_{jk}^{(1)} x_k + b_j^{(1)}$$

$$a_j^{(1)} = \sigma(z_j^{(1)}) = z_j^{(1)}$$

In output layer, $j=1$

$$z_j^{(2)} = \sum_{k=1}^2 w_{jk}^{(2)} a_k^{(1)} + b_j^{(2)}$$

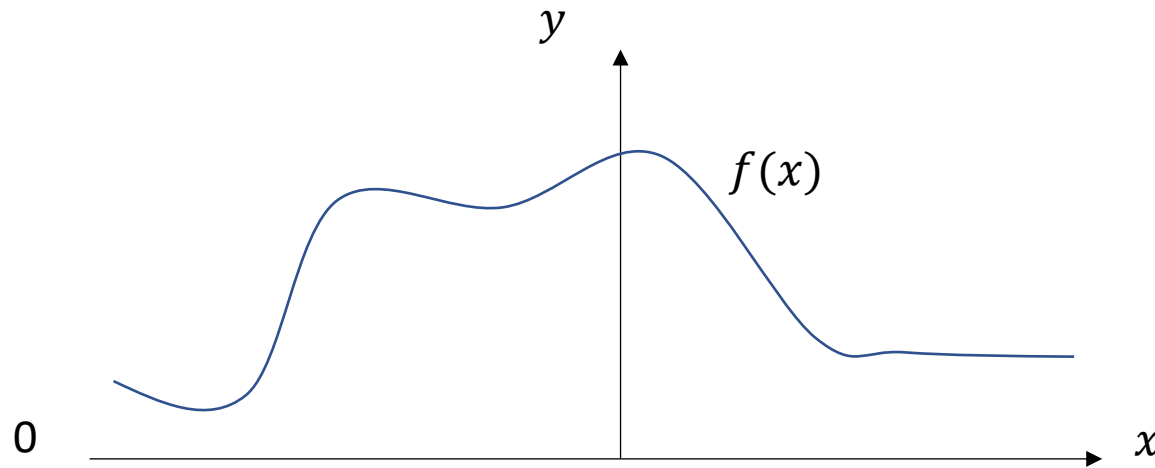
$$a_j^{(2)} = \sigma(z_j^{(2)}) = z_j^{(2)}$$

$$y = a_1^{(2)} = \sum_{k=1}^2 w_{1k}^{(2)} \left(\sum_{l=1}^2 w_{kl}^{(1)} x_l + b_k^{(1)} \right) + b_1^{(2)}$$

- If activation function is linear, NN can only represent a linear function

NN is a Universal Function Approximator

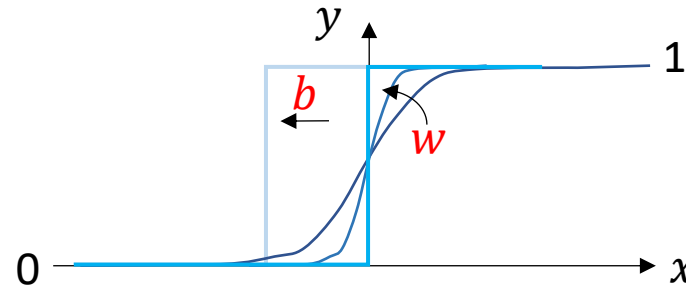
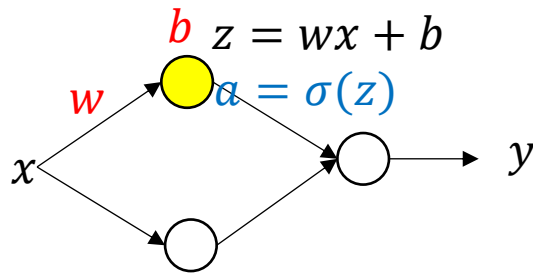
- NN can approximate continuous and smooth functions
A visual proof (for sigmoid activation)



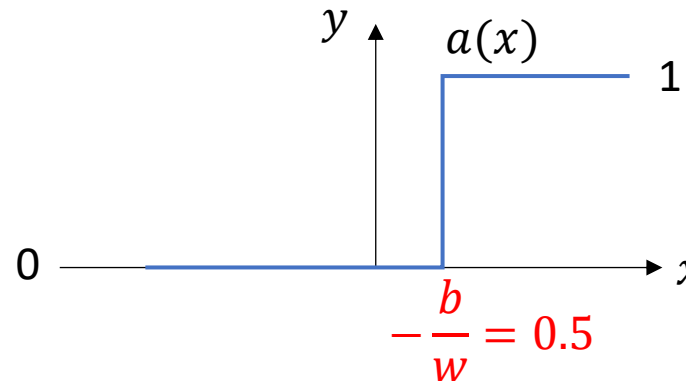
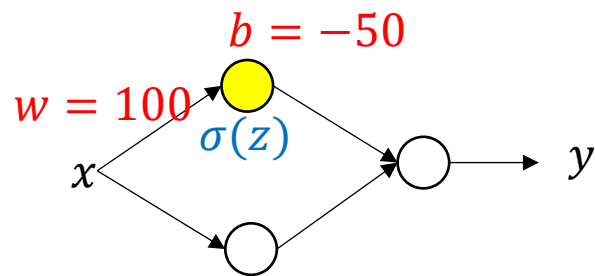
Goal:

NN is a Universal Function Approximator

- NN can approximate continuous and smooth functions
A visual proof (for sigmoid activation)



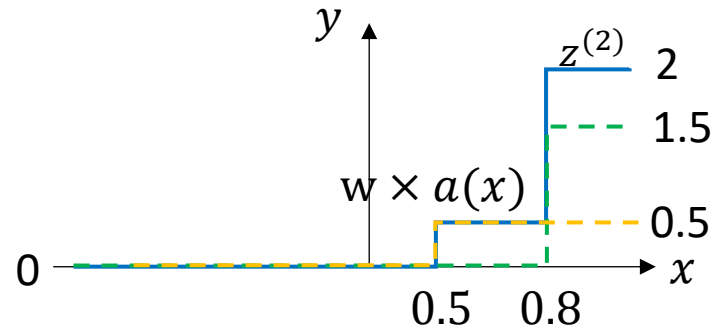
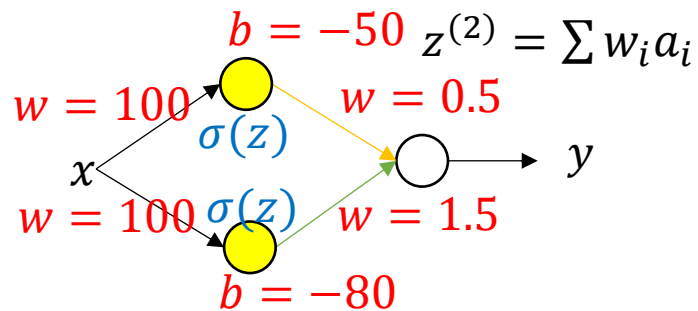
Large w gives a step



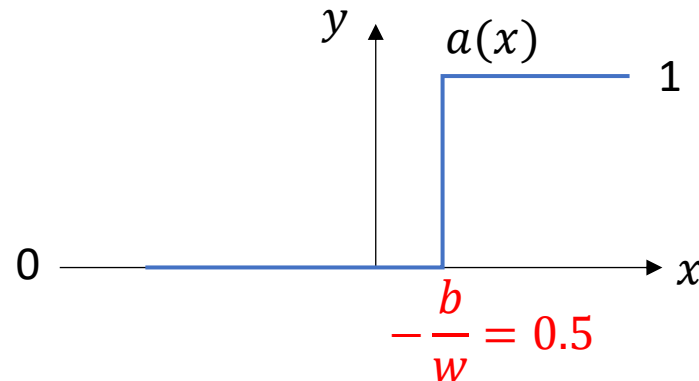
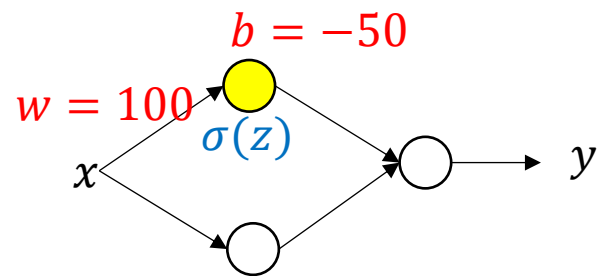
$-\frac{b}{w}$ determines the location of the step

NN is a Universal Function Approximator

- NN can approximate continuous and smooth functions
A visual proof (for sigmoid activation)



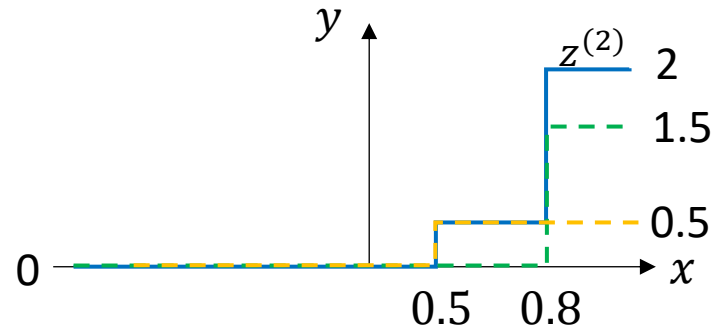
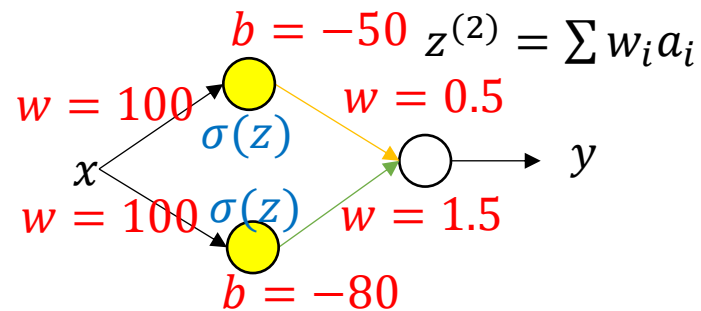
Superposition of two steps



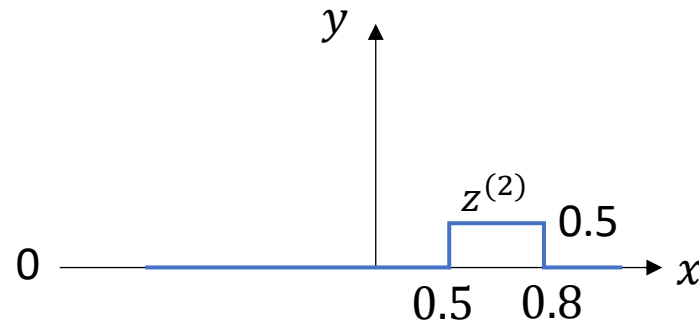
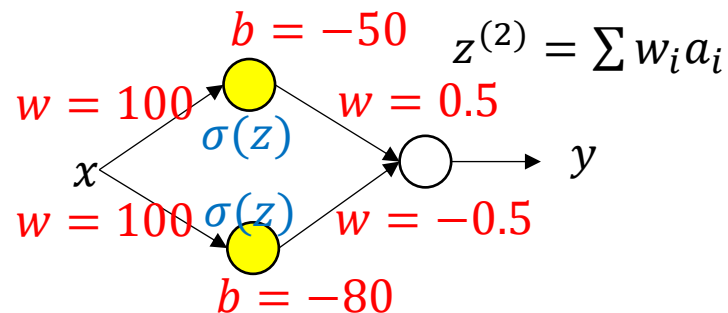
$-\frac{b}{w}$ determines the location of the step

NN is a Universal Function Approximator

- NN can approximate continuous and smooth functions
A visual proof (for sigmoid activation)



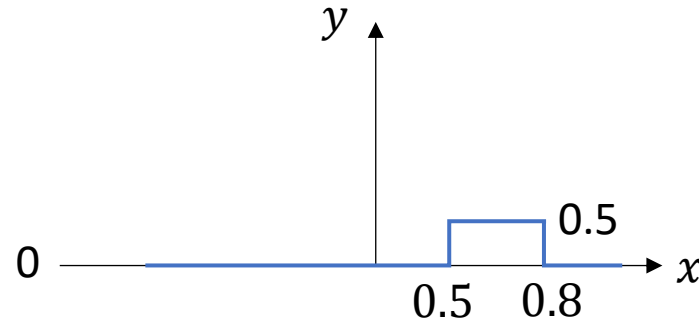
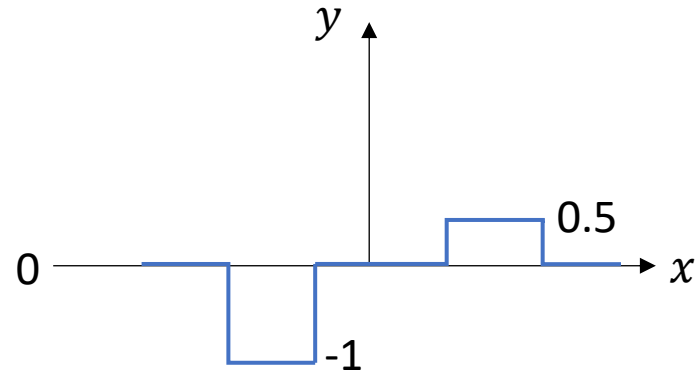
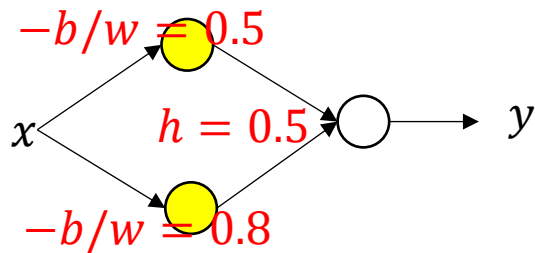
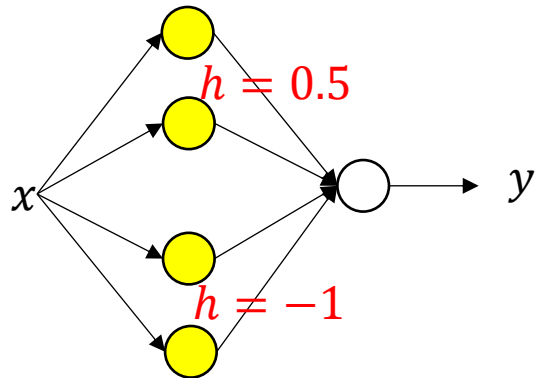
Superposition of two steps



Create a column of height $h=0.5$

NN is a Universal Function Approximator

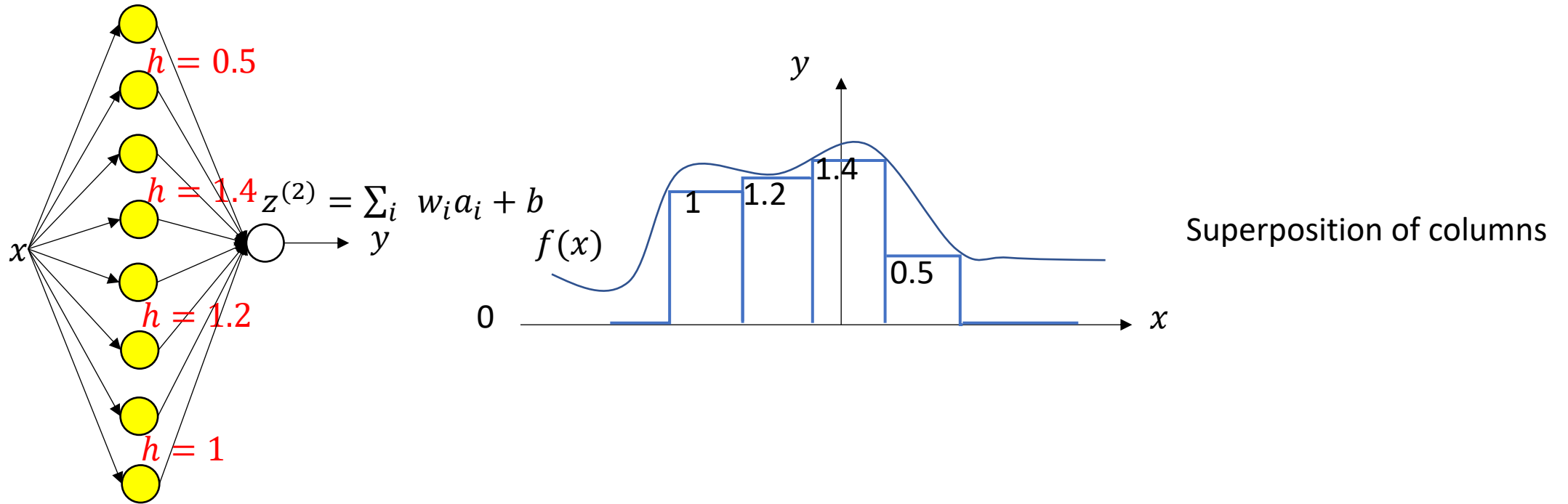
- NN can approximate continuous and smooth functions
A visual proof (for sigmoid activation)



Create a column of height $h=0.5$

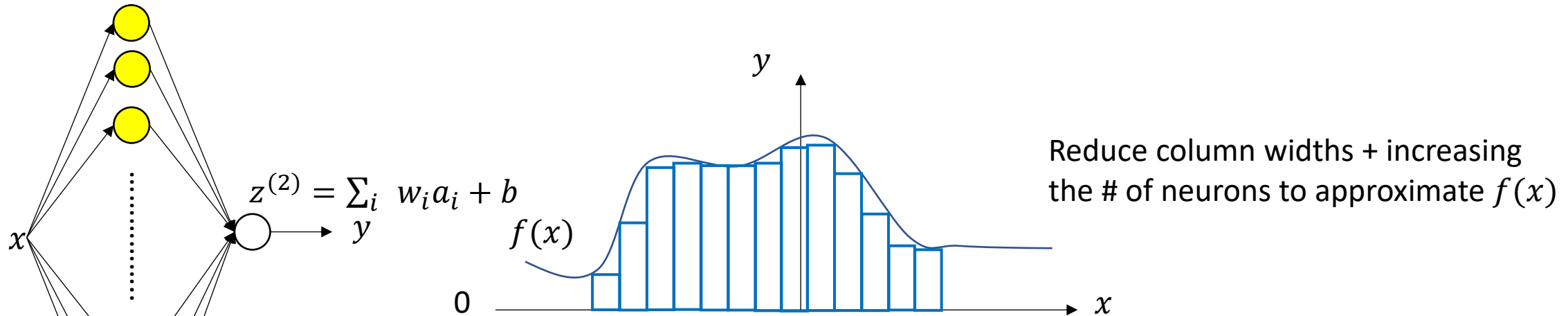
NN is a Universal Function Approximator

- NN can approximate continuous and smooth functions
A visual proof (for sigmoid activation)



NN is a Universal Function Approximator

- NN can approximate continuous and smooth functions
A visual proof (for sigmoid activation)



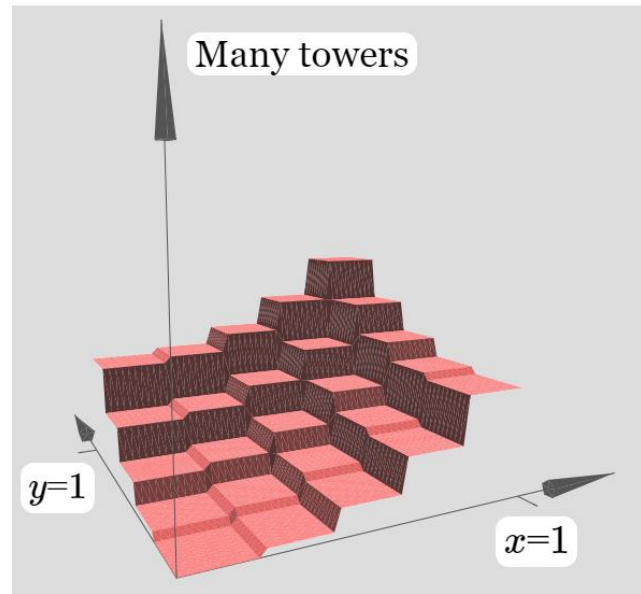
For a given continuous smooth $f(x)$ and an arbitrarily small $\epsilon > 0$
There exist $z^{(2)}$ so that

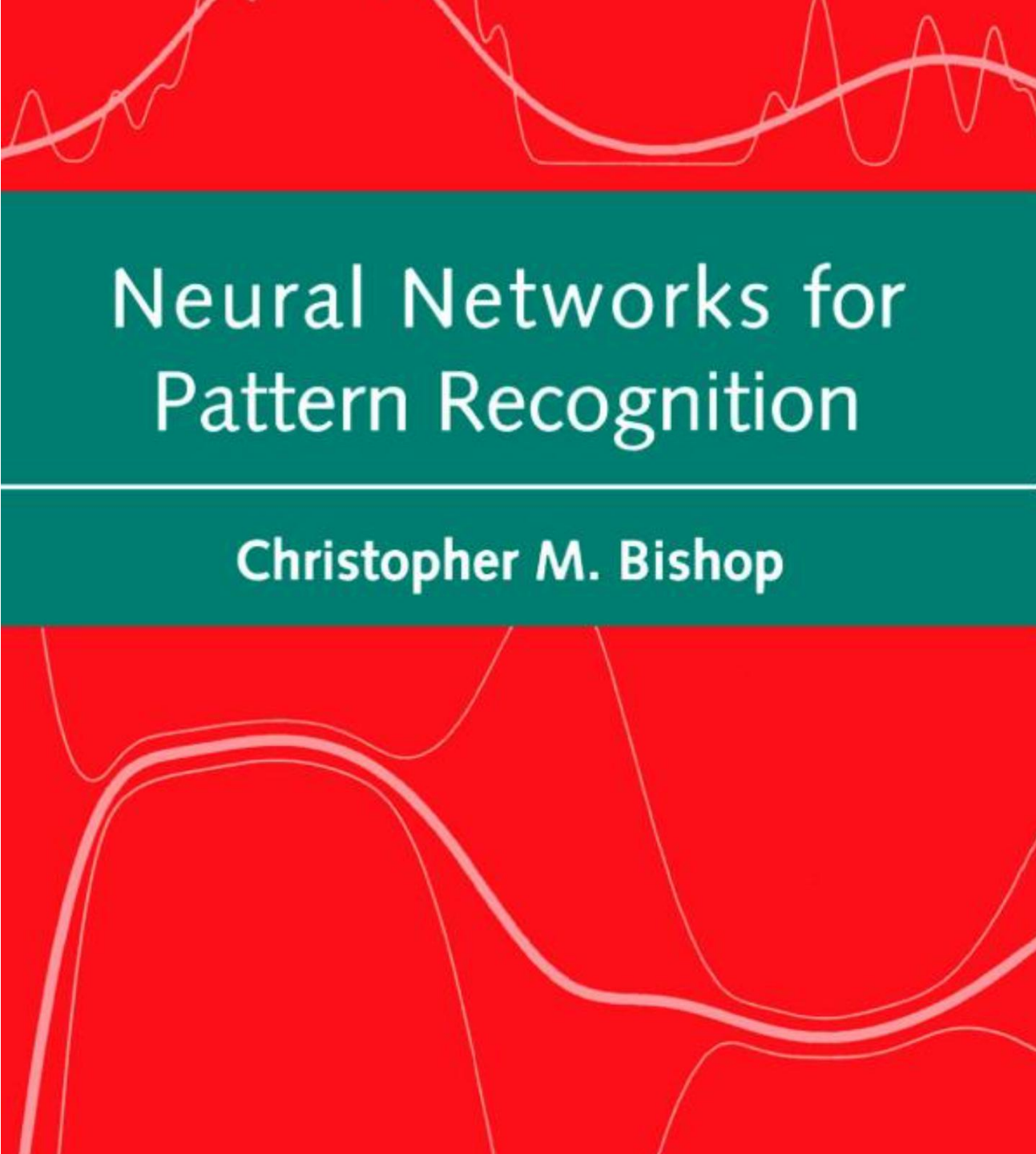
$$\int |z^{(2)}(x) - f(x)| dx < \epsilon$$

NN is a Universal Function Approximator

- Approximate a 2D surface

What should be the
input/output units of NN?





Neural Networks for Pattern Recognition

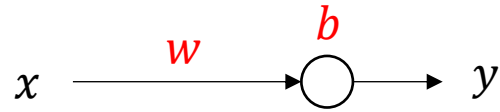
Christopher M. Bishop

Chapter 4 **The Multi-layer Perceptron**

Now we know it is possible to tune weights and biases in a NN to approximate functions. We still need an automated method to find the correct weights and biases.

How to find w and b ?

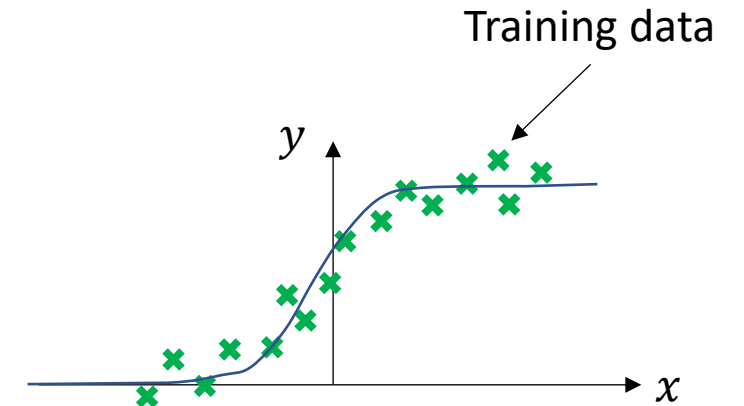
- E.g., Model $y = \sigma(\mathbf{w}x + b)$



Given observations of $\{x_d^i, y_d^i\}_i^m$

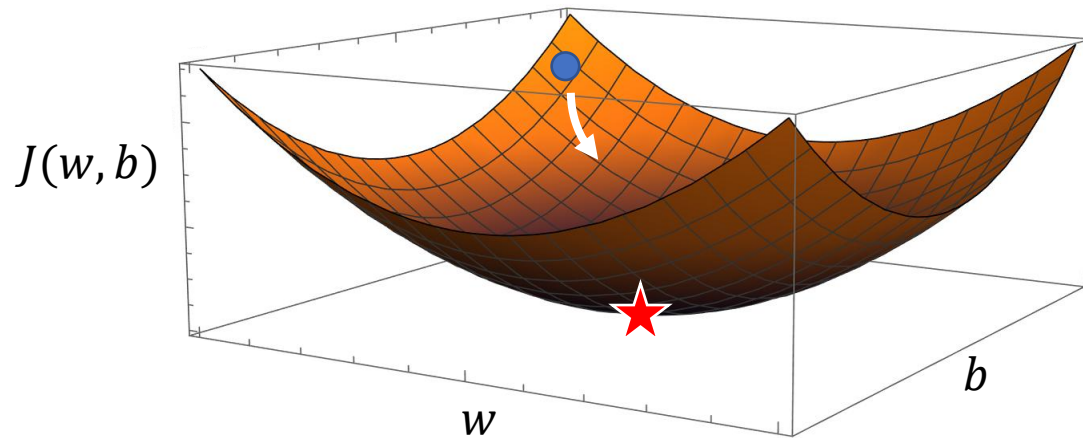
Our goal is to find the model parameters $\mathbf{w}, b$ that minimizes the cost function

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m (y(x_d^i) - y_d^i)^2$$



How to find w and b ?

- E.g., Model $y = \sigma(\mathbf{w}x + b)$
- Cost function $J(w, b) = \frac{1}{m} \sum_{i=1}^m (y(x_d^i) - y_d^i)^2$

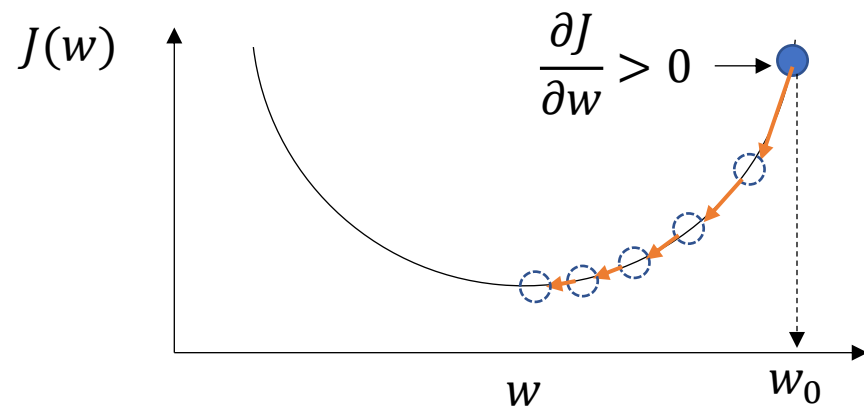


Find the w, b that minimizes $J(w, b)$

i.e. $\frac{\partial J}{\partial w} = \frac{\partial J}{\partial b} = 0$

How to find w and b? Gradient descent

- E.g., Model $y = \sigma(\mathbf{w}x + b)$
- Cost function $J(w, b) = \frac{1}{m} \sum_{i=1}^m (y(x_d^i) - y_d^i)^2$



I. calculate the slope $\frac{\partial J}{\partial w}$ corresponds to an initial $\mathbf{w}$

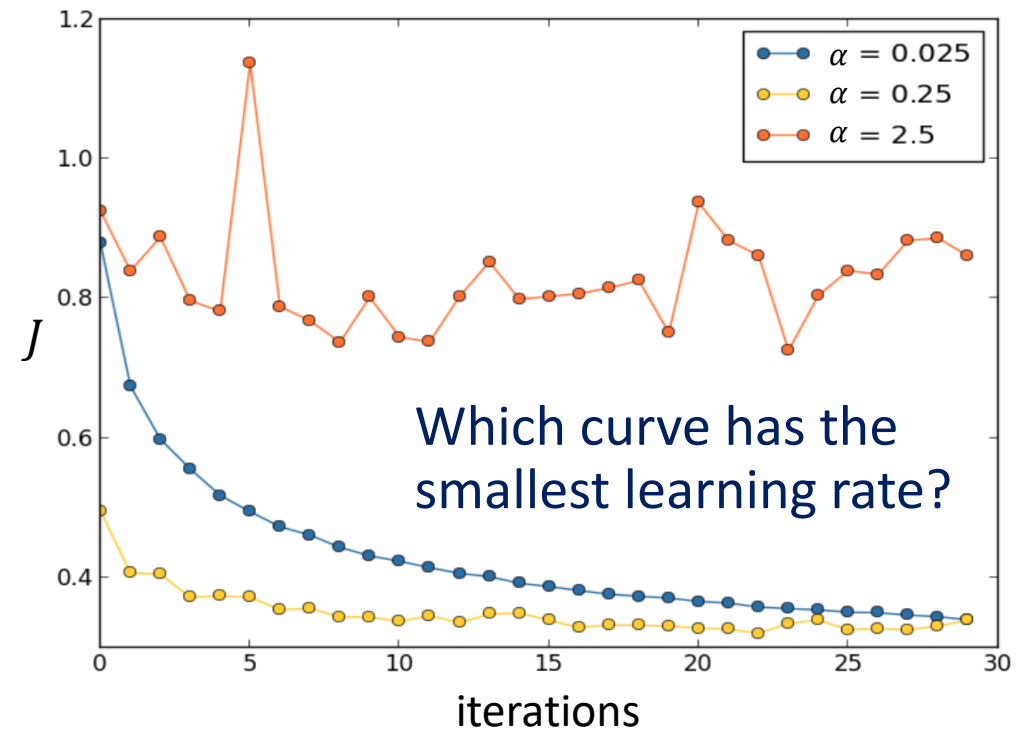
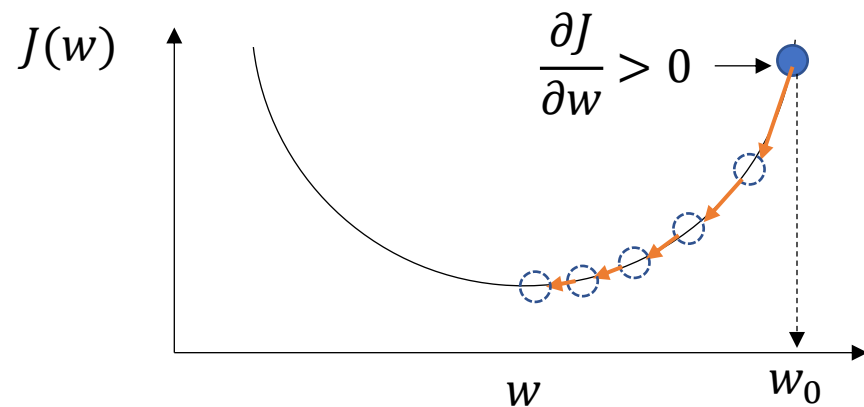
II. Adjust $\mathbf{w}$ according to the local slope

$$w_{new} = w_{old} - \alpha \frac{\partial J}{\partial w}, \quad \alpha > 0 \text{ is the learning rate}$$

III. Iterate until $\frac{\partial J}{\partial w} = 0$

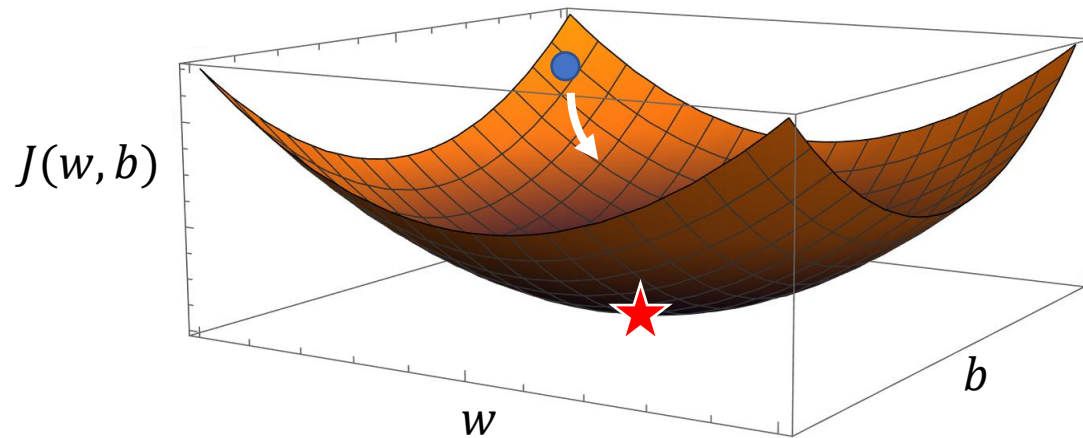
How to find w and b ? Gradient descent

- E.g., Model $y = \sigma(\mathbf{w}x + \mathbf{b})$
- Cost function $J(\mathbf{w}, \mathbf{b}) = \frac{1}{m} \sum_{i=1}^m (y(x_d^i) - y_d^i)^2$



How to find w and b? Gradient descent

- E.g., Model $y = \sigma(\mathbf{w}x + b)$
- Cost function $J(w, b) = \frac{1}{m} \sum_{i=1}^m (y(x_d^i) - y_d^i)^2$



Gradient on a surface

- $-\nabla J$ gives the direction of the steepest decrease of J

$$-\nabla J(w, b) = -\left(\frac{\partial J}{\partial w}, \frac{\partial J}{\partial b}\right)$$

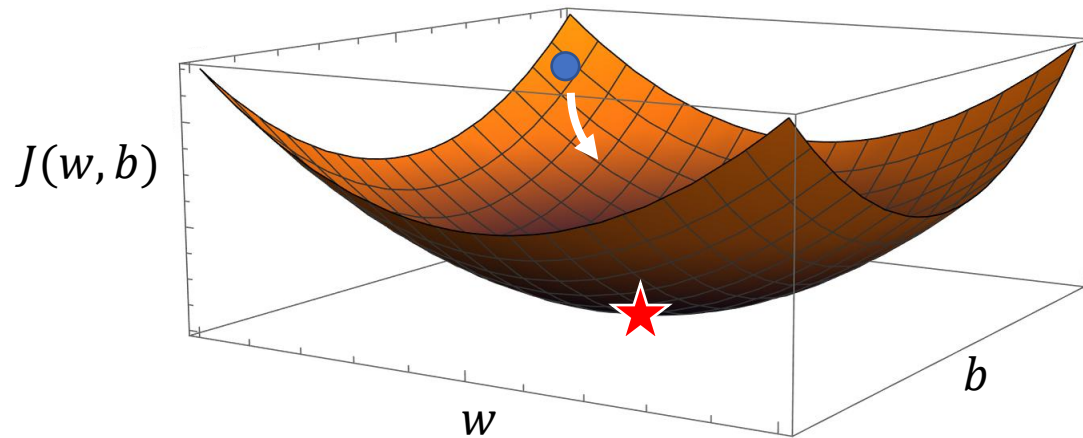
e.g. $-\nabla J(w_0, b_0) = -(10, 1)$ **What does this mean?**

Changing w reduces J 10 times faster than changing b

- $-\nabla J$ tells you which weights and biases reduce cost function J the fastest!

How to find w and b? Gradient descent

- E.g., Model $y = \sigma(\mathbf{w}x + b)$
- Cost function $J(w, b) = \frac{1}{m} \sum_{i=1}^m (y(x_d^i) - y_d^i)^2$



Gradient on a surface

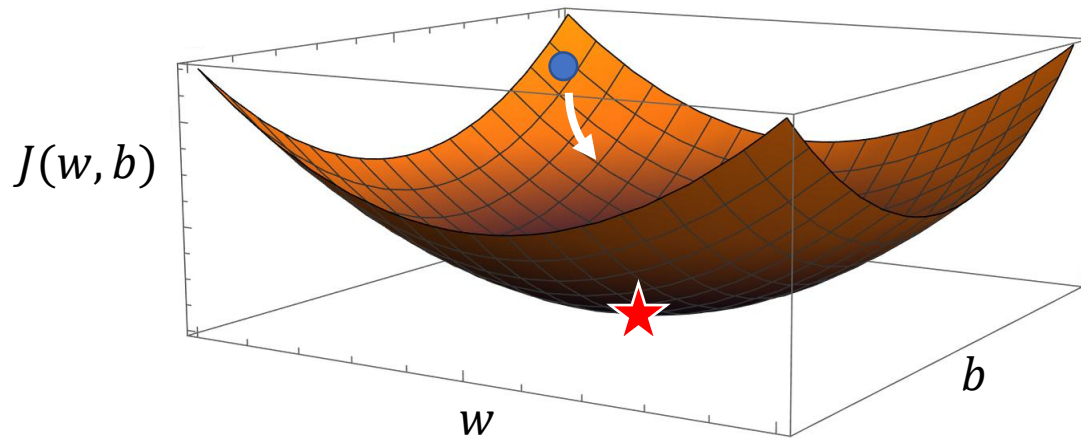
- $-\nabla J$ gives the direction of the steepest decrease of J

$$-\nabla J(w, b) = -\left(\frac{\partial J}{\partial w}, \frac{\partial J}{\partial b}\right)$$

- $(w_{new}, b_{new}) = (w_{old}, b_{old}) - \alpha \nabla J(w, b)$,
 α is the learning rate
- Iterate until $\nabla J = 0$

How to find w and b? Gradient descent

- E.g., Model $y = \sigma(\mathbf{w}x + b)$
- Cost function $J(w, b) = \frac{1}{m} \sum_{i=1}^m (y(x_d^i) - y_d^i)^2$



Gradient on a surface

- $-\nabla J$ gives the direction of the steepest decrease of J

$$-\nabla J(w, b) = -\left(\frac{\partial J}{\partial w}, \frac{\partial J}{\partial b}\right)$$

But how are $\frac{\partial J}{\partial w}$, $\frac{\partial J}{\partial b}$ calculated?

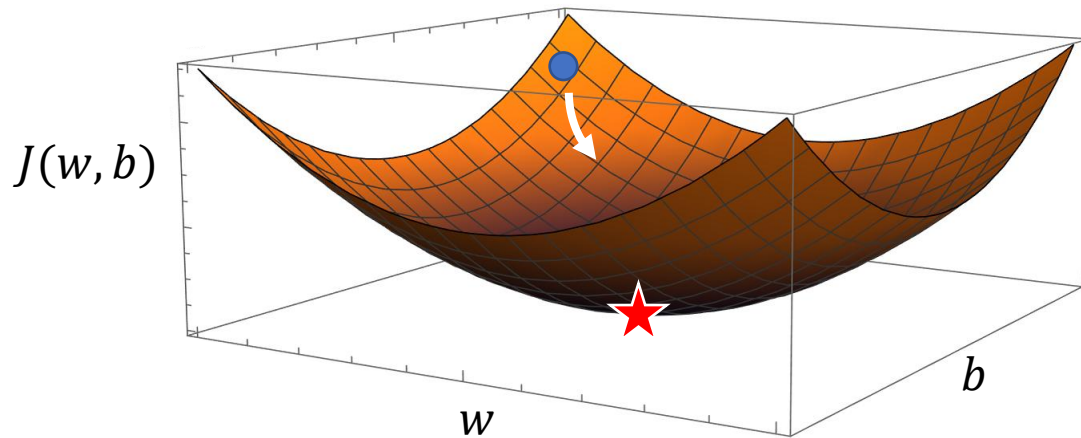
How to find w and b? Gradient descent

- E.g., Model $y = \sigma(\mathbf{w}x + b)$

Defined for every example i

- Cost function $J(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m (y(\mathbf{x}_d^i) - \mathbf{y}_d^i)^2 = \frac{1}{m} \sum_{i=1}^m \overbrace{L(\mathbf{y}^i, \mathbf{y}_d^i)}$

$$L \equiv (y(x) - \mathbf{y}_d)^2, \quad J = \frac{1}{m} \sum_{i=1}^m L \rightarrow \frac{\partial J}{\partial \mathbf{w}} = \frac{1}{m} \sum_{i=1}^m \frac{\partial L}{\partial \mathbf{w}}$$



Back propagation (chain rule)

$$\frac{\partial L}{\partial \mathbf{w}} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial (\mathbf{w}x + b)} \frac{\partial (\mathbf{w}x + b)}{\partial \mathbf{w}} = 2(y - y_d)\sigma'x$$

1) For **one example**:

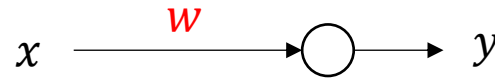
$$\frac{\partial L}{\partial \mathbf{w}}(\mathbf{w}, b, \mathbf{x}_d^i, \mathbf{y}_d^i) = 2(\sigma(\mathbf{w}\mathbf{x}_d^i + b) - \mathbf{y}_d^i)\sigma' \mathbf{x}_d^i$$

2) For the **full data set**:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{m} \sum_{i=1}^m \frac{\partial L}{\partial \mathbf{w}} = \frac{1}{m} \sum_{i=1}^m 2(\sigma(\mathbf{w}\mathbf{x}_d^i + b) - \mathbf{y}_d^i)\sigma' \mathbf{x}_d^i$$

What would $\frac{dL}{dw}$ be for a linear model $\sigma(z) = z$?

Example:

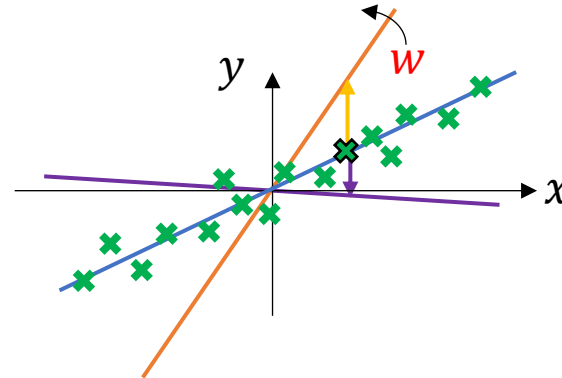
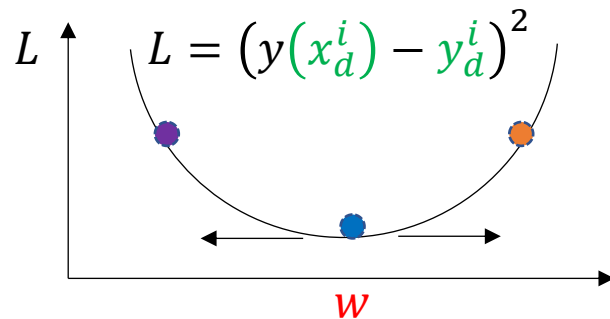


- Linear Regression Model $y = wx$

Defined for every example i

- Cost function $J(w, b) = \frac{1}{m} \sum_{i=1}^m (y(x_d^i) - y_d^i)^2 = \frac{1}{m} \sum_{i=1}^m \overbrace{L(y^i, y_d^i)}$

1) For one example:

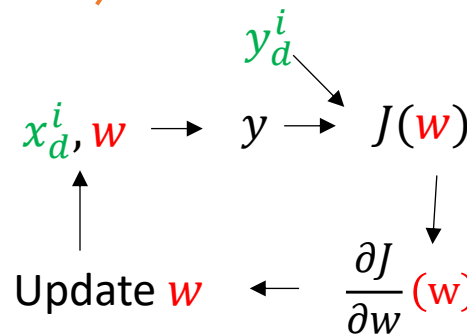
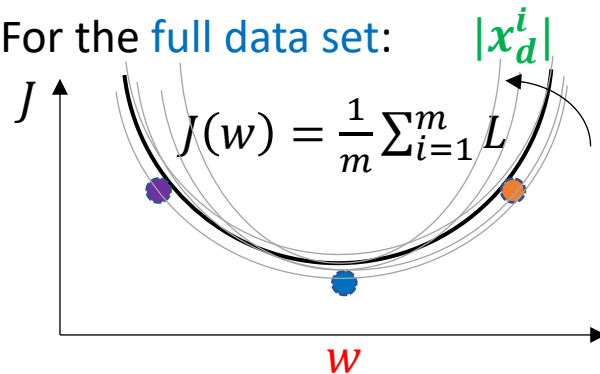


$$L \equiv (y(x) - y_d)^2, \quad J = \frac{1}{m} \sum_{i=1}^m L \rightarrow \frac{\partial J}{\partial w} = \frac{1}{m} \sum_{i=1}^m \frac{\partial L}{\partial w}$$

Back propagation (chain rule)

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial (wx)} \frac{\partial (wx)}{\partial w} = 2(y - y_d)x$$

2) For the full data set:



1) For one example:

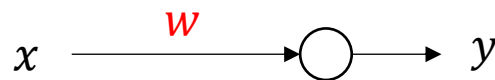
$$\frac{\partial L}{\partial w}(w, b, x_d^i, y_d^i) = 2(wx_d^i - y_d^i)x_d^i$$

This gradient is defined for every example i , and is a function of w

2) For the full data set:

$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum_{i=1}^m \frac{\partial L}{\partial w} = \frac{1}{m} \sum_{i=1}^m 2(wx_d^i - y_d^i)x_d^i$$

Example:

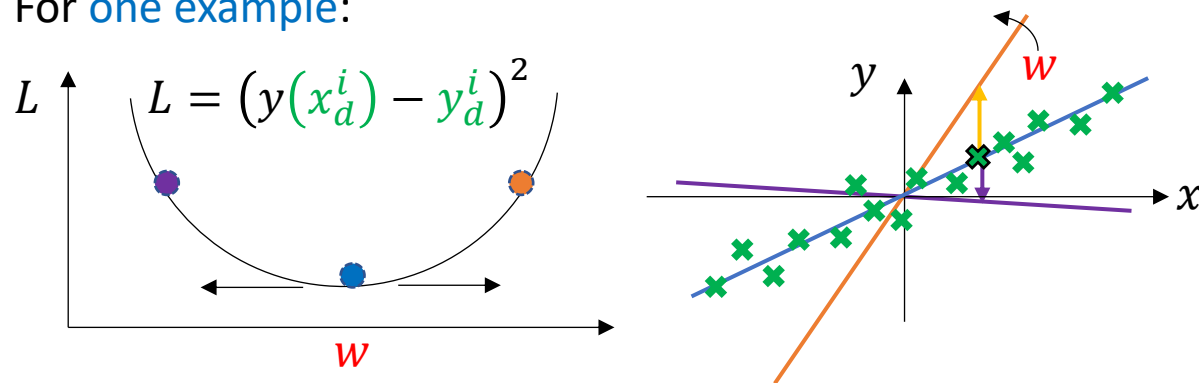


- Linear Regression Model $y = wx$

Defined for every example i

- Cost function $J(w, b) = \frac{1}{m} \sum_{i=1}^m (y(x_d^i) - y_d^i)^2 = \frac{1}{m} \sum_{i=1}^m \overbrace{L(y^i, y_d^i)}$

1) For one example:



$$L \equiv (y(x) - y_d)^2, \quad J = \frac{1}{m} \sum_{i=1}^m L \rightarrow \frac{\partial J}{\partial w} = \frac{1}{m} \sum_{i=1}^m \frac{\partial L}{\partial w}$$

Back propagation (chain rule)

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial (wx)} \frac{\partial (wx)}{\partial w} = 2(y - y_d)x$$

1) For one example:

$$\frac{\partial L}{\partial w} (w, b, x_d^i, y_d^i) = 2(wx_d^i - y_d^i) x_d^i$$

This gradient is defined for every example i , and is a function of w

2) For the full data set:

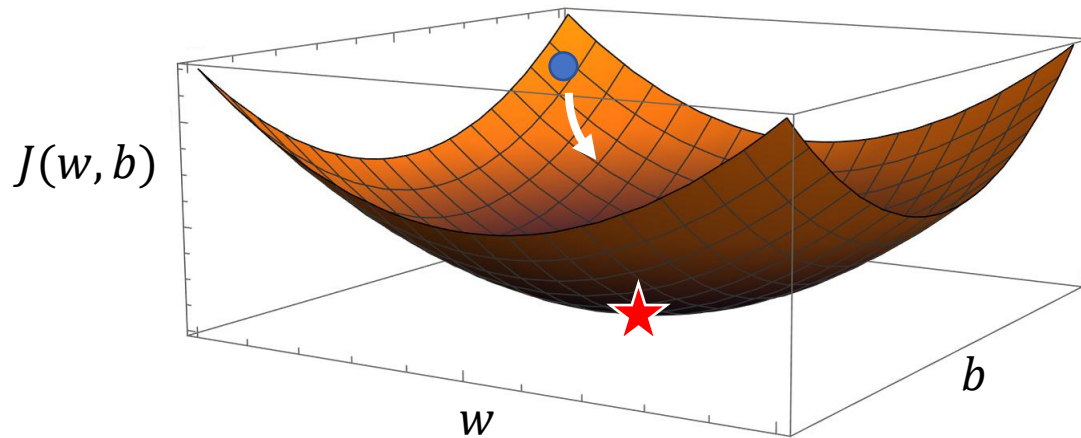
$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum_{i=1}^m \frac{\partial L}{\partial w} = \frac{1}{m} \sum_{i=1}^m 2(wx_d^i - y_d^i) x_d^i$$

What would happen if x_d^i are very large/small?

Note that $\frac{\partial J}{\partial w}$ is analytical fn of w, x_d^i, y_d^i !

How to find w and b ? Gradient descent

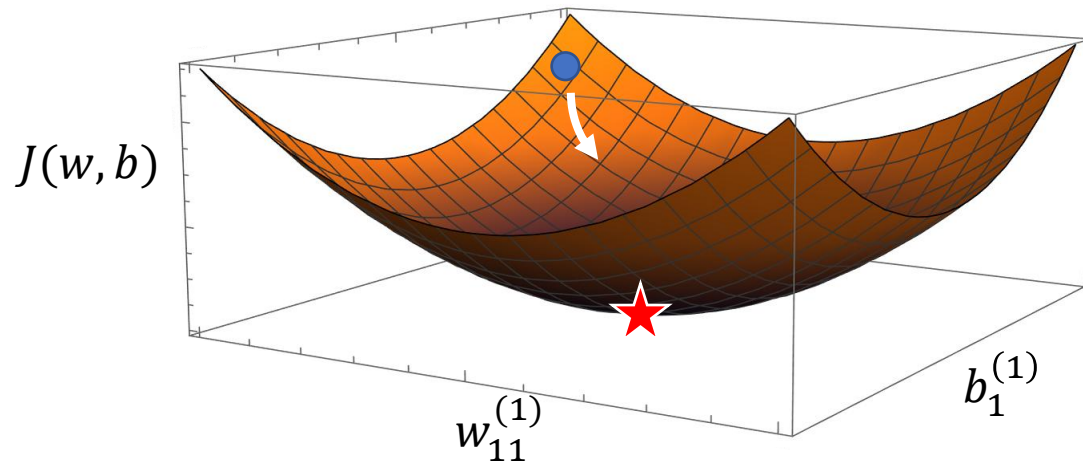
- E.g., Model $y = \sigma(wx + b)$
- Cost function $J(w, b) = \frac{1}{m} \sum_{i=1}^m (y(x_d^i) - y_d^i)^2 = \frac{1}{m} \sum_{i=1}^m L(y^i, y_d^i)$



- $-\nabla J(w, b) = -\left(\frac{\partial J}{\partial w}, \frac{\partial J}{\partial b}\right)$ for a given data set at a given w, b is known analytically
- $(w_{new}, b_{new}) = (w_{old}, b_{old}) - \alpha \nabla J(w, b)$, α is the learning rate
- Iterate until $\nabla J = 0$

How to find w and b in a neural network?

- E. g., Model $y = a^{(2)} = \sigma(\mathbf{W}^{(2)}\sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)})$
- Cost function $J(\mathbf{W}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(1)}, \mathbf{b}^{(2)}) = \frac{1}{m} \sum_{i=1}^m L(y^i, y_d^i)$



- For the full data set ($i=1\dots m$), compute

$$-\nabla J = -\left(\frac{\partial J}{\partial w_{jk}^{(l)}}, \dots, \frac{\partial J}{\partial b_j^{(l)}}\right)$$

- $w_{jk_{new}}^{(l)} = w_{jk_{old}}^{(l)} - \alpha \frac{\partial J}{\partial w_{jk}^{(l)}}$, $b_{j_{new}}^{(l)} = b_{j_{old}}^{(l)} - \alpha \frac{\partial J}{\partial b_j^{(l)}}$
 α is the learning rate
- Iterate until $\nabla J = 0$

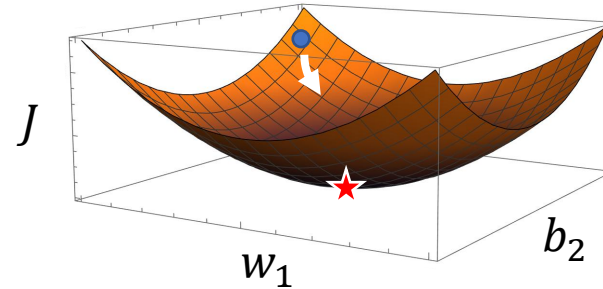
So far we have discussed...

- Universal function approximator
- Gradient descent: a method to find weights and biases that minimize J
- Calculate ∇J
 - One example
 - A full dataset
- Forward and backpropagation (a clever way to get gradients of J)

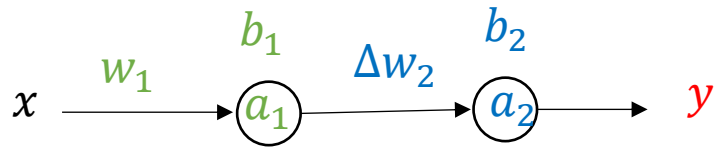
Forward & Back Propagation: 1. single neuron

Exercise:

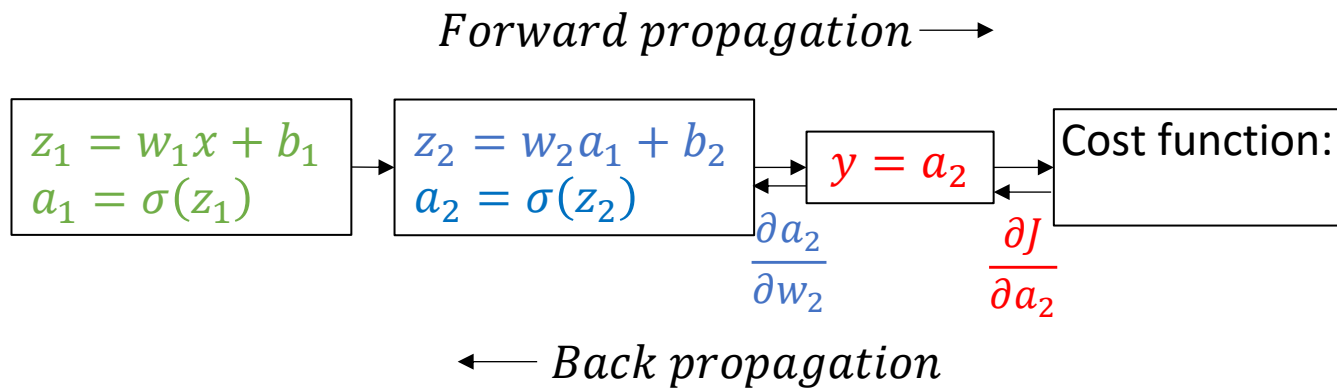
Model: $y = \sigma(w_2 \sigma(w_1 x + b_1) + b_2)$



∇J vary with
 w_1, w_2, b_1, b_2



How important is w_2 for changing the cost function J ?

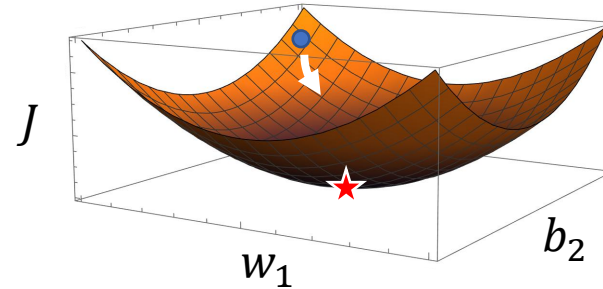


$$\begin{aligned} \frac{\partial J}{\partial w_2} &= \frac{\partial J}{\partial a_2} \frac{\partial a_2}{\partial w_2} \\ &= 2(y - y_d) \sigma'(z_2) a_1 \end{aligned}$$

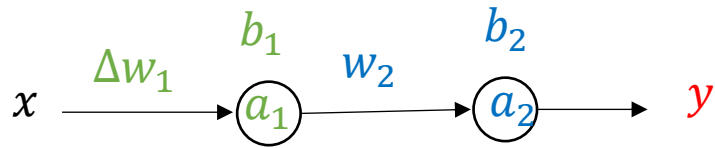
Forward & Back Propagation: 1. single neuron

Exercise:

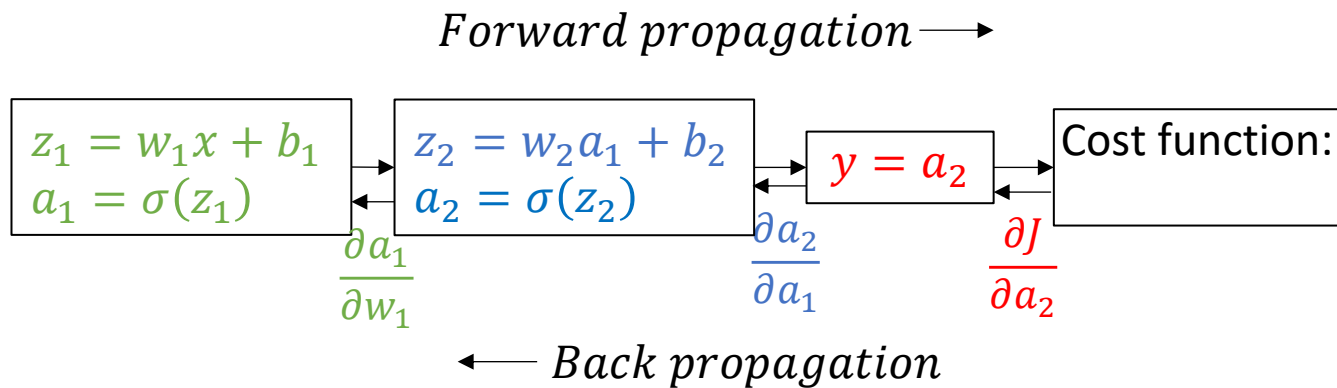
Model: $y = \sigma(w_2 \sigma(w_1 x + b_1) + b_2)$



∇J vary with
 w_1, w_2, b_1, b_2



How important is w_1 for changing the cost function J ?



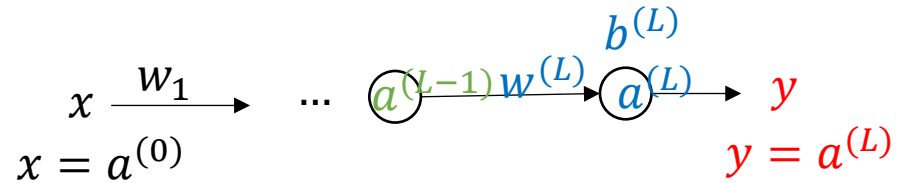
$$\begin{aligned} \frac{\partial J}{\partial w_1} &= \frac{\partial J}{\partial a_2} \frac{\partial a_2}{\partial a_1} \frac{\partial a_1}{\partial w_1} \\ &= 2(y - y_d) \sigma'(z_2) w_2 \sigma'(z_1) x \end{aligned}$$

When training a neural network

Repeat these steps:

1. Forward propagate an input
2. Compute the cost function
3. Compute the gradients of the cost with respect to parameters using backpropagation
4. Update each parameter using the gradients, according to the optimization algorithm

Forward & Back Propagation: 1. single neuron



Back propagation (use chain rules)

Forward propagation

Input: $a^{(0)} = x$

...

$$z^{(L-1)} = w^{(L-1)} a^{(L-2)} + b^{(L-1)}$$

$$a^{(L-1)} = \sigma(z^{(L-1)})$$

$$z^{(L)} = w^{(L)} a^{(L-1)} + b^{(L)}$$

$$a^{(L)} = \sigma(z^{(L)})$$

Output: $y = a^{(L)}$

Cost function $J = (y - y_d)^2$

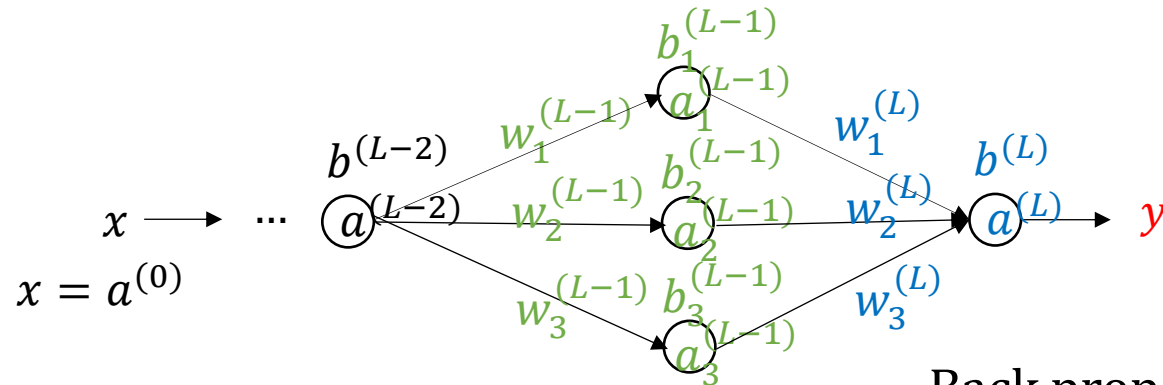
$$\frac{\partial J}{\partial w^{(L)}} = \frac{\partial J}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial w^{(L)}} = 2(a^{(L)} - y_d) \sigma'(z^{(L)}) a^{(L-1)}$$

$$\frac{\partial J}{\partial b^{(L)}} = \frac{\partial J}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial b^{(L)}} = 2(a^{(L)} - y_d) \sigma'(z^{(L)})$$

$$\begin{aligned} \frac{\partial J}{\partial w^{(L-1)}} &= \frac{\partial J}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial a^{(L-1)}} \frac{\partial a^{(L-1)}}{\partial w^{(L-1)}} \\ &= 2(a^{(L)} - y_d) \sigma'(z^{(L)}) w^{(L)} \sigma'(z^{(L-1)}) a^{(L-2)} \end{aligned}$$

$$\begin{aligned} \frac{\partial J}{\partial b^{(L-1)}} &= \frac{\partial J}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial a^{(L-1)}} \frac{\partial a^{(L-1)}}{\partial b^{(L-1)}} \\ &= 2(a^{(L)} - y_d) \sigma'(z^{(L)}) w^{(L)} \sigma'(z^{(L-1)}) \end{aligned}$$

Forward & Back Propagation: 2. multiple hidden units



Back propagation (use chain rules)

Forward propagation

Input: $a^{(0)} = x$

...

$$z_k^{(L-1)} = w_k^{(L-1)} a^{(L-2)} + b_k^{(L-1)}$$

$$a_k^{(L-1)} = \sigma(z_k^{(L-1)})$$

$$z^{(L)} = \sum_{k=1}^3 w_k^{(L)} a_k^{(L-1)} + b^{(L)}$$

$$a^{(L)} = \sigma(z^{(L)})$$

Output: $y = a^{(L)}$

Cost function $J = (y - y_d)^2$

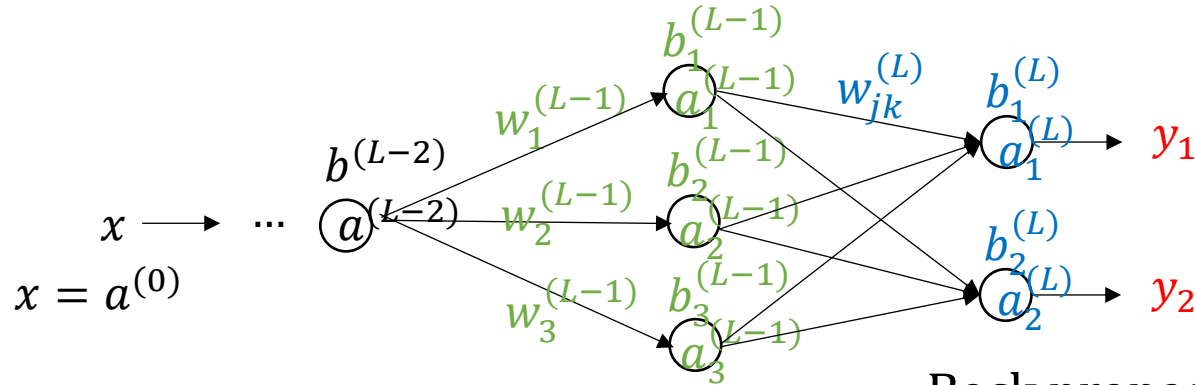
$$\frac{\partial J}{\partial w_k^{(L)}} = \frac{\partial J}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial w_k^{(L)}} = 2(a^{(L)} - y_d) \sigma'(z^{(L)}) a_k^{(L-1)}$$

$$\frac{\partial J}{\partial b^{(L)}} = \frac{\partial J}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial b^{(L)}} = 2(a^{(L)} - y_d) \sigma'(z^{(L)})$$

$$\begin{aligned} \frac{\partial J}{\partial w_k^{(L-1)}} &= \frac{\partial J}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial a_k^{(L-1)}} \frac{\partial a_k^{(L-1)}}{\partial w_k^{(L-1)}} \\ &= 2(a^{(L)} - y_d) \sigma'(z^{(L)}) w_k^{(L)} \sigma'(z_k^{(L-1)}) a^{(L-2)} \end{aligned}$$

$$\begin{aligned} \frac{\partial J}{\partial b_k^{(L-1)}} &= \frac{\partial J}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial a_k^{(L-1)}} \frac{\partial a_k^{(L-1)}}{\partial b_k^{(L-1)}} \\ &= 2(a^{(L)} - y_d) \sigma'(z^{(L)}) w_k^{(L)} \sigma'(z_k^{(L-1)}) \end{aligned}$$

Forward & Back Propagation: 3. multiple outputs



Back propagation (use chain rules)

Forward propagation

Input: $a^{(0)} = x$

...

$$z_k^{(L-1)} = w_k^{(L-1)} a^{(L-2)} + b_k^{(L-1)}$$

$$a_k^{(L-1)} = \sigma(z_k^{(L-1)})$$

$$z_j^{(L)} = \sum_{k=1}^3 w_{jk}^{(L)} a_k^{(L-1)} + b_j^{(L)}$$

$$a_j^{(L)} = \sigma(z_j^{(L)})$$

Output: $y_j = a_j^{(L)}$

Cost function $J = \sum_{j=1}^2 (y_j - y_{d,j})^2$

$$\frac{\partial J}{\partial w_{jk}^{(L)}} = \frac{\partial J}{\partial a_j^{(L)}} \frac{\partial a_j^{(L)}}{\partial w_{jk}^{(L)}} = 2(a_j^{(L)} - y_{d,j}) \sigma'(z_j^{(L)}) a_k^{(L-1)}$$

$$\frac{\partial J}{\partial b_j^{(L)}} = \frac{\partial J}{\partial a_j^{(L)}} \frac{\partial a_j^{(L)}}{\partial b_j^{(L)}} = 2(a_j^{(L)} - y_{d,j}) \sigma'(z_j^{(L)})$$

$$\begin{aligned} \frac{\partial J}{\partial w_k^{(L-1)}} &= \sum_{j=1}^2 \frac{\partial J}{\partial a_j^{(L)}} \frac{\partial a_j^{(L)}}{\partial a_k^{(L-1)}} \frac{\partial a_k^{(L-1)}}{\partial w_k^{(L-1)}} \\ &= \sum_{j=1}^2 2(a_j^{(L)} - y_{d,j}) \sigma'(z_j^{(L)}) w_{jk}^{(L)} \sigma'(z_k^{(L-1)}) a^{(L-2)} \end{aligned}$$

$$\begin{aligned} \frac{\partial J}{\partial b_k^{(L-1)}} &= \sum_{j=1}^2 \frac{\partial J}{\partial a_j^{(L)}} \frac{\partial a_j^{(L)}}{\partial a_k^{(L-1)}} \frac{\partial a_k^{(L-1)}}{\partial b_k^{(L-1)}} \\ &= \sum_{j=1}^2 2(a_j^{(L)} - y_{d,j}) \sigma'(z_j^{(L)}) w_{jk}^{(L)} \sigma'(z_k^{(L-1)}) \end{aligned}$$

Summary

- Universal function approximator
- Gradient descent: a method to find weights and biases that minimize J
- Calculate ∇J
 - One example
 - A full dataset
- Forward and backpropagation (a clever way to get gradients of J)

Outline

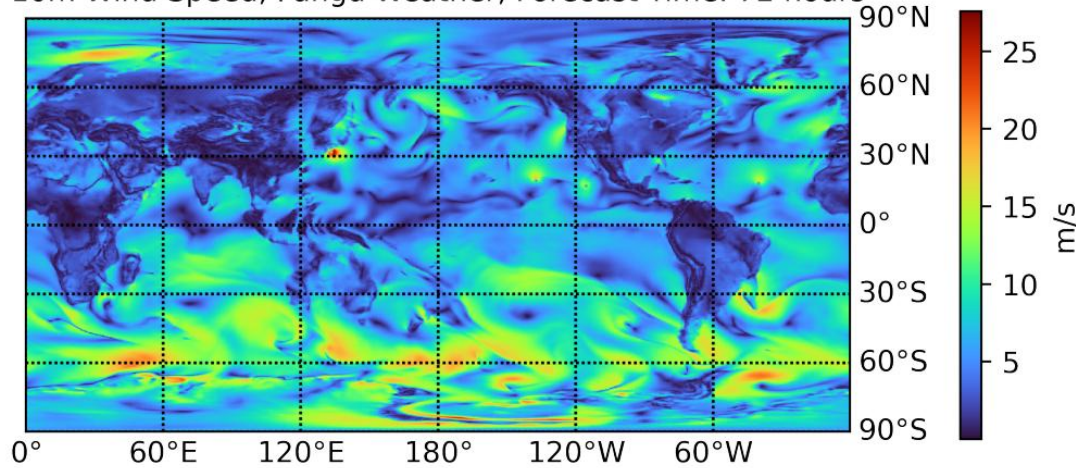
- Why neural networks?
- NN basics:
 - Universal function approximation
 - Gradient descent
 - Back propagation
- **Advanced topics: challenges of using NN in scientific problems**

Some challenges:

AI weather forecast

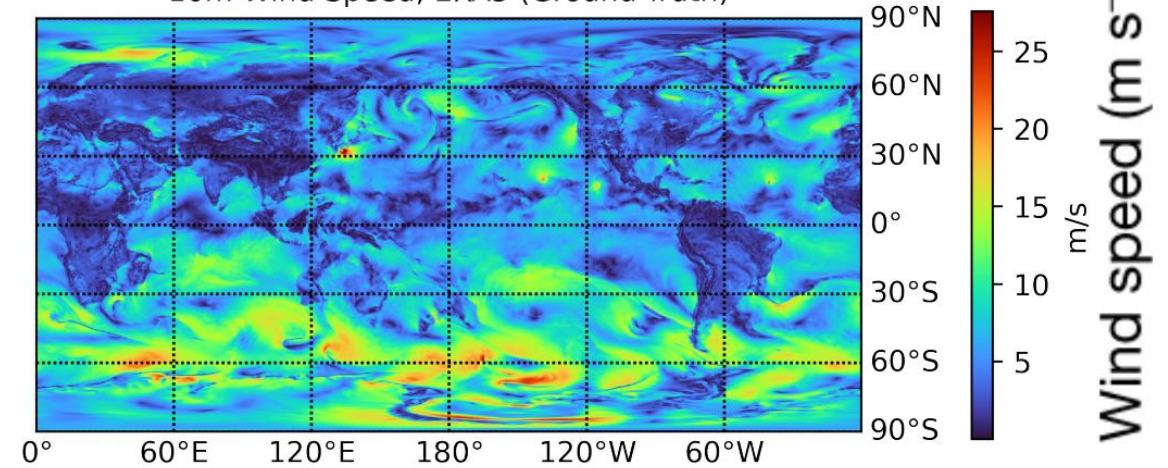
Bi et al (2023) Nature

10m Wind Speed, Pangu-Weather, Forecast Time: 72 hours



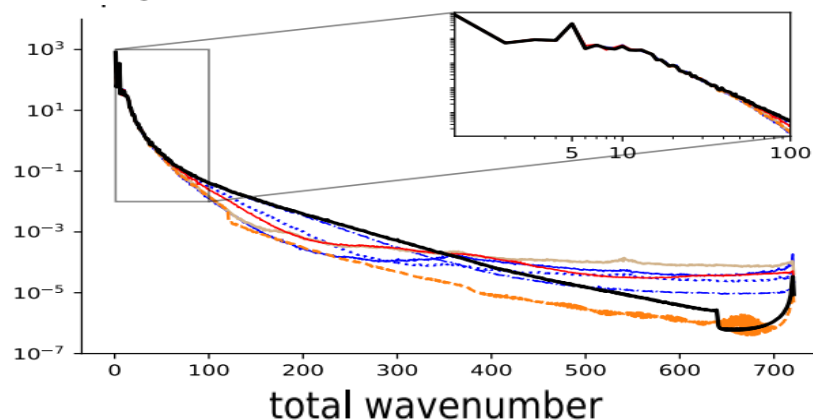
Training data (Re-analysis)

10m Wind Speed, ERA5 (Ground-Truth)



- Spectral bias of NN:

spectrum of u250 at Δt



spectrum of u250 at day 10

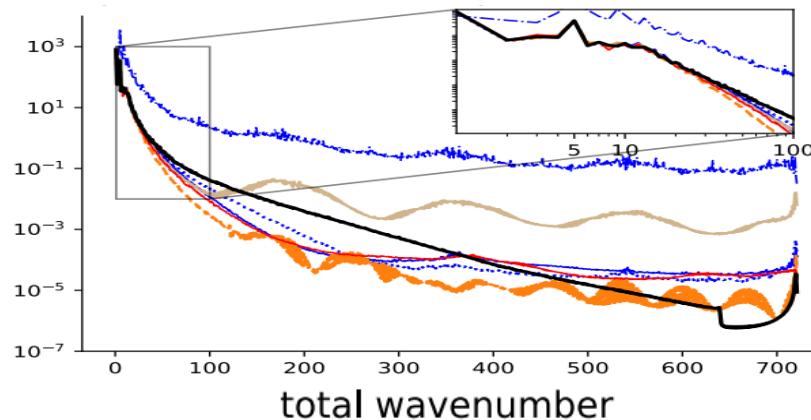


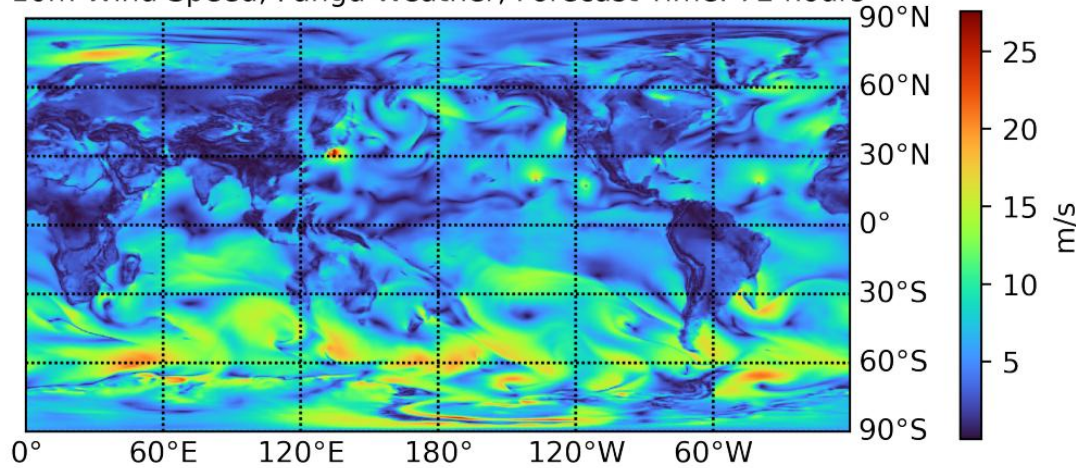
Image source: Qiang Sun & Pedram Hassanzadeh

Spectral bias is bad for multiscale problems

AI weather forecast

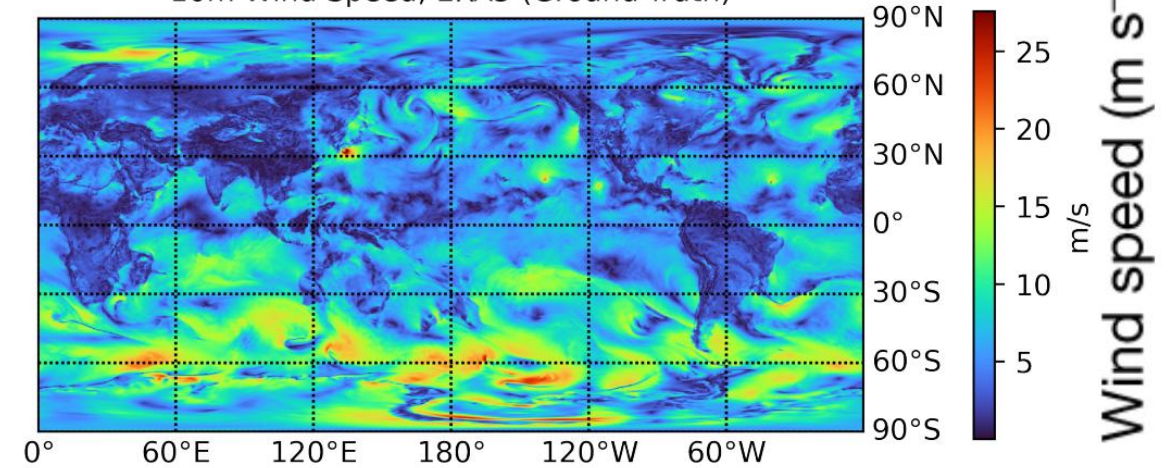
Bi et al (2023) Nature

10m Wind Speed, Pangu-Weather, Forecast Time: 72 hours

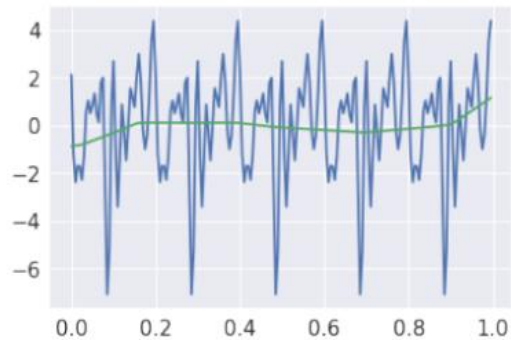


Training data (Re-analysis)

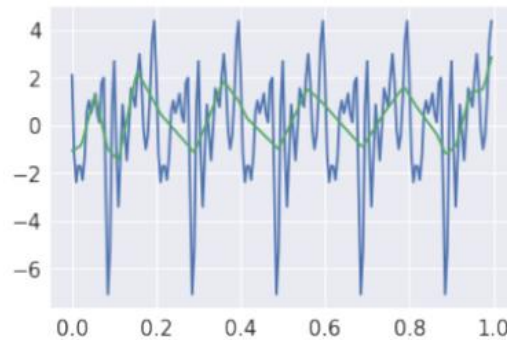
10m Wind Speed, ERA5 (Ground-Truth)



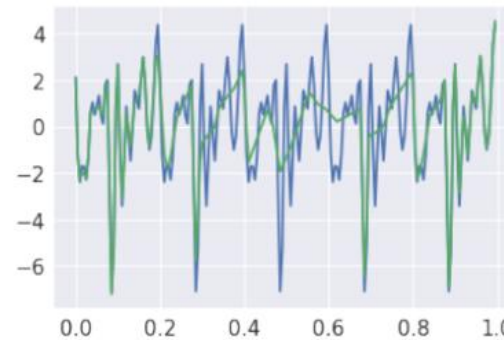
- **Spectral bias of NN:** Rahaman et al (2019), Frequency Principle: Xu et al (2020)



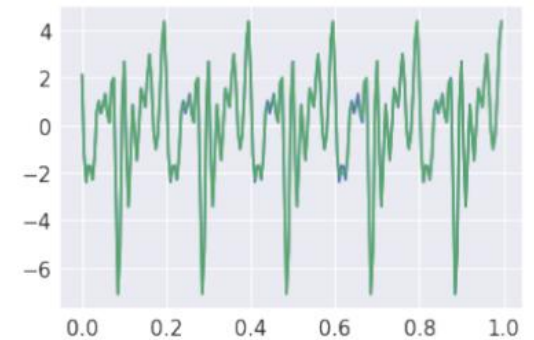
(a) Iteration 100



(b) Iteration 1000



(c) Iteration 10000

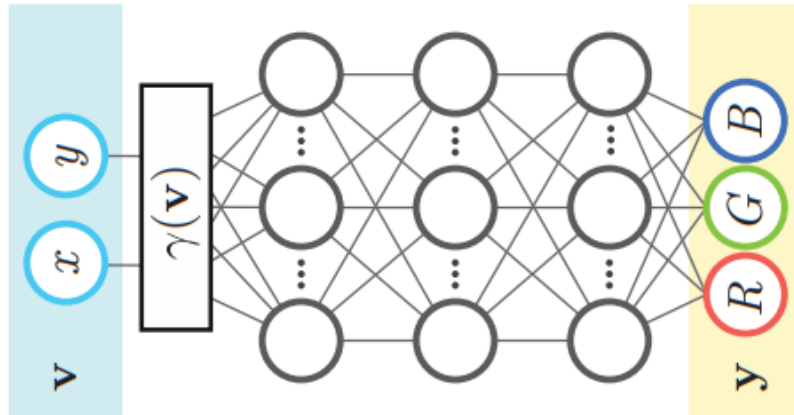


(d) Iteration 80000

Spectral bias is bad for multiscale problems

- One solution: Fourier feature $\gamma(\mathbf{v})$ Tancik et al (2020), Wang et al (2021)
 $\gamma(\mathbf{v})$ maps the input coordinates to a feature space before passing them to NN layers.

$$\gamma(\mathbf{v}) = [a_1 \cos(2\pi \mathbf{b}_1^T \mathbf{v}), a_1 \sin(2\pi \mathbf{b}_1^T \mathbf{v}), \dots, a_m \cos(2\pi \mathbf{b}_m^T \mathbf{v}), a_m \sin(2\pi \mathbf{b}_m^T \mathbf{v})]^T$$

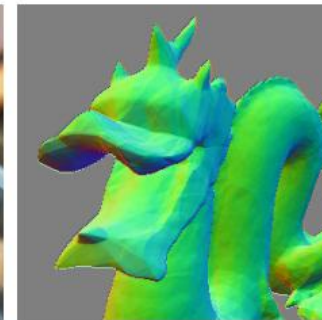


No Fourier features
 $\gamma(\mathbf{v}) = \mathbf{v}$

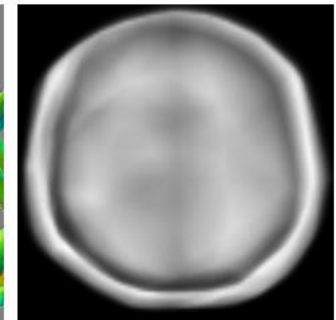
With Fourier features
 $\gamma(\mathbf{v}) = \text{FF}(\mathbf{v})$



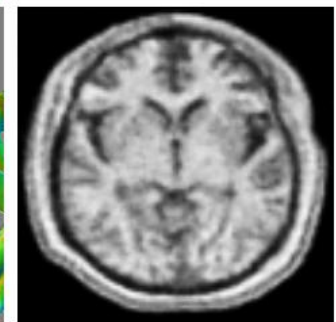
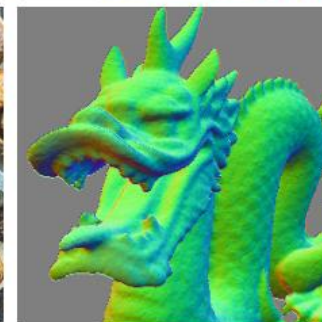
(b) Image regression
 $(x, y) \rightarrow \text{RGB}$



(c) 3D shape regression
 $(x, y, z) \rightarrow \text{occupancy}$



(d) MRI reconstruction
 $(x, y, z) \rightarrow \text{density}$



Spectral bias: A Fourier analysis of NN Xu et al. (2022)

For a single-hidden-layer NN with input $x \in \mathbb{R}$ and output $u \in \mathbb{R}$

$$u(x) = \sum_{j=1}^m w_j^{(1)} \sigma(w_j^{(0)} x + b_j), \quad \text{where } \theta_j = \{w_j^{(0)}, w_j^{(1)}, b_j\}$$

The mean square loss measures the distance between the NN output $u(x)$ and ground truth $u_g(x)$

$$J \equiv \int_{-\infty}^{\infty} \frac{1}{2} (u(x) - u_g(x))^2 dx = \int_{-\infty}^{\infty} \frac{1}{2} |\hat{u}(k) - \hat{u}_g(k)|^2 dk, \quad \text{where } \hat{u}(k) \text{ denotes the Fourier transform of } u(x)$$

The loss at frequency k is $J(k) = \frac{1}{2} |\hat{u} - \hat{u}_g|^2$

For tanh activation function, the gradient of loss at $J(k)$ with respect to NN parameters θ_j is

$$\left| \frac{\partial J(k)}{\partial \theta_j} \right| \approx A(k) e^{-\left| \frac{\pi k}{2w_j^{(0)}} \right|}, \quad \text{where } A(k) \in [0, +\infty) \text{ is the amplitude of } \hat{u} - \hat{u}_g$$

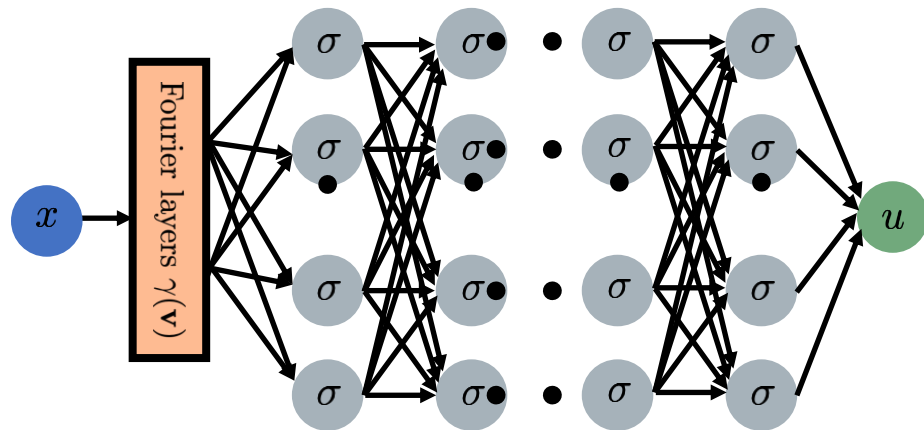
This gradient decay rapidly with frequency k !! This is a simple explanation of the spectral bias.

Capturing high-frequency functions

- For multiscale problems, it is important to rescale the NN weights so that it activates the learning of high-frequency (large k) functions

$$\left| \frac{\partial J(k)}{\partial \theta_j} \right| \approx A(k) e^{-\left| \frac{\pi k}{2w_j^{(0)}} \right|} \quad \text{We make } w_j^{(0)} \propto k$$

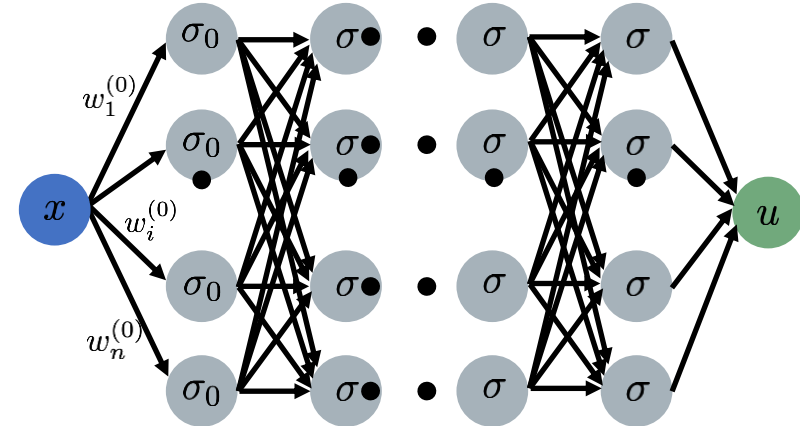
Fourier Feature nets Tancik *et. al.* (2020)



$$\Phi(\mathbf{x}) = \begin{bmatrix} \cos(\mathbf{B}\mathbf{x}) \\ \sin(\mathbf{B}\mathbf{x}) \end{bmatrix}, \quad \mathbf{B} \sim \mathcal{N}(0, \kappa^2)$$

Gaussian distribution with standard deviation κ

Our implementation: Wang & Lai, J. Comput. Phys. (2024)



$$\text{First layer: } \sigma_0(x) = \sin\left(\kappa w_i^{(0)} x + b_i^{(0)}\right)$$

κ : scaling factor

Main idea

Wang & Lai, J. Comput. Phys. (2024)

- When NN training error is small, the learning becomes really slow. NN learns better when $u \approx O(1)$. Inspired by the principals of perturbation theory, MSNN includes a superposition of multi-stage NNs, with each stage using a new NN to fit the residue rescaled to $O(1)$ from the previous NN.

$$u_c^{(n)}(x) = u_0(x) + \epsilon_1 u_1(x) + \epsilon_2 u_2(x) + \dots + \epsilon_n u_n(x) = \sum_{j=0}^n \epsilon_j u_j(x)$$

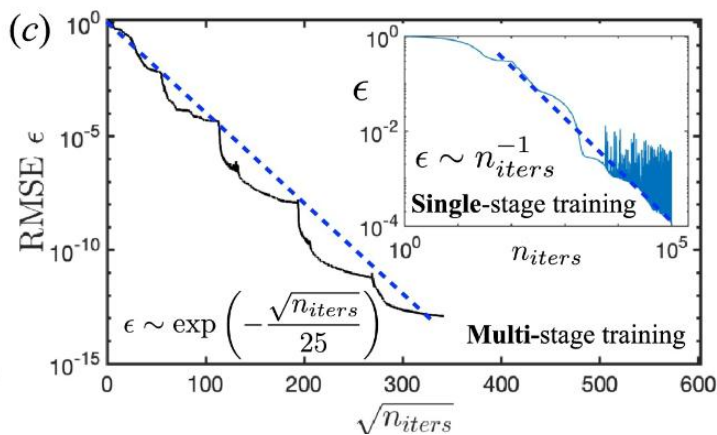
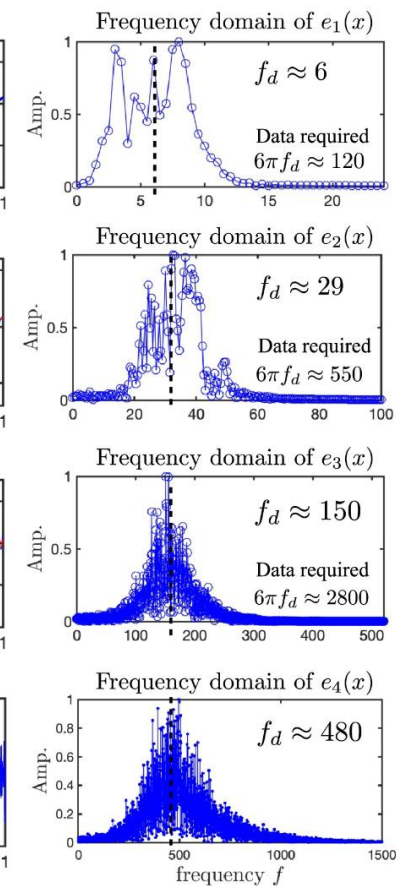
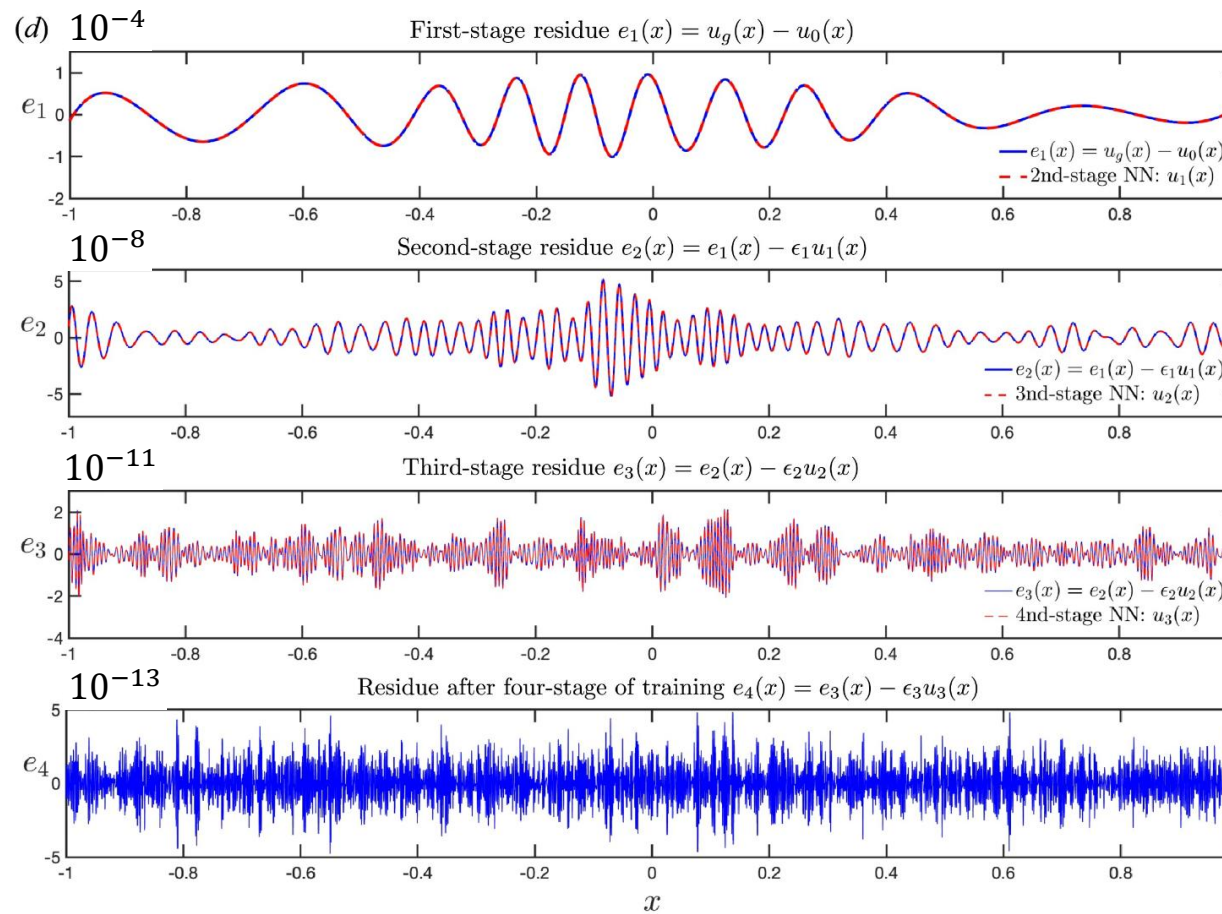
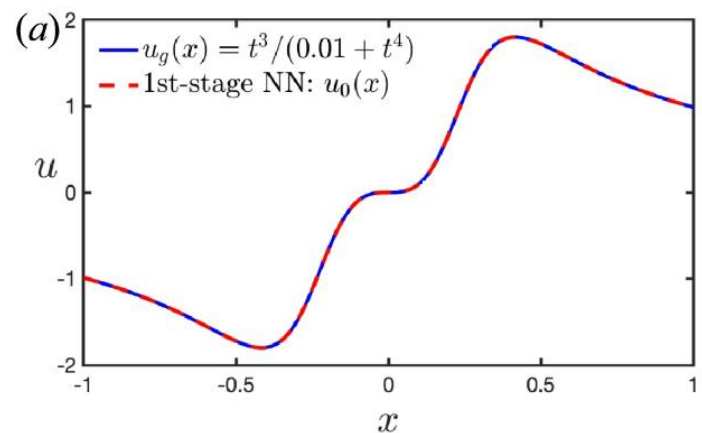
where $1 \gg \epsilon_1 \gg \epsilon_2 \gg \dots \gg \epsilon_n$

$u_0(x)$, $u_1(x)$ and $u_n(x)$ are all of order $O(1)$ NNs of different stages

Related work with similar flavors: Multi-level neural networks: [Aldirany et al. \(2023\)](#), Multifidelity neural networks: [Howard et al. \(2023, 2024\)](#), Precision Machine Learning: [Michaud et al. \(2023\)](#)...etc

Multistage-NN (MSNN)

Wang & Lai, J. Comput. Phys. (2024)



Can multistage-NN be used to tackle the spectral bias in multiscale problems?

Example:

How precisely can a NN approximate a 2D fluid flow?

2D incompressible Navier-Stokes eqn:

ω : vorticity

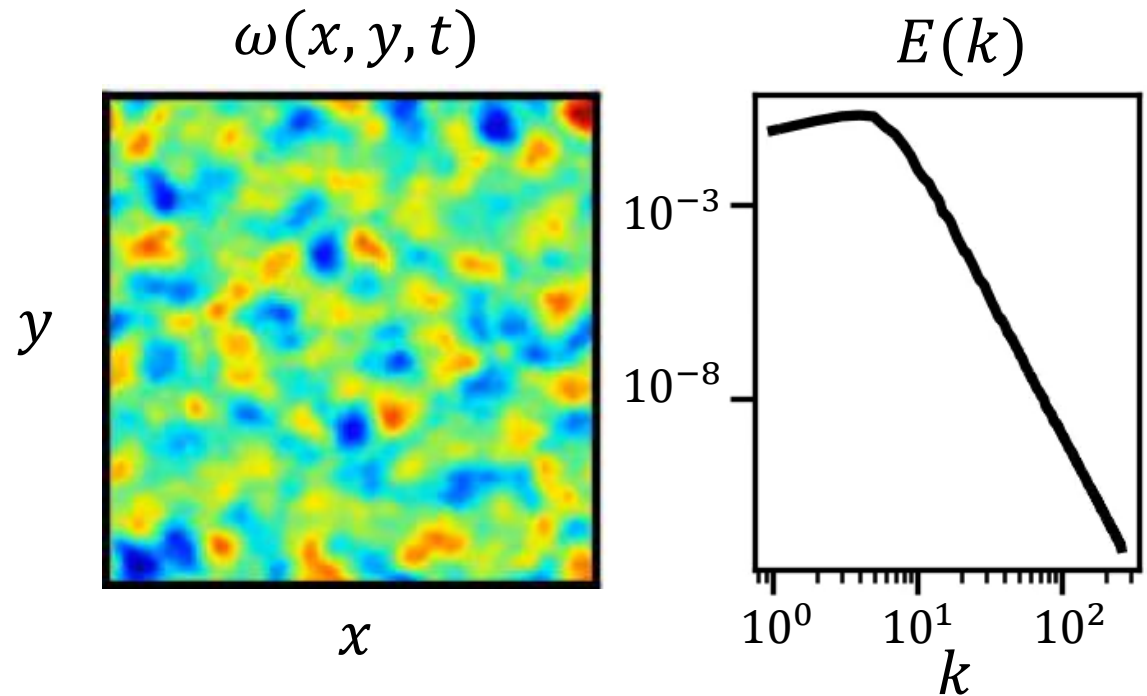
ψ : stream function

$$\frac{\partial \omega}{\partial t} + \frac{\partial \psi}{\partial y} \frac{\partial \omega}{\partial x} - \frac{\partial \psi}{\partial x} \frac{\partial \omega}{\partial y} = \frac{1}{\text{Re}} \nabla^2 \omega$$

$$\nabla^2 \psi = -\omega$$

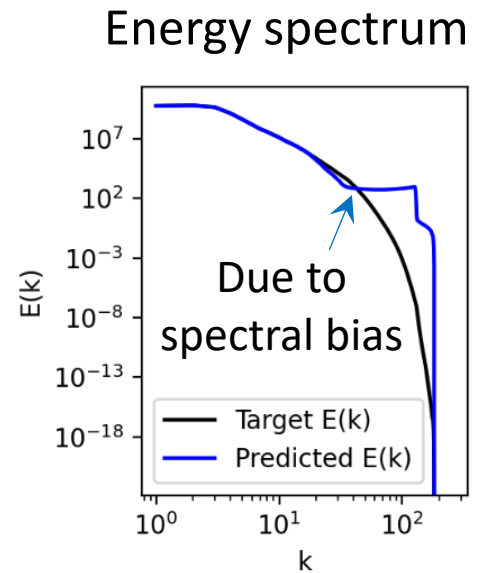
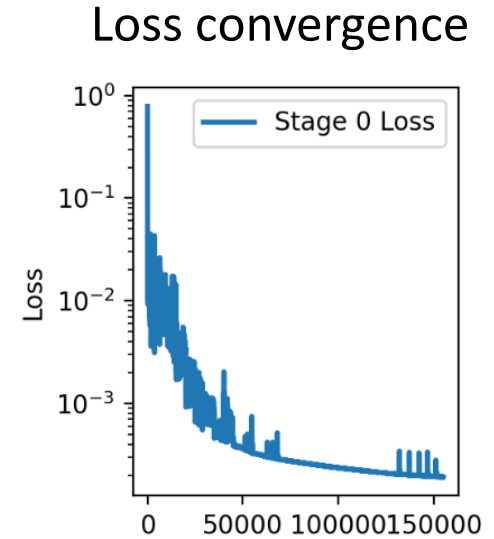
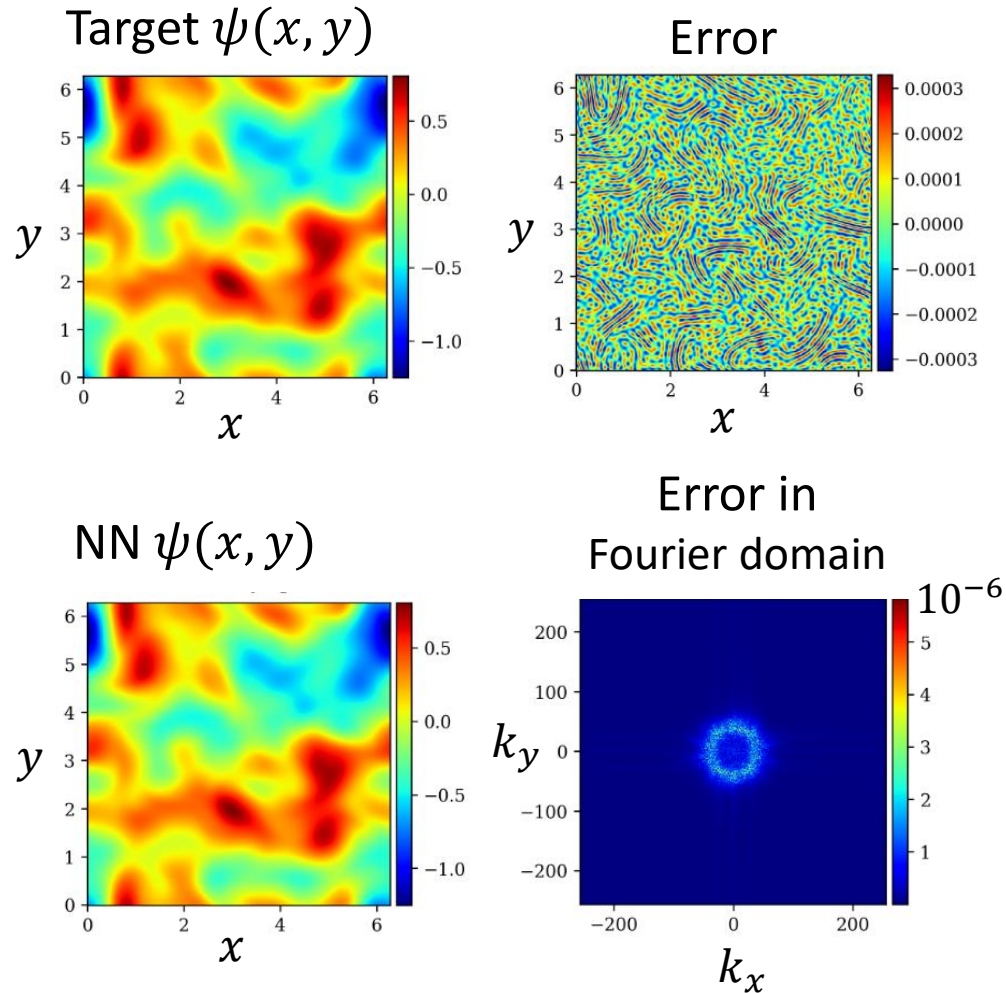
Training data:

Numerical solution at $\text{Re} = 2000$

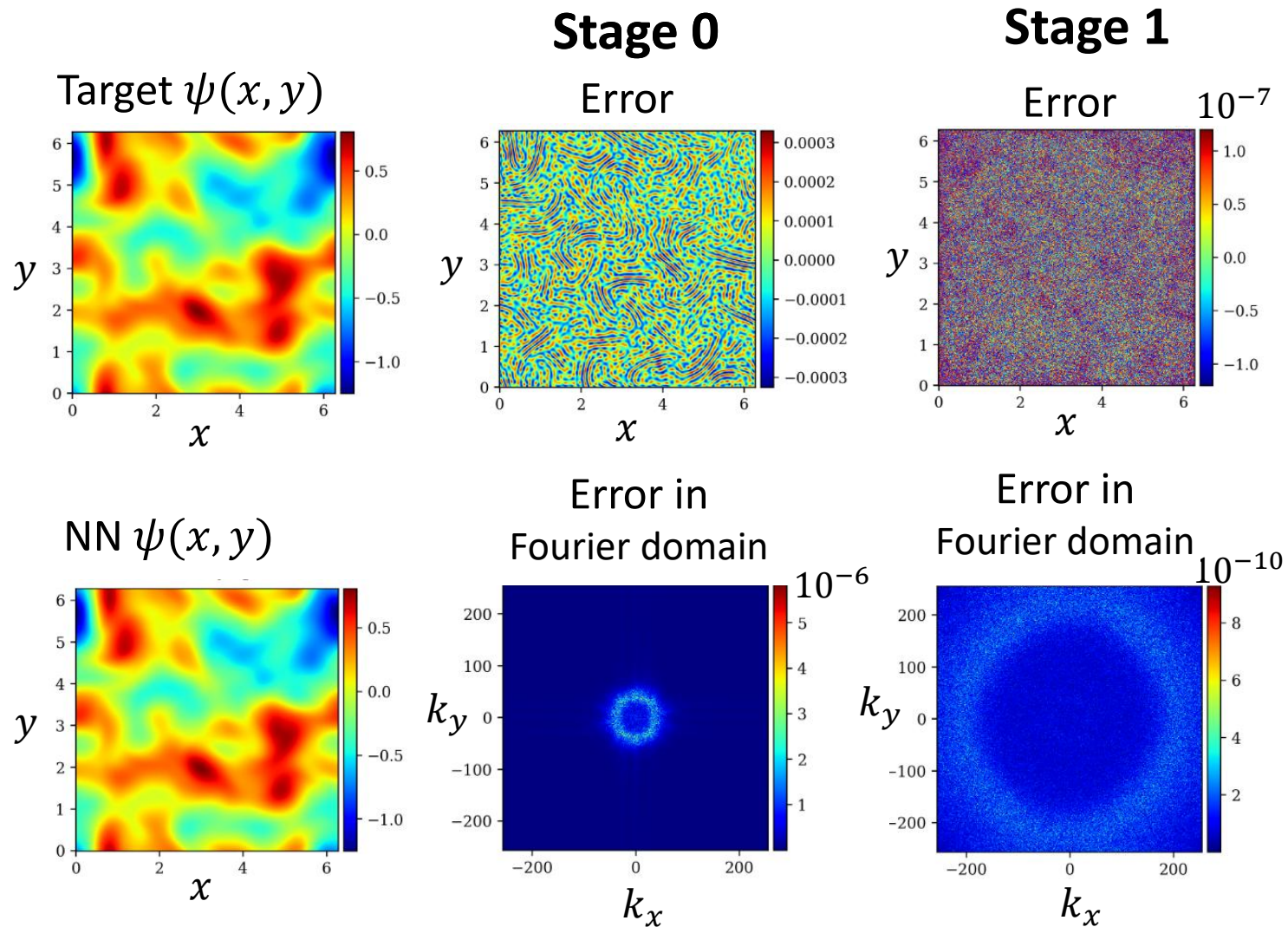


Energy spectrum: $E(k, t) = \sum_{k-\Delta k \leq |\mathbf{k}| \leq k+\Delta k} \frac{1}{2} k^2 |\tilde{\psi}(\mathbf{k}, t)|^2$

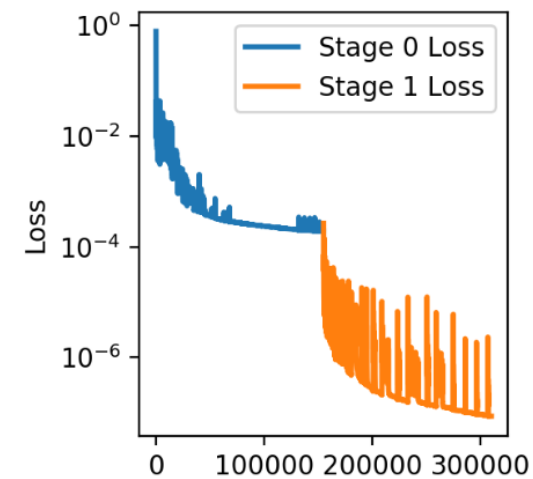
E.g. 2D incompressible Navier-Stokes



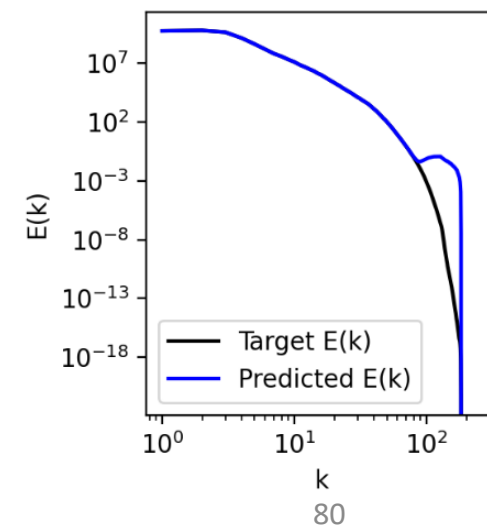
E.g. 2D incompressible Navier-Stokes



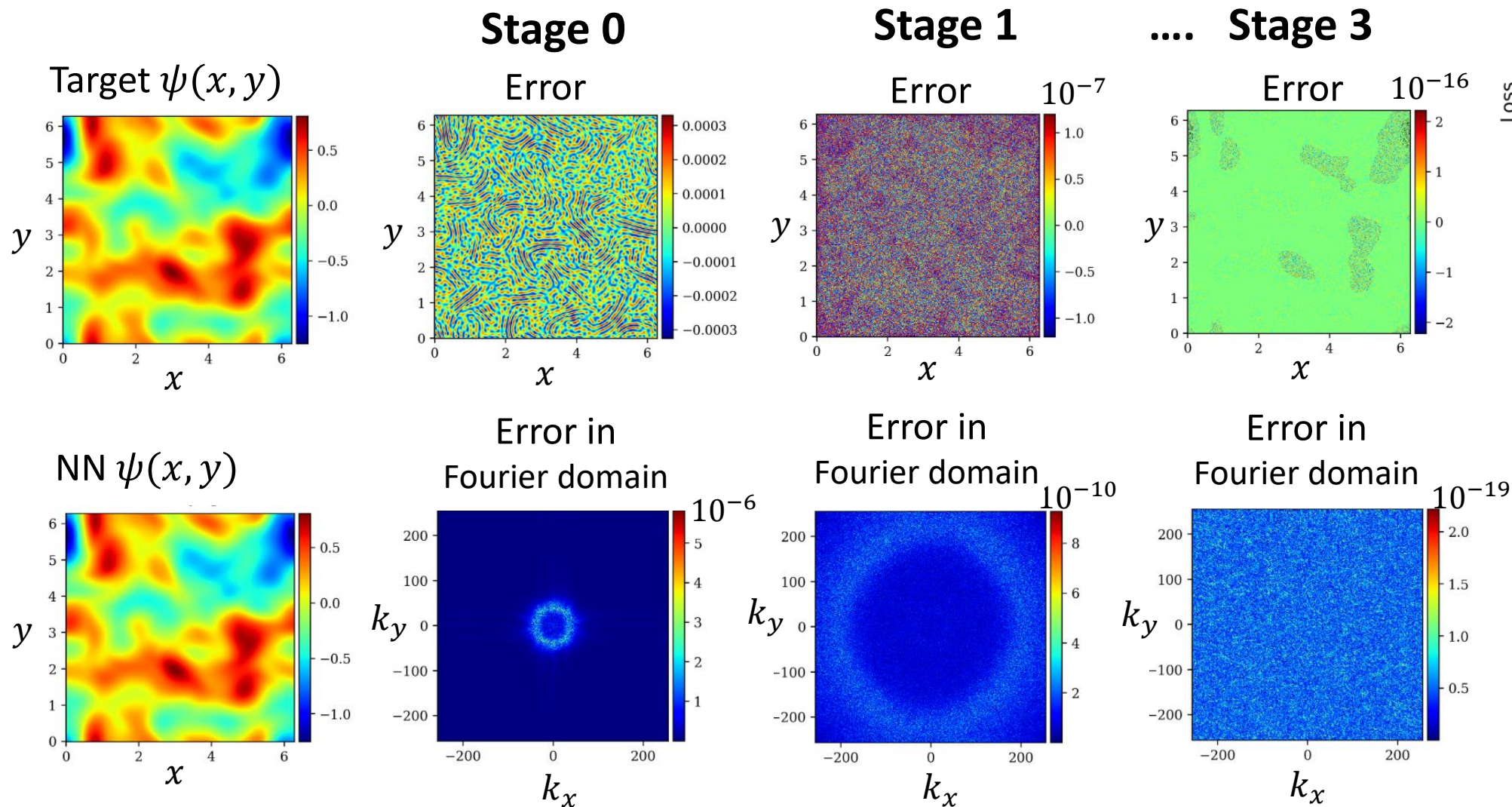
Loss convergence



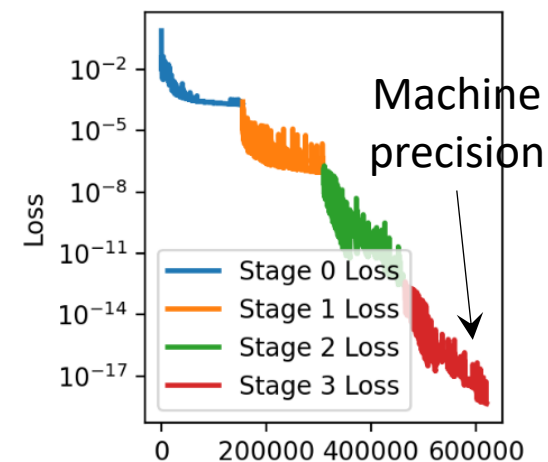
Energy spectrum



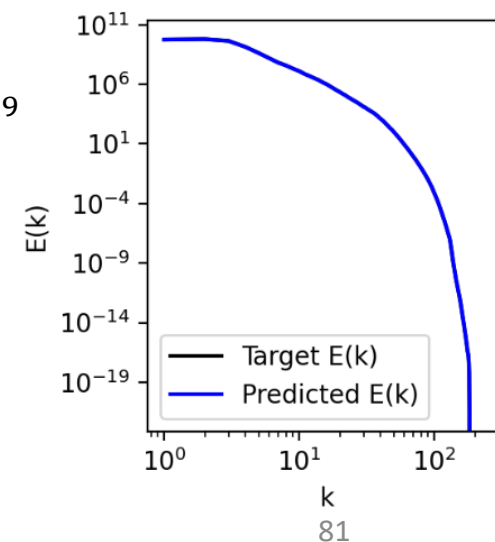
E.g. 2D incompressible Navier-Stokes



Loss convergence



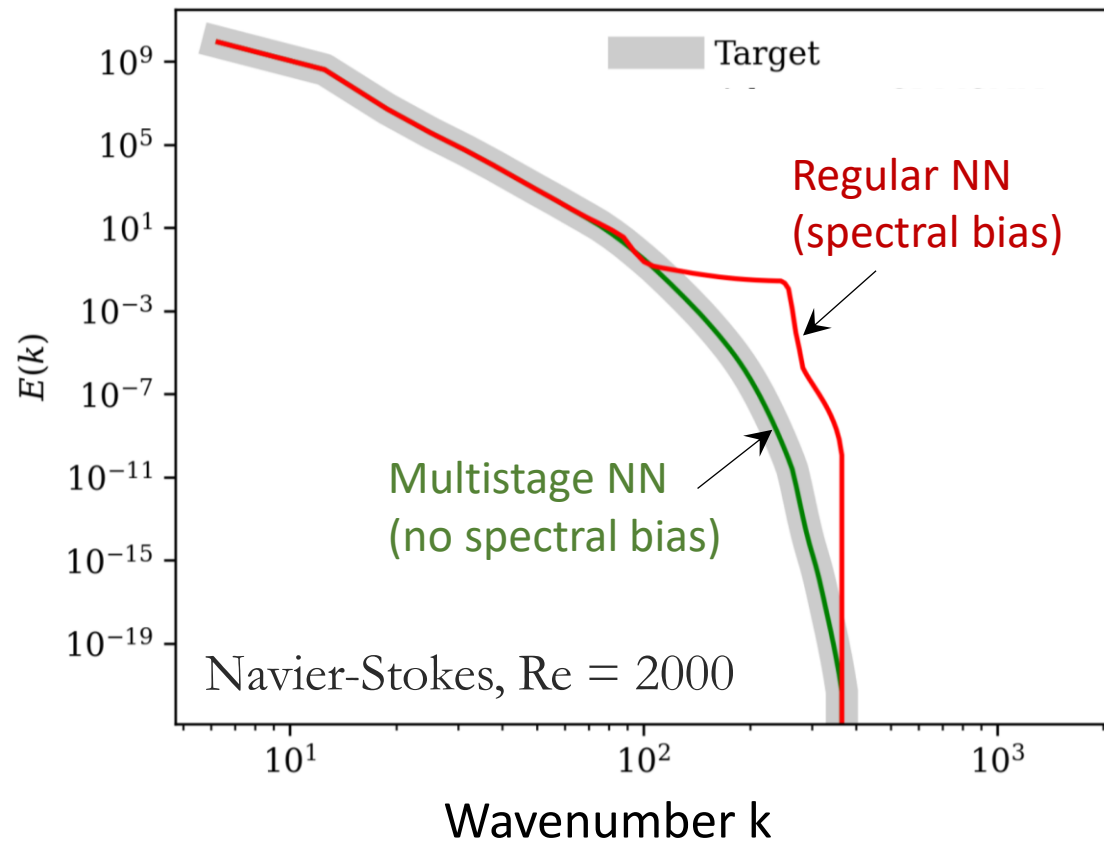
Energy spectrum



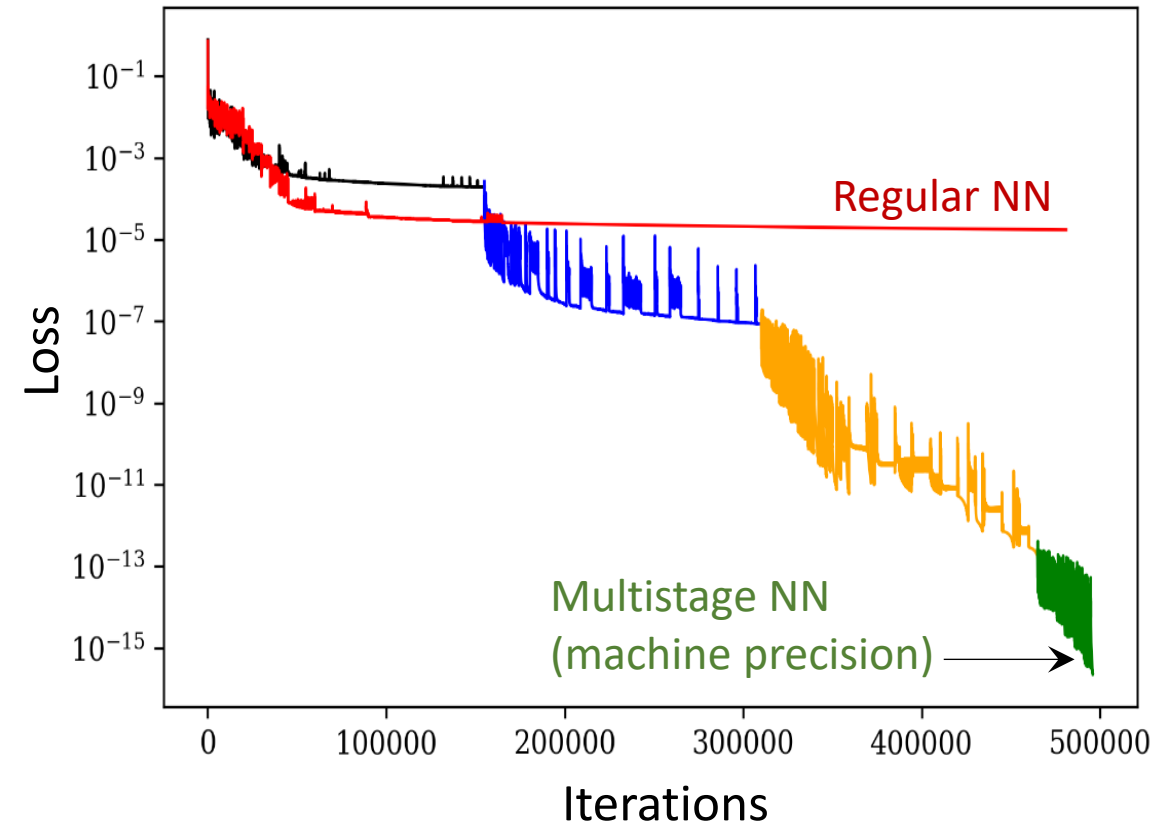
Multistage NN can tackle the spectral bias & accelerate loss convergence

Ng, Wang & Lai, ICML workshop (2024),
arXiv:2407.17213

Energy spectrum

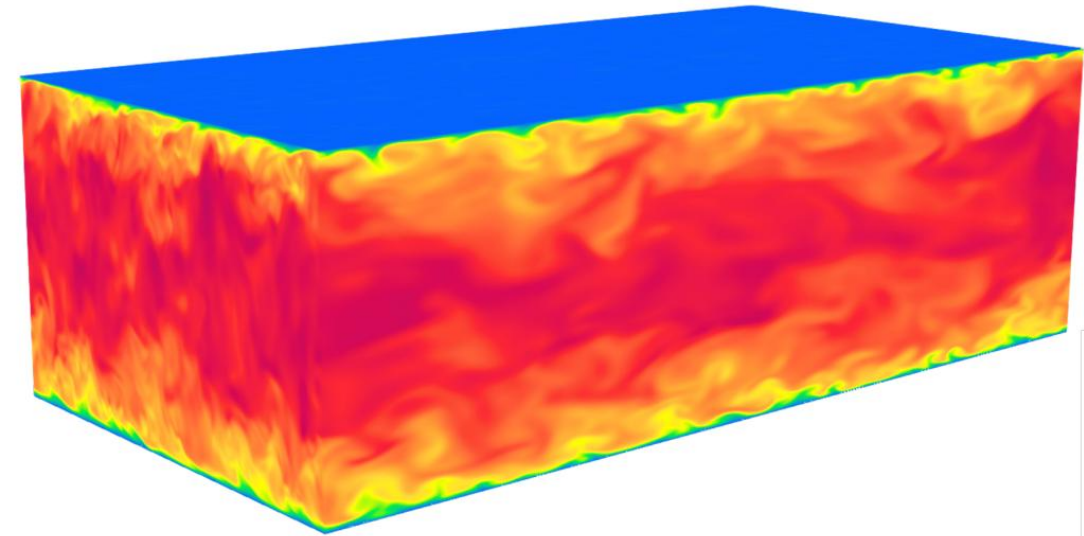
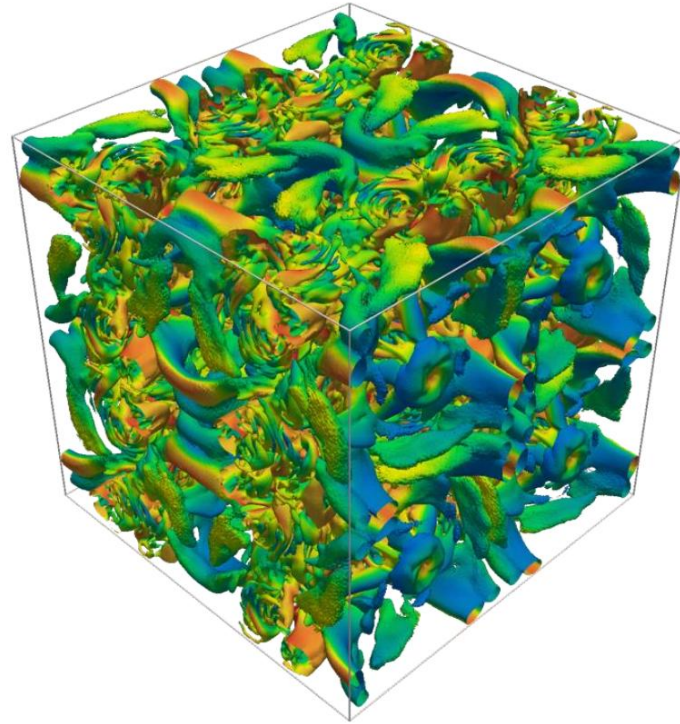
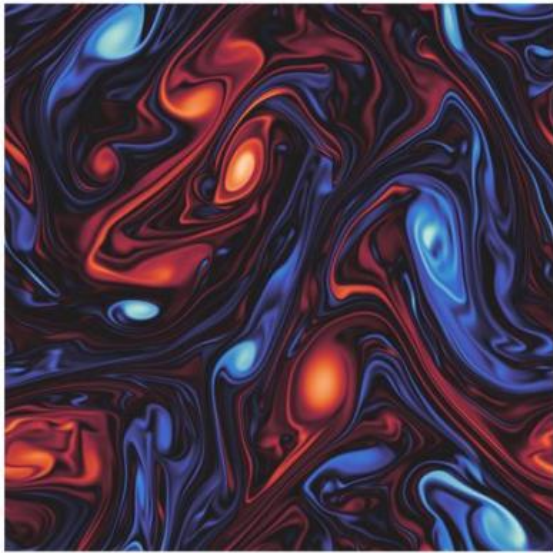


Loss convergence



Capturing high-frequency features

Applications of Fourier Feature to physics-informed NNs:
e.g., Incompressible Navier Stokes equation



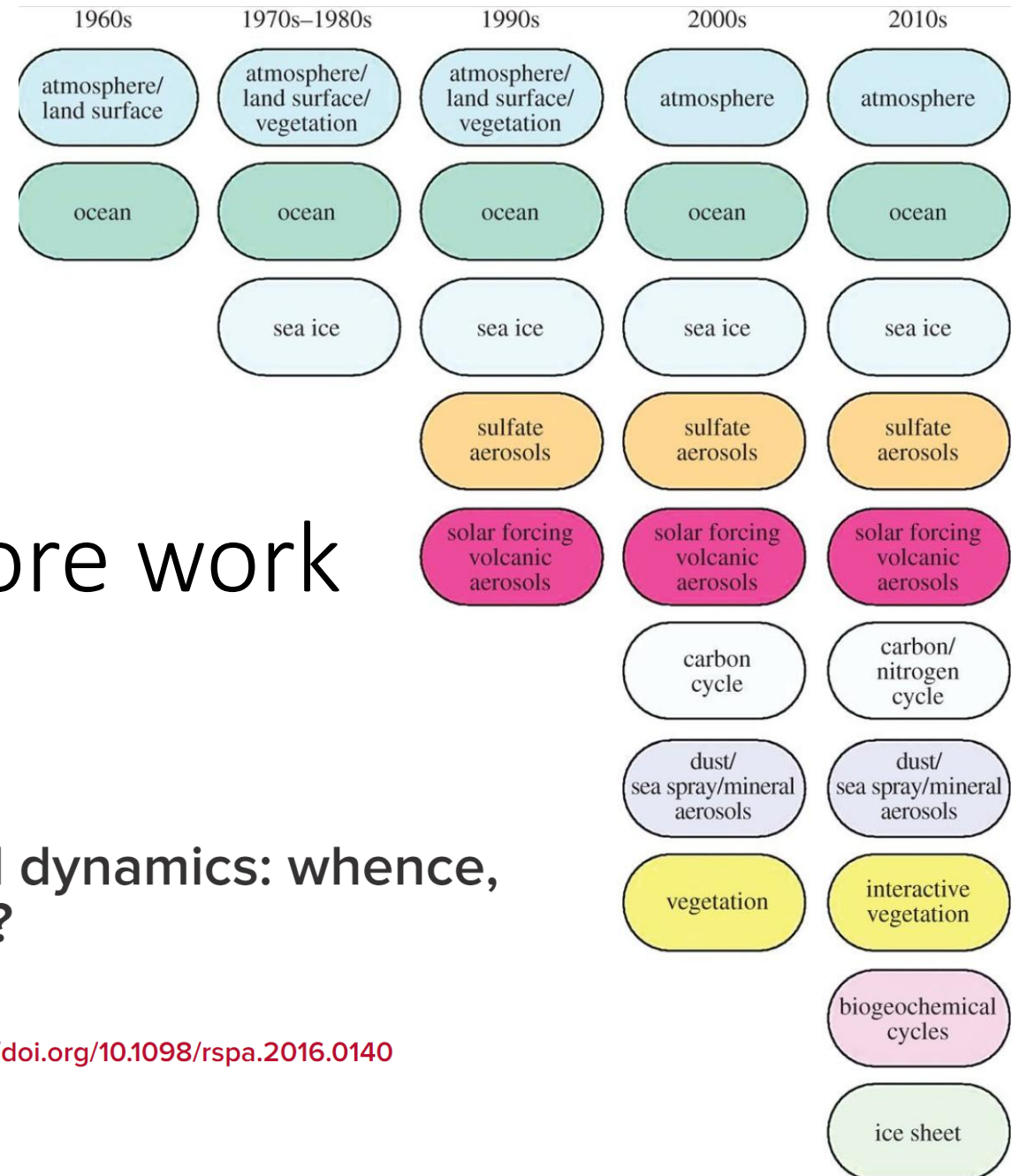
Wang, Sifan, Shyam Sankaran, Xiantao Fan, Panos Stinis, and Paris Perdikaris. "Simulating three-dimensional turbulence with physics-informed neural networks." *arXiv preprint arXiv:2507.08972* (2025).

Random slide: Ice-sheet model needs more work

Geophysical fluid dynamics: whence, whither and why?

Geoffrey K. Vallis ✉

Published: 01 August 2016 | <https://doi.org/10.1098/rspa.2016.0140>



The best way to learn ML is through projects

- Don't worry about failures. In fact, the more the better. This is the only way neural networks learn. This is how we learn!
- Analogy:

